



Carreras de Ingeniería Electrónica e
Ingeniería en Computación

Informe del Trabajo de Graduación

Desarrollo de un dispositivo de seguimiento ocular como herramienta de apoyo académico para personas con movilidad reducida

Autores:
Natalia Carolina Borbón
Facundo Ordóñez Nadra

Directores:
Federico Marcelo Vargas
Ezequiel Alberto Musa

Jurados:
Esteban Daniel Volentini
Martín Gustavo Juárez
Luis Roque Di Pinto

Este trabajo fue realizado en la Ciudad de San Miguel de Tucumán,
entre noviembre de 2025 y octubre de 2026.

Resumen

Este trabajo desarrolló L.E.V.I., un dispositivo de seguimiento ocular de bajo costo, pensado para que estudiantes con limitaciones motrices puedan utilizarlo tanto para rendir evaluaciones de opción múltiple como para comunicarse con su entorno educativo. El dispositivo está compuesto por dos módulos basados en microcontroladores económicos y opera de manera autónoma, ya que genera su propia red inalámbrica sin necesidad de conexión a internet. A través del mismo, los docentes administran los cuestionarios, los tutores realizan el seguimiento académico del alumno y los compañeros se comunican con él.

Su valor principal es ofrecer una alternativa considerablemente más económica que las soluciones comerciales existentes y, a diferencia de estas, está diseñada específicamente para el ámbito académico. Esta propuesta amplía la autonomía y participación de estos estudiantes en las aulas.

En la presente memoria se detallan el proceso de selección del método de detección de la mirada, el diseño y la configuración de la red inalámbrica del sistema, el desarrollo del firmware, de la base de datos y de la interfaz web, el diseño y fabricación de las placas de circuito propias, las mediciones de consumo y autonomía realizadas, las conclusiones y trabajos futuros propuestos. Se aplicaron contenidos de sistemas embebidos, redes, diseño de circuitos impresos, bases de datos y desarrollo web correspondientes a las carreras de Ingeniería electrónica e Ingeniería en computación.

Índice general

Resumen **I**

1. Introducción General **1**
1.1. Introducción . . . 1
1.2. Estado del arte . . . 1
1.3. Motivación . . . 2
1.4. Alcance y objetivos . . . 3

2. Introducción Específica **4**
2.1. Seguimiento Ocular y Visión por Computadora . . . 4
2.1.1. Conceptos generales . . . 4
2.1.2. OpenCV y coincidencia de patrones . . . 5
2.1.3. Redes neuronales artificiales . . . 7
2.2. Sistemas Embebidos y Control Gráfico . . . 8
2.2.1. Microcontroladores y familia ESP32 . . . 8
2.2.2. Interfaces gráficas embebidas y biblioteca LVGL . . . 9
2.3. Redes y Comunicaciones . . . 10
2.3.1. Redes locales Wi-Fi . . . 10
2.3.2. Protocolos de comunicación y transmisión de video . . . 11
2.4. Arquitectura de una aplicación web . . . 12
2.4.1. Modelo Cliente/Servidor . . . 13
2.4.2. Patrón de diseño Modelo-Vista-Controlador (MVC) . . . 13
2.4.3. Frontend . . . 14
2.4.4. Backend . . . 16
2.4.5. Base de datos . . . 17

3. Diseño e implementación **20**
3.1. Requisitos y especificaciones del sistema . . . 20
3.1.1. Requisitos funcionales . . . 20
3.1.2. Requisitos técnicos . . . 21
3.1.3. Restricciones de diseño . . . 22
3.2. Detector ocular . . . 22
3.2.1. Procesamiento local como planteamiento inicial . . . 23
3.2.2. Procesamiento remoto por coordenadas . . . 32
3.2.3. Template Matching como solución adoptada . . . 35
3.3. Arquitectura del sistema . . . 38
3.3.1. Funciones de cada módulo . . . 39
3.3.2. Interacción entre módulos . . . 40
3.3.3. Mapa de memoria . . . 40
3.4. Microcontroladores y periféricos . . . 42
3.4.1. Elección de la familia ESP32 . . . 42
3.4.2. Módulo de captura . . . 43

3.4.3. Módulo central . . . 45
3.5. Red y comunicación . . . 48
3.5.1. Red Wi-Fi local . . . 49
3.5.2. Acceso a la aplicación . . . 50
3.5.3. Transmisión de video . . . 50
3.5.4. Comunicación con el módulo central . . . 51
3.6. Aplicación web . . . 52
3.6.1. Base de datos . . . 53
3.6.2. Backend . . . 57
3.6.3. Frontend . . . 62
3.7. Interfaz de usuario táctil . . . 69
3.7.1. Bibliotecas y controladores . . . 69
3.7.2. Funcionamiento . . . 70
3.7.3. Vistas . . . 70
3.8. Alimentación . . . 73
3.8.1. Requisitos previos . . . 73
3.8.2. Presupuesto de consumo por módulo . . . 73
3.8.3. Alternativas propuestas . . . 74
3.8.4. Detalle de la solución adoptada . . . 75
3.8.5. Circuito de alimentación del *ONX3248G035* . . . 76
3.8.6. Verificaciones operativas finales . . . 77
3.9. Diseño físico . . . 78
3.9.1. Circuitos . . . 78
3.9.2. PCB . . . 79
3.9.3. Montaje . . . 81
3.10. Costo total de desarrollo . . . 82

4. Conclusiones **84**
4.1. Resultados obtenidos . . . 84
4.2. Trabajo futuro . . . 85

Bibliografía **87**

Índice de figuras

1.1. *Tobii Glasses* 2

2.1. Representación gráfica del seguimiento ocular 4
2.2. Imagen ilustrativa del proceso de *Template Matching* 6
2.3. Imagen ilustrativa de una matriz de resultados de *Template Matching* 6
2.4. El proceso del *Machine Learning* 7
2.5. Diagrama simplificado de una red neuronal 7
2.6. Ejemplos de placas de desarrollo de la familia *ESP32* 9
2.7. Placa de desarrollo *ESP32-CYD* (vista frontal y trasera) 10
2.8. Red Wi-Fi con un microcontrolador *ESP32* como punto de acceso 11
2.9. Modelo TCP/IP con los protocolos utilizados en el sistema 12
2.10. Modelo *Cliente/Servidor* 13
2.11. Patrón de diseño Modelo-Vista-Controlador (MVC) 14
2.12. Tecnologías del *frontend*: HTML (estructura), CSS (apariencia) y *JavaScript* (comportamiento) 15
2.13. Arquitectura en capas del *backend* y su correspondencia con el patrón MVC . . 16
2.14. Almacenamiento de contraseñas mediante *hash* con *salt* 17
2.15. Elementos del modelo relacional 18

3.1. Diferencias entre procesamiento local y remoto 23
3.2. Distintas partes del ojo y su nivel de oscuridad 24
3.3. Detección por umbral de oscuridad 25
3.4. Posición del *glint* respecto de la pupila 26
3.5. Pantalla de creación de modelos de *Edge Impulse* 26
3.6. Reconocimiento de vegetales con *Edge Impulse* 27
3.7. Tiempo de inferencia estimado por *Edge Impulse* para el *ESP32* clásico 28
3.8. Entorno de *Google Colab* 29
3.9. Una muestra de las imágenes utilizadas para el *dataset* 30
3.10. Planilla con etiquetado de imágenes 30
3.11. Resultados del modelo desarrollado en *Google Colab*. Con un punto verde se indica el valor etiquetado, mientras que la cruz roja muestra la posición que la red predice 31
3.12. Error de precisión con *TFLite* (15,9 píxeles) 32
3.13. Detección de rostro con *MediaPipe* 33
3.14. Facciones del rostro en *WebGazer* 33
3.15. Detección del iris con el método de círculos de Hough 35
3.16. Plantillas para comparación 36
3.17. Imagen de 128x128 píxeles con un recuadro de 96x96 37
3.18. Video en vivo con el umbral de similitud 38
3.19. Diagrama de bloques del sistema definitivo 38
3.20. Mapa de memoria del módulo central 41
3.21. Placa *ESP32-CAM* con su cámara. 43
3.22. Cámaras *OV2640* con distintos ángulos de visión y longitud de conector 44

3.23. Conexión entre el *ESP32* y la cámara *OV2640* en la placa *ESP32-CAM* 45
3.24. Placa *ONX3248G035* (vista frontal y trasera) 46
3.25. Conexiones de la placa *ONX3248G035* 48
3.26. Red Wi-Fi local del sistema 49
3.27. Transmisión de video desde el módulo de captura 51
3.28. Comunicación entre los dispositivos cliente y el módulo central 52
3.29. Estructura de la base de datos en *DB Browser for SQLite* 54
3.30. Diagrama de transición de estados de un cuestionario 56
3.31. Esquema relacional de la base de datos 57
3.32. Estructura interna del *Frontend* 63
3.33. Sistema de ejecución de la aplicación web 64
3.34. Pantallas y sub-pantallas de acceso 66
3.35. Vistas de la pantalla del alumno 67
3.36. Pantallas del panel 69
3.37. Vista de bienvenida de la interfaz táctil 71
3.38. Vista de estado de la interfaz táctil 71
3.39. Vista de examen de la interfaz táctil 72
3.40. Vista de configuración inicial de la interfaz táctil 72
3.41. Módulos comerciales de alimentación del módulo de captura 76
3.42. Esquema de conexiones del circuito del módulo de captura 78
3.43. Esquema simplificado del circuito del módulo de captura 79
3.44. Esquema simplificado del circuito del módulo central inicial (en desuso) 79
3.45. *Layouts* de las PCB 80
3.46. Carcasas impresas en 3D 81

Índice de Tablas

3.1. Costo de placas compatibles con cámaras, a octubre de 2026 43
3.2. Señales de conexión entre el *ESP32* y la cámara *OV2640* 44
3.3. Comparación de placas para el módulo central 47
3.4. Conexiones de la tarjeta microSD 47
3.5. Conexiones de la pantalla y del panel táctil 48
3.6. Capas de una aplicación web 52
3.7. Tiempos de las sesiones según el rol 61
3.8. Tamaño del código compilado con *React* y con *Preact* 63
3.9. Presupuesto de consumo de corriente con fuente de 5 voltios 74
3.10. Presupuesto de consumo estimado 74
3.11. Comparativa de alternativas de alimentación 75
3.12. Parámetros de operación del módulo *TP4056* 76
3.13. Parámetros de operación del módulo *MT3608* 76
3.14. Consumo de corriente empírico con celda de litio 77
3.15. Costo de los componentes del sistema (octubre 2026) 82

Capítulo 1

Introducción General

En este capítulo se presenta el dispositivo de seguimiento ocular desarrollado. Se detallan la motivación que impulsó el proyecto, el estado actual de esta tecnología, los objetivos planteados y el alcance definido para el sistema.

1.1. Introducción

En el ámbito educativo, los estudiantes con movilidad reducida enfrentan dificultades tanto para ser evaluados académicamente como para comunicarse con docentes, tutores y compañeros. Esta situación limita su autonomía y participación en las clases, principalmente porque las instituciones no cuentan con herramientas adaptadas a sus posibilidades físicas.

Frente a esto, la tecnología de seguimiento ocular (*eye-tracking*) se presenta como una alternativa práctica para atacar este problema. Esta herramienta permite a un usuario interactuar con un sistema mediante la mirada. De este modo, se elimina la necesidad de recurrir al movimiento de los brazos o a la asistencia de terceros.

Sin embargo, los sistemas de seguimiento ocular comerciales presentan dos grandes inconvenientes. Tienen costos muy elevados y no están enfocados en la dinámica educativa. En la práctica, no existen soluciones accesibles que permitan a los profesores evaluar a alumnos con estas condiciones y que, al mismo tiempo, faciliten su comunicación en el ámbito de estudio.

Para dar respuesta a esta necesidad, este trabajo presenta el desarrollo de un dispositivo de seguimiento ocular de bajo costo, diseñado para que un estudiante con movilidad reducida pueda resolver cuestionarios de opción múltiple y comunicarse con su entorno académico.

1.2. Estado del arte

La tecnología del *eye-tracking* no es una novedad. Desde hace décadas, múltiples empresas han implementado sistemas de detección ocular para diversas aplicaciones. El caso más destacado es el de la compañía *Tobii AB* [1], pionera en el desarrollo de *hardware* especializado. Entre sus productos se encuentran los *Tobii Pro Glasses*, anteojos diseñados para analizar el comportamiento visual del usuario en interacción libre con su entorno físico (figura 1.1). A su vez, alternativas como el *Tobii Pro Fusion* y el *Tobii Pro Spectrum* consisten en sistemas fijos y más robustos, orientados a la investigación científica de alta precisión.



Figura 1.1: *Tobii Glasses*

Por otro lado, la firma *EyeComTec* [2] desarrolla productos exclusivamente de *software* como *ECTKeyboard*, *ECTMouse* y *ECTTracker*, que permiten controlar computadoras con tan solo el movimiento de los ojos.

La compañía *Eyegaze Inc.* [3] ofrece el sistema *Eyegaze Edge*, un equipo que integra *hardware* y *software* dedicados. Está diseñado como una tecnología de asistencia médica de alta precisión y comunicación alternativa para personas con discapacidades motrices severas.

También se puede mencionar a *EyeHarp* [4], un instrumento musical digital que permite tocar música mediante el movimiento de los ojos. Funciona principalmente con rastreadores oculares *Tobii*, aunque también puede utilizarse con otros dispositivos que controlan el puntero del *mouse*. Está diseñado especialmente para personas con discapacidades físicas y ofrece una forma inclusiva de expresión musical.

Todos estos desarrollos demuestran que el seguimiento ocular es una tecnología sumamente avanzada y madura en la actualidad. Sin embargo, el presente proyecto se enfoca específicamente en el ámbito educativo.

1.3. Motivación

Todos los ejemplos de sistemas que utilizan *eye-tracking* expuestos anteriormente, ya sea por *hardware*, *software* o una combinación de ambos, son herramientas de calidad que pueden aplicarse en una gran variedad de campos. Han aportado una amplia gama de dispositivos versátiles que ayudan tanto a investigaciones científicas y de marketing, como también a facilitar acciones cotidianas.

Sin embargo, el principal problema de estos equipos es su elevado precio. El uso de tecnologías de tan alto nivel implica, a su vez, altos costos de desarrollo. En consecuencia, muchos de ellos no resultan accesibles para usuarios en una situación económica desfavorable.

Otro detalle observado durante la investigación es que ninguna de estas soluciones está centrada exclusivamente en el ámbito educativo, a pesar de su versatilidad. Por lo tanto, se necesita una alternativa más optimizada, intuitiva y sencilla para resolver este problema.

Por todo lo expuesto se propone este proyecto. Se trata de una solución que mantiene la ergonomía y la portabilidad de los dispositivos mencionados y que, al mismo tiempo, reduce los costos de producción. Esto último se logra mediante el uso de *hardware* y *software* de bajos recursos. Además, su orientación es exclusivamente educativa.

Este equipo tiene la intención de ser utilizado por instituciones académicas y familias de escasos recursos. De este modo, se garantiza una accesibilidad mucho mayor que la de cualquier otro producto de alta tecnología.

1.4. Alcance y objetivos

El objetivo general de este proyecto es desarrollar un dispositivo de bajo costo basado en microcontroladores económicos. Debe permitir que personas con movilidad reducida sean evaluadas mediante exámenes de opción múltiple y se puedan comunicar a través del seguimiento ocular (*eye-tracking*). Además, tiene que funcionar de forma autónoma y sin conexión a internet (*offline*).

Para alcanzar esta meta se plantean los siguientes objetivos específicos:

- Diseñar un dispositivo económico, ergonómico y portable, que funcione sin necesidad de acceso a internet.
- Brindar una interfaz intuitiva y de fácil uso, accesible desde cualquier equipo con navegador web.
- Permitir que los profesores gestionen cuestionarios de opción múltiple personalizados.
- Posibilitar que los tutores realicen un seguimiento académico integral del alumno.
- Facilitar un canal de comunicación directo para que docentes, tutores y compañeros puedan interactuar con el estudiante mediante preguntas informales.
- Implementar un sistema de registro e inicio de sesión, con una base de datos local para el almacenamiento de usuarios, cuestionarios y respuestas.
- Realizar el procesamiento de la imagen del ojo del alumno mediante un microcontrolador embebido en el dispositivo.

En cuanto al alcance, el proyecto comprende el desarrollo integral de un dispositivo autónomo capaz de capturar la imagen ocular del alumno, procesar la dirección de la mirada y traducir esa información en acciones concretas sobre una interfaz web. El sistema está delimitado para operar en un entorno local, sin depender de conexión a internet ni de infraestructura de red externa. El acceso a la plataforma se realiza mediante cualquier navegador web compatible.

Respecto a los usuarios del sistema, se delimitan los siguientes roles y permisos:

- Alumno: interactúa con el sistema mediante la mirada para resolver las evaluaciones de opción múltiple y responder a preguntas informales del entorno.
- Profesor: encargado de crear, editar y administrar sus cuestionarios académicos.
- Tutor: capacitado para consultar todos los cuestionarios, junto a las respuestas y los resultados obtenidos por el alumno.
- Compañero: habilitado para realizar preguntas esporádicas de opción múltiple con el fin de comunicarse con el alumno. Estas preguntas no quedan registradas en el sistema.

Capítulo 2

Introducción Específica

En este capítulo se exploran en términos muy generales los distintos conceptos, tecnologías, arquitecturas, paradigmas y técnicas que resultaron útiles para el desarrollo del trabajo.

2.1. Seguimiento Ocular y Visión por Computadora

El seguimiento ocular puede realizarse de distintas formas, entre ellas mediante la visión por computadora. Esta disciplina se complementa con el procesamiento de imágenes, y una de las bibliotecas más utilizadas para trabajar con ambos es *OpenCV* [5]. Entre las técnicas que ofrece esta biblioteca se encuentra la coincidencia de patrones [6, 7]. Las redes neuronales artificiales [8], por su parte, constituyen otro enfoque posible dentro de la misma disciplina. Primero se presentan los conceptos generales. Luego se describen *OpenCV* junto con la coincidencia de patrones y, por último, las redes neuronales artificiales.

2.1.1. Conceptos generales

El seguimiento ocular, conocido en inglés como *eye-tracking*, es el proceso tecnológico que permite medir y registrar el punto de la mirada de una persona o el movimiento de sus ojos en relación con la cabeza. Para ello, comúnmente se utiliza como punto de referencia el centro de la pupila. A diferencia del seguimiento facial (que analiza músculos, cejas o expresiones), el seguimiento ocular se concentra estrictamente en la forma y movimientos del globo ocular (Fig. 2.1).

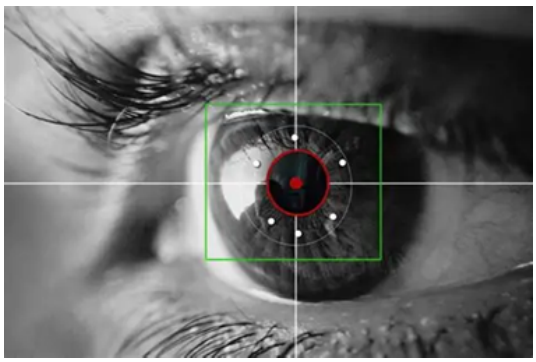


Figura 2.1: Representación gráfica del seguimiento ocular

Si bien esta tecnología cuenta con décadas de investigación y ha sido ampliamente utilizada en laboratorios de psicología, neuromarketing y análisis de usabilidad, el desarrollo de *hardware* más accesible ha permitido expandir sus aplicaciones. Uno de estos usos directos es la *interacción*

basada en la mirada (*gaze interaction*), donde los datos del movimiento ocular se utilizan para manipular un entorno digital.

El procesamiento de imágenes puede definirse como una técnica que modifica una imagen de entrada, mediante algoritmos matemáticos, para obtener otra distinta a la salida. Por ejemplo, permite cambiar su resolución, recortarla, modificar su paleta de colores y varias otras operaciones. En este contexto, dichas modificaciones se realizan con el fin de acondicionar la imagen para posteriores análisis computacionales.

Usualmente, la imagen procesada es el punto de partida de la visión por computadora. Esta disciplina consiste en el análisis que realiza una máquina sobre una imagen para extraer características, interpretarlas y aprender de ellas. De este modo se simula la percepción visual humana. El seguimiento ocular, descrito anteriormente, es un claro ejemplo de un proceso de visión por computadora.

En el caso del presente trabajo, orientado a la inclusión en el ámbito educativo, el seguimiento ocular se emplea para detectar hacia cuál de las opciones de respuesta a una pregunta dirige la mirada el alumno. Estas opciones se disponen de manera clara en una pantalla. Luego esa información se procesa para registrar la selección correspondiente. De este modo, el estudiante puede responder cuestionarios de opción múltiple sin necesidad de manipular un cursor ni de contar con destreza física.

2.1.2. OpenCV y coincidencia de patrones

Lograr que una máquina interprete de manera autónoma el comportamiento de las imágenes es una tarea compleja, motivo por el cual se emplean herramientas especializadas que facilitan el proceso. Entre ellas se destaca *OpenCV (Open Source Computer Vision Library)*, una de las bibliotecas de código abierto más utilizadas en el campo del análisis visual. Fue creada por la empresa *Intel* y cuenta con más de 2500 algoritmos matemáticos optimizados que sirven tanto para el procesamiento de imágenes como para la visión por computadora. Entre sus principales capacidades destacan la manipulación estructural, la detección de características geométricas (como bordes, esquinas o líneas), la detección de objetos y el análisis de movimiento.

Esta biblioteca también implementa la coincidencia de patrones, conocida en inglés como *Template Matching*. Esta técnica permite encontrar la ubicación de una imagen pequeña, denominada molde o plantilla (*template*), dentro de una imagen de mayor tamaño, denominada imagen de entrada (*input*). El proceso consiste en deslizar la plantilla píxel por píxel sobre la imagen de entrada, de izquierda a derecha y de arriba hacia abajo.

El recorrido comienza con las coordenadas $X = 0$ e $Y = 0$ de ambas imágenes alineadas, es decir, con sus esquinas superiores izquierdas superpuestas. En esa posición se realiza un cálculo matemático. Luego, la plantilla se desplaza un píxel hacia la derecha, hasta las coordenadas $X = 1$ e $Y = 0$ y el cálculo se vuelve a realizar. Este desplazamiento continúa hasta completar la primera fila. A continuación, la plantilla se alinea con la imagen de entrada una fila más abajo ($X = 0$, $Y = 1$) y el recorrido vuelve a comenzar. Esta secuencia se repite en cada fila hasta cubrir por completo la imagen de entrada (Fig. 2.2).

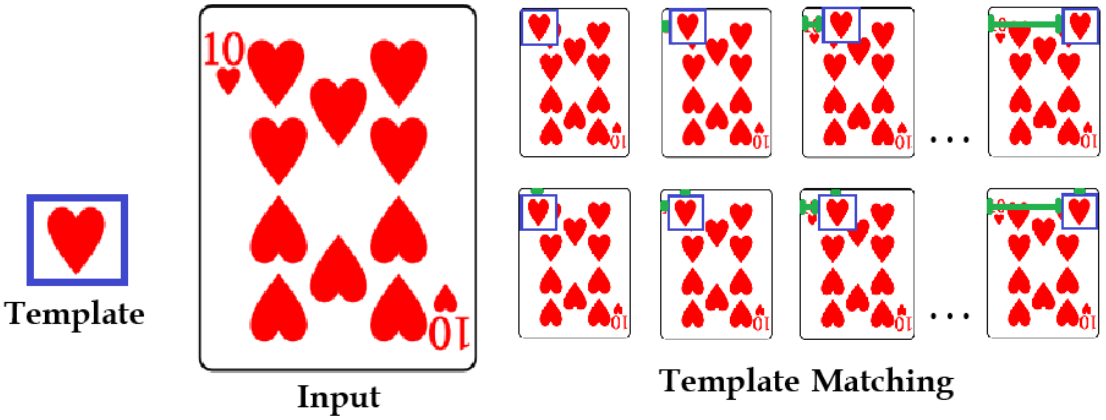


Figura 2.2: Imagen ilustrativa del proceso de *Template Matching*

En cada posición de la plantilla, el algoritmo aplica una fórmula estadística denominada *coeficiente de correlación normalizado con media cero*, presentada a continuación:

$$R(x,y) = \frac{\sum_{x',y'}(T(x',y') - \bar{T}) \cdot (I(x+x',y+y') - \bar{I})}{\sqrt{\sum_{x',y'}(T(x',y') - \bar{T})^2 \cdot \sum_{x',y'}(I(x+x',y+y') - \bar{I})^2}} \tag{2.1}$$

En esta fórmula, T corresponde a la imagen de la plantilla e I a la imagen de entrada. Ambas son matrices cuyos elementos representan sus píxeles, y su valor depende del formato de las imágenes. Los valores (x, y) son las coordenadas de referencia, en las que se alinean la plantilla y la imagen de entrada. Los valores (x', y'), en cambio, son las coordenadas de los píxeles dentro de la plantilla.

Esta expresión se basa en el *coeficiente de correlación de Pearson*. Evalúa qué tan similar es la plantilla respecto de la sección de la imagen de entrada sobre la que está superpuesta y, en función de eso, otorga un valor entre $-1{,}0$ y $+1{,}0$. Un valor de $+1{,}0$ indica que la plantilla es idéntica a la porción de imagen analizada. Un valor de $-1{,}0$ significa que la plantilla tiene los colores opuestos a los de la imagen, mientras que un valor de 0 refleja que no existe correlación alguna.

El resultado tras todo el recorrido de la plantilla es una matriz bidimensional de valores. El punto con el valor más alto indica la coordenada en la que el algoritmo encontró la mayor coincidencia (Fig. 2.3).

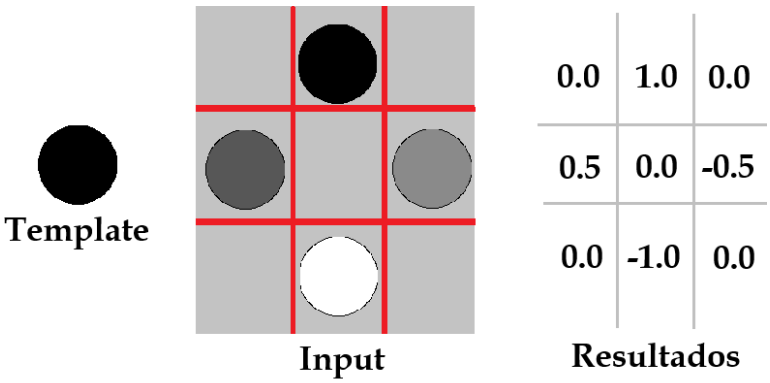


Figura 2.3: Imagen ilustrativa de una matriz de resultados de *Template Matching*

Esta matriz permite obtener no solo la ubicación de la coincidencia con la plantilla, sino también su nivel de semejanza.

2.1.3. Redes neuronales artificiales

Las redes neuronales artificiales son un tipo de modelo de aprendizaje automático, conocido en inglés como *Machine Learning* [9]. Esta rama de las ciencias de la computación permite a los sistemas aprender y resolver problemas sin ser programados de forma tradicional. El proceso consiste en suministrar datos a una computadora para que construya un modelo predictivo basado en esa experiencia (Fig. 2.4).



Figura 2.4: El proceso del *Machine Learning*

Reciben este nombre debido a que su estructura, basada en celdas o nodos interconectados para el procesamiento de información, se inspira en el comportamiento biológico del sistema nervioso. Dentro del aprendizaje automático, se encuentran entre los modelos más utilizados para el análisis de imágenes.

Estas redes organizan sus neuronas en tres niveles jerárquicos principales. La capa de entrada recibe los datos en crudo, como los píxeles de una imagen. Las capas ocultas constituyen el núcleo de procesamiento y detectan desde características básicas hasta patrones complejos. La capa de salida entrega el resultado final, como las coordenadas de una pupila (Fig. 2.5).

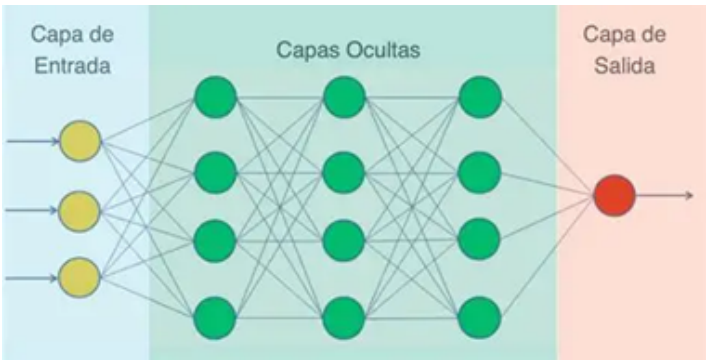


Figura 2.5: Diagrama simplificado de una red neuronal

El aprendizaje del modelo se logra mediante el entrenamiento, un proceso iterativo donde la red evalúa datos conocidos y ajusta sus parámetros internos para minimizar el margen de error. En cada neurona se calcula la siguiente expresión.

$$z = \sum w_i x_i + b \tag{2.2}$$

Donde x_i representa el dato de entrada individual, w_i es el peso (la importancia relativa asignada a esa entrada específica) y b es el sesgo o *bias*, un valor que también se ajusta durante el entrenamiento y que desplaza el resultado con independencia de las entradas. El resultado z es evaluado finalmente por una función de activación, la cual determina con qué intensidad la neurona transmite la señal hacia la siguiente capa.

La forma en que se organizan y conectan estas capas define la arquitectura de la red, que varía según la naturaleza del problema a resolver. Se distinguen tres categorías principales.

- Prealimentadas (*Feedforward*): la información viaja de forma unidireccional, de la entrada a la salida. Se utilizan en problemas sencillos con datos livianos que no requieren memoria temporal.
- Convolucionales (*CNN*): utilizan filtros matemáticos para analizar datos espaciales por secciones. Son la arquitectura más utilizada para procesar imágenes y video.
- Recurrentes (*RNN*): poseen bucles internos que les otorgan memoria secuencial. Se aplican en análisis temporales, como el procesamiento de lenguaje natural o las secuencias largas de video.

En función de los requerimientos de este proyecto, se planteó como hipótesis inicial el entrenamiento e implementación de un modelo convolucional liviano, cuyo desempeño y viabilidad sobre *hardware* de bajos recursos se analizan en el capítulo 3.

2.2. Sistemas Embebidos y Control Gráfico

Los sistemas embebidos suelen construirse en torno a un microcontrolador, como los de la familia *ESP32* [10]. Para mostrar interfaces gráficas con sus recursos limitados existen bibliotecas como *LVGL* [11]. Primero se presentan los microcontroladores y luego las interfaces gráficas embebidas.

2.2.1. Microcontroladores y familia ESP32

Un microcontrolador (MCU) es un sistema informático completo encapsulado en un único circuito integrado. Su estructura reúne un microprocesador central junto con bloques de memoria interna (tanto volátil para datos como no volátil para el programa) y múltiples periféricos. Estos módulos adicionales pueden incluir temporizadores, protocolos de comunicación serial y puertos de entrada/salida de propósito general (GPIO), lo que le permite interactuar directamente con el entorno físico y dispositivos externos de manera autónoma.

Una familia de placas de desarrollo que ha ganado enorme popularidad en el ámbito de los sistemas embebidos y el Internet de las Cosas (IoT) es la de los microcontroladores *ESP32* (Fig. 2.6). Creados por la compañía *Espressif Systems*, estos dispositivos surgieron como una solución de bajo costo y alta eficiencia energética diseñada para suplir la creciente necesidad de conectividad inalámbrica en proyectos electrónicos. A diferencia de las placas más tradicionales, el *ESP32* destaca por integrar módulos de Wi-Fi y Bluetooth directamente dentro del mismo chip.

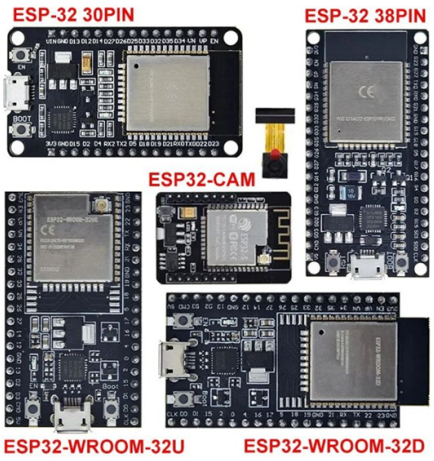


Figura 2.6: Ejemplos de placas de desarrollo de la familia *ESP32*

Al estar generalmente equipado con un procesador de doble núcleo y una memoria considerablemente mayor que la de sus predecesores, permite manejar tareas más complejas, alojar servidores web locales y procesar datos a mayor velocidad. Con todo esto logra un equilibrio entre rendimiento, conectividad y tamaño reducido.

2.2.2. Interfaces gráficas embebidas y biblioteca LVGL

El desarrollo de interfaces gráficas de usuario (GUI, por sus siglas en inglés) en sistemas embebidos presenta desafíos asociados a las restricciones de memoria RAM, capacidad de cómputo y consumo de energía de los microcontroladores. A diferencia de los entornos convencionales, donde el sistema operativo gestiona los recursos visuales mediante unidades de procesamiento gráfico dedicadas (GPU), en un microcontrolador el renderizado debe ejecutarse de forma eficiente. Así se garantiza fluidez visual y respuesta táctil en tiempo real sin sobrecargar la CPU.

Para resolver esta problemática, se recurre a bibliotecas gráficas optimizadas para *hardware* limitado, entre las que destaca *LVGL* (*Light and Versatile Graphics Library*). *LVGL* es una herramienta de código abierto escrita en lenguaje C, diseñada para operar con requerimientos mínimos de memoria. Implementa un modelo orientado a objetos basado en elementos visuales predefinidos (*widgets*) como botones, etiquetas, listas, deslizadores y contenedores, los cuales pueden personalizarse mediante estilos y animaciones.

LVGL no depende de un modelo de pantalla en particular. La biblioteca dibuja la interfaz en un búfer de memoria e interpreta las acciones del usuario, mientras que la comunicación con el *hardware* queda a cargo de dos controladores (*drivers*). Estas son funciones que se escriben para cada placa:

- Controlador de pantalla (*Display Driver*): envía el contenido del búfer al circuito que maneja el panel, por ejemplo mediante una interfaz SPI. Para evitar parpadeos, puede trabajar con dos búferes, de modo que uno se dibuja mientras el otro se envía a la pantalla.
- Controlador de dispositivo de entrada (*Input Device Driver*): lee periféricos como una pantalla táctil, un teclado o un codificador rotativo, e informa a la biblioteca qué acción realizó el usuario, por ejemplo en qué punto tocó la pantalla.

De esta forma, para usar *LVGL* con otra pantalla u otro periférico de entrada, alcanza con cambiar estos controladores, sin modificar la interfaz.

En el ámbito del *hardware* comercial de bajo costo, esta combinación de pantalla, interfaz táctil y microcontrolador suele integrarse en módulos compactos denominados comúnmente *Cheap Yellow Display* (*CYD*). Estas placas incorporan un SoC *ESP32*, un panel TFT y un controlador táctil resistivo (Fig. 2.7). El uso de *LVGL* sobre estas plataformas permite ejecutar interfaces interactivas fluidas directamente en el dispositivo embebido de manera autónoma.



Figura 2.7: Placa de desarrollo *ESP32-CYD* (vista frontal y trasera)

2.3. Redes y Comunicaciones

Las redes locales Wi-Fi son uno de los medios por los que pueden comunicarse los dispositivos. Esta comunicación se rige por protocolos organizados en capas, y algunos de ellos permiten transmitir video de forma continua. Primero se presentan las redes locales Wi-Fi. Luego se describen los protocolos de comunicación junto con la transmisión de video.

2.3.1. Redes locales Wi-Fi

El estándar IEEE 802.11 [12], conocido comúnmente como Wi-Fi, define los protocolos para la implementación de redes de área local inalámbricas (WLAN). La topología de estas redes se estructura a través de dos modos de operación:

- Modo Estación (*Station* - STA): en esta configuración, el nodo inalámbrico actúa como cliente. Su función consiste en explorar el espectro de radiofrecuencia y detectar una red existente a través de su identificador (SSID). Luego se autentica mediante los protocolos de seguridad correspondientes (como WPA2/WPA3) y se asocia a un punto de acceso central para integrarse al tráfico de dicha red.
- Modo Punto de Acceso (*Access Point* - AP o *SoftAP*): en este modo, el dispositivo asume el rol de concentrador e iniciador de la red. Emite periódicamente tramas de sincronización para anunciar la presencia de la red, autentica a los nodos que solicitan conexión y gestiona la tabla de clientes asociados. Así, crea una infraestructura de red local independiente.

En el marco de este trabajo, uno de los módulos del sistema se configura en modo *Access Point* para generar una red local cerrada e independiente, mientras que los dispositivos de los usuarios se conectan a dicha red en modo Station (Fig. 2.8). Esto permite la comunicación directa entre todos los componentes sin depender de un *router* externo ni de acceso a internet.

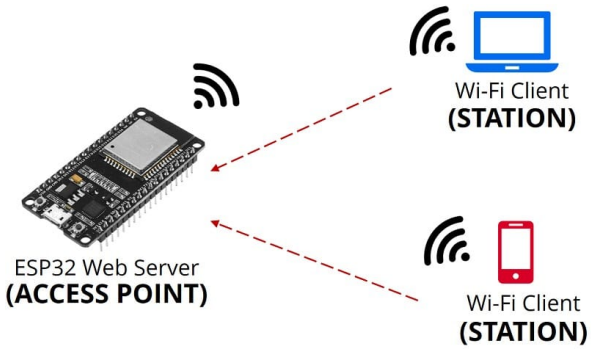


Figura 2.8: Red Wi-Fi con un microcontrolador *ESP32* como punto de acceso

2.3.2. Protocolos de comunicación y transmisión de video

La comunicación en una red de datos se rige por protocolos que definen cómo se estructura, transmite y recibe la información. Estos protocolos se organizan en cuatro capas, según el modelo TCP/IP [13] (Fig. 2.9). La capa de acceso a la red transmite los datos a través del medio físico, como por ejemplo las redes Wi-Fi presentadas anteriormente. Sobre ella, la capa de Internet identifica a cada dispositivo y dirige los datos hacia su destino. La capa de transporte organiza el intercambio de datos entre los programas de cada equipo. Por último, la capa de aplicación reúne los protocolos que utilizan directamente esos programas. Cada capa se apoya en los servicios de la inferior.

- Protocolo IP (*Internet Protocol*): estructura los datos en paquetes que contienen las direcciones de origen y destino para su entrega en la red. Estas direcciones identifican a cada dispositivo de manera unívoca. En redes locales, la dirección de cada dispositivo se establece mediante dos modalidades:
  - Asignación estática: la dirección se define de manera manual en el equipo y permanece fija a lo largo del tiempo, lo que permite localizar siempre al dispositivo en esa misma ubicación.
  - Asignación dinámica: la dirección se asigna de forma automática mediante el protocolo DHCP.
- Protocolo ICMP (*Internet Control Message Protocol*): permite que los dispositivos intercambien mensajes de control y de error. Su uso más conocido es el *ping*, una herramienta que envía una solicitud a un dispositivo y espera su respuesta. Si la respuesta llega, el dispositivo se encuentra conectado y accesible.
- Protocolo TCP (*Transmission Control Protocol*): antes del intercambio de datos, establece una conexión entre ambos equipos. Luego garantiza que la información llegue completa, sin alteraciones y en el mismo orden en que se envió. Si un paquete se pierde, lo vuelve a transmitir.
- Protocolo UDP (*User Datagram Protocol*): envía los datos sin establecer una conexión previa ni confirmar su recepción. Por este motivo resulta más rápido y liviano, pero es menos confiable que TCP.
- Protocolo DHCP (*Dynamic Host Configuration Protocol*): asigna de forma automática una dirección IP disponible a cada equipo que se asocia a la red, lo que agiliza la incorporación de nuevos dispositivos sin requerir configuraciones individuales. Se apoya en UDP.

- Protocolo mDNS (*Multicast Domain Name System*) [14]: permite que los dispositivos de una red local se identifiquen mediante nombres de dominio legibles por humanos, en lugar de direcciones IP numéricas. A diferencia del sistema DNS tradicional, no requiere un servidor de nombres. El equipo envía la consulta a toda la red local, y responde aquel que tiene asignado ese nombre. Se apoya en UDP.
- Protocolo HTTP (*Hypertext Transfer Protocol*) [15]: opera bajo el modelo cliente-servidor mediante un esquema de peticiones y respuestas. Se apoya en TCP, lo que asegura que la información no sufra pérdidas ni alteraciones. A través de este mecanismo se transfieren páginas web, estilos y datos estructurados. Estos últimos suelen enviarse en formato JSON (*JavaScript Object Notation*) [16]. Este formato organiza la información como texto, en pares que asocian un nombre con un valor. Además, admite listas y permite agrupar elementos dentro de otros. Al tratarse de texto plano, programas escritos en lenguajes distintos pueden generarlo e interpretarlo sin dificultad.
- Transmisión continua de video (*Streaming* MJPEG): en lugar de realizar una petición HTTP por cada imagen, la técnica MJPEG (*Motion* JPEG) mantiene una única conexión abierta para enviar una secuencia ininterrumpida de fotogramas JPEG independientes delimitados por un separador [17, 18]. El cliente actualiza la imagen en pantalla a medida que recibe cada cuadro. De este modo, se logra una visualización de video fluida en tiempo real.

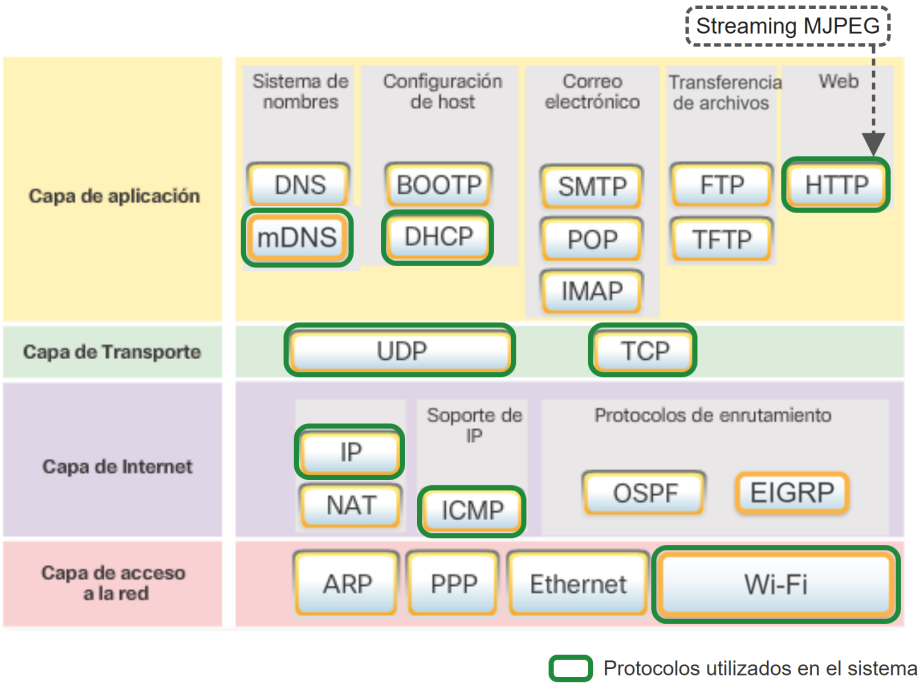


Figura 2.9: Modelo TCP/IP con los protocolos utilizados en el sistema

2.4. Arquitectura de una aplicación web

En el desarrollo de *software*, las aplicaciones web suelen combinar el modelo cliente/servidor con el patrón MVC [19]. Con este enfoque, la aplicación se divide en tres partes, que son el *frontend*, el *backend* y la base de datos. En esta sección se explican primero el modelo cliente/servidor y el patrón MVC, y luego cada una de estas tres partes.

2.4.1. Modelo Cliente/Servidor

La arquitectura cliente/servidor (Fig. 2.10) es un modelo de diseño de *software* y redes en el cual las tareas a realizar se distribuyen entre un proveedor de recursos denominado *Servidor* y un demandante de dichos recursos denominado *Cliente*. Se basa en una dinámica de petición/respuesta entre ambas entidades, las cuales se detallan a continuación:

- El cliente: es el programa que interactúa con el usuario final. Su función principal es elaborar y enviar peticiones de datos o ejecución de tareas a través de una red. Luego, recibe, formatea y presenta los resultados obtenidos en la interfaz de usuario. Es el componente que siempre inicia la comunicación.
- El servidor: es el programa en ejecución continua diseñado para escuchar, recibir y procesar las peticiones de uno o múltiples clientes. Su función es la de administrar los recursos y servicios para devolver al cliente la respuesta solicitada.

La separación entre cliente y servidor es lógica. Ambos pueden ejecutarse en equipos distintos o en un mismo equipo.

Esta modalidad es muy utilizada en los sistemas modernos para centralizar la carga de trabajo en el servidor. De este modo, los clientes no necesitan su propia base de datos ni sus propios recursos. De requerirlos, se incrementarían significativamente los costos y la complejidad de *hardware* en cada terminal. Además, los procesos se volverían ineficientes.

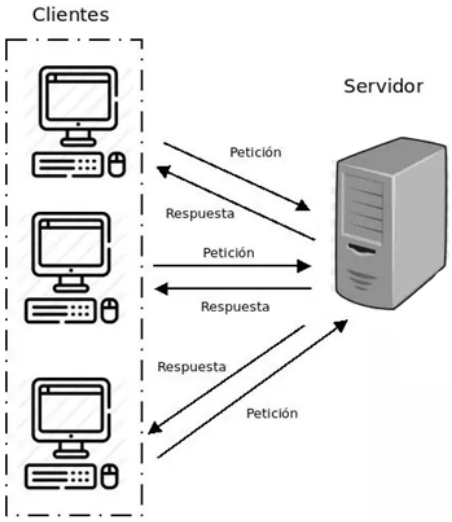


Figura 2.10: Modelo *Cliente/Servidor*

2.4.2. Patrón de diseño Modelo-Vista-Controlador (MVC)

El patrón Modelo-Vista-Controlador (*Model-View-Controller* - MVC) es una estrategia de diseño de *software* orientada a desacoplar la lógica de negocio, la gestión de datos y la interfaz de usuario en tres componentes con responsabilidades bien definidas (Fig. 2.11):

- Modelo (*Model*): representa la estructura de los datos del sistema y gestiona las reglas de negocio. Se encarga de interactuar con los mecanismos de almacenamiento y persistencia para consultar, insertar, modificar o eliminar registros. Así, asegura la integridad de la información sin vincularse con la interfaz gráfica.

- Vista (*View*): constituye la capa de presentación con la que interactúa el usuario. Su función es recibir los datos procesados y mostrarlos en un formato comprensible. Además, captura los eventos y acciones del usuario para transferirlos al Controlador.
- Controlador (*Controller*): actúa como intermediario entre el Modelo y la Vista. Procesa las solicitudes y eventos que esta genera, ejecuta la lógica correspondiente, solicita las operaciones necesarias al Modelo y actualiza la Vista con el resultado.

Esta separación de responsabilidades favorece la modularidad, simplifica el mantenimiento del sistema y permite modificar la interfaz visual o el almacenamiento de datos de forma independiente sin alterar la lógica central de la aplicación. Por estas características, es un patrón muy utilizado en el desarrollo de aplicaciones web.

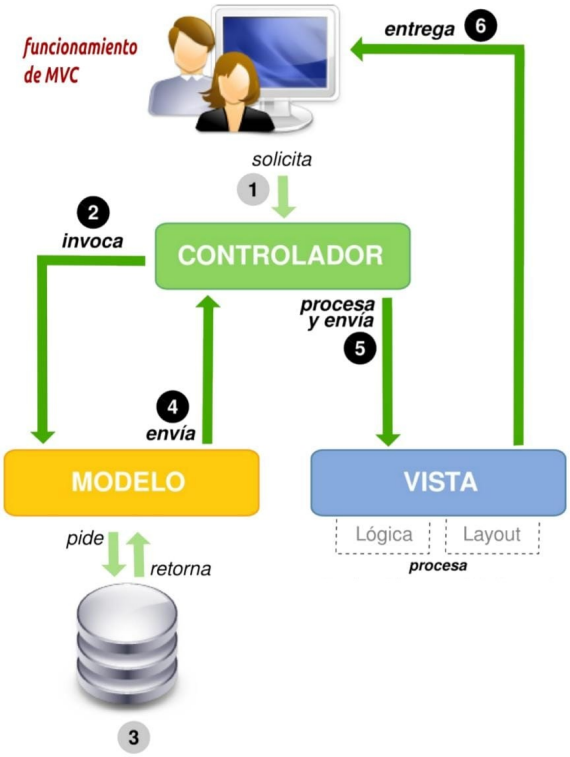


Figura 2.11: Patrón de diseño Modelo-Vista-Controlador (MVC)

2.4.3. Frontend

El *frontend* opera como la Vista del patrón MVC. Presenta la interfaz gráfica, captura las acciones del usuario y emite las peticiones.

Al solicitar la aplicación, el navegador descarga un conjunto de archivos que procesa localmente para generar la interfaz. Esta tarea se apoya en tres tecnologías complementarias (Fig. 2.12):

- HTML (*HyperText Markup Language*, lenguaje de marcado de hipertexto) [20]: define la estructura y el contenido del documento. Mediante etiquetas, especifica qué función cumple cada elemento en la interfaz (títulos, bloques de texto, botones o casillas para ingresar datos) y los organiza en un orden lógico que el navegador interpreta para armar la pantalla.
- CSS (*Cascading Style Sheets*, hojas de estilo en cascada) [21]: controla el aspecto visual y la ubicación de los elementos definidos por HTML. Aplica reglas para establecer colores,

tipografías, tamaños y espacios. También organiza cómo se distribuyen los bloques en la pantalla. De este modo, la interfaz se adapta al tamaño del dispositivo.

- *JavaScript* [22]: es el lenguaje de programación que se ejecuta directamente en el navegador. Mientras que el HTML define qué elementos existen y el CSS define cómo se ven, *JavaScript* hace que la página responda a las acciones del usuario. Detecta eventos como clics o el envío de un formulario y altera el contenido en pantalla en tiempo real según esa interacción. Además, se comunica con el servidor para intercambiar información mientras la aplicación está en uso.

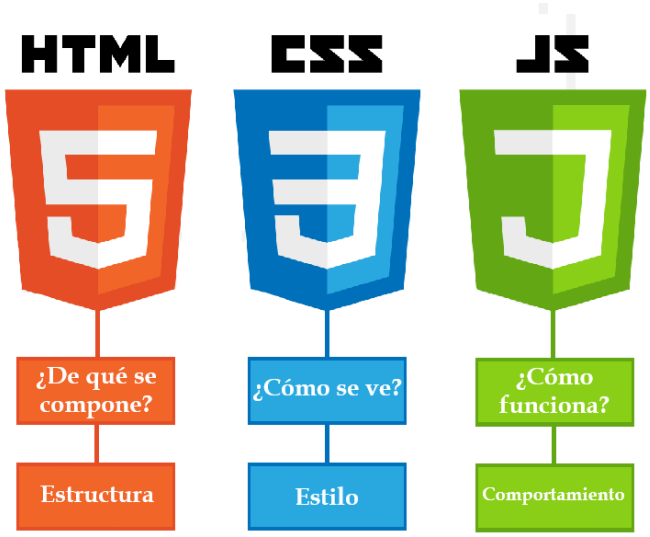


Figura 2.12: Tecnologías del *frontend*: HTML (estructura), CSS (apariencia) y *JavaScript* (comportamiento)

A medida que una aplicación suma pantallas y funciones, mantener el código fuente en archivos globales vuelve el sistema denso y difícil de ordenar. Para resolver esto, el desarrollo adopta una arquitectura basada en componentes (CBA, *Component-Based Architecture*). En este enfoque, la interfaz se estructura a partir de componentes independientes, donde cada elemento funciona como una entidad autónoma que reúne su estructura, presentación y lógica asociada (sus propios códigos HTML, CSS y *JavaScript* independientes del resto).

Bajo este modelo, el código fuente se divide en múltiples archivos modulares que se importan y articulan en un archivo principal. La única relación de dependencia entre componentes se da cuando uno contiene a otro. En esa situación, únicamente el componente padre puede alterar el estado del hijo. Esta estructura ofrece una fácil reusabilidad de elementos. Además, garantiza la modularidad y el aislamiento de fallos. Es decir, si un componente específico presenta un error, no compromete el funcionamiento del programa en su totalidad.

Existen diversas herramientas disponibles para implementar este esquema, entre las cuales destaca *React*, una biblioteca de *JavaScript* creada y mantenida por *Meta*. Está enfocada en la construcción de interfaces de usuario mediante componentes reutilizables. Por otra parte, *Preact* [23] se presenta como una alternativa más ligera y eficiente para entornos con recursos limitados. Creada por una comunidad independiente, esta herramienta ofrece las mismas capacidades de desarrollo que *React*, pero omite características accesorias, lo que reduce el peso del

empaquetado final. En consecuencia, disminuye el tamaño de los archivos estáticos que el microcontrolador debe alojar en su memoria y transferir posteriormente al navegador al momento de utilizar la interfaz.

2.4.4. Backend

El *backend* reúne las funciones del Controlador y del Modelo del patrón MVC. Atiende las solicitudes entrantes, procesa la lógica de la aplicación y administra la persistencia de los datos.

En este desarrollo, el *backend* adopta una arquitectura en capas basada en el patrón Controlador-Servicio-Repositorio, que deriva directamente de MVC. La Vista se resuelve de forma independiente en el cliente web. En consecuencia, los Controladores conservan su rol original y el Modelo tradicional se divide en dos capas separadas. Esta división diferencia la lógica de la persistencia de datos (Fig. 2.13).

- Controladores (*controllers*): reciben las peticiones que llegan desde el cliente web, extraen los datos de la solicitud y llaman al Servicio correspondiente para devolver luego la respuesta. No aplican reglas de funcionamiento ni interactúan con los datos guardados.
- Servicios (*services*): representan la parte del Modelo encargada de la lógica del sistema. Reciben los datos procesados por el Controlador, aplican las reglas operativas, realizan validaciones y coordinan cuándo consultar o modificar información mediante los Repositorios.
- Repositorios (*repositories*): representan la parte del Modelo dedicada exclusivamente a los datos y su almacenamiento. Son los únicos que leen y escriben en la base de datos, aislando los detalles de la persistencia del resto del sistema.

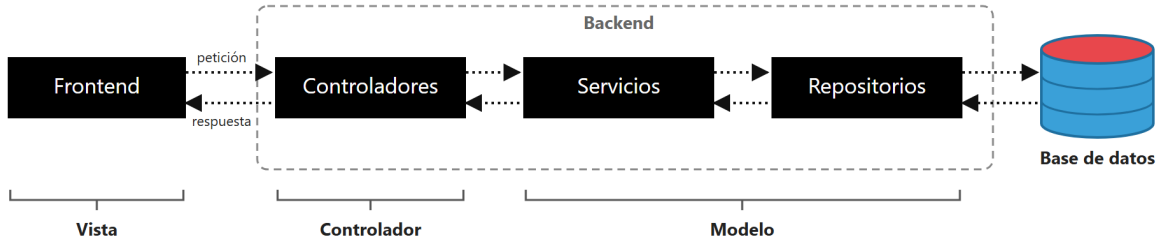


Figura 2.13: Arquitectura en capas del *backend* y su correspondencia con el patrón MVC

Las peticiones que atienden los Controladores llegan a través de una interfaz de programación de aplicaciones (API, *Application Programming Interface*). Esta interfaz define un conjunto de rutas, cada una asociada a un tipo de información, como por ejemplo los usuarios de una aplicación. Sobre cada ruta, el método HTTP de la petición indica la operación solicitada. Los más utilizados son:

- GET: consulta información sin modificarla.
- POST: crea un registro nuevo.
- PUT: modifica un registro completo.
- PATCH: modifica solo una parte de un registro.
- DELETE: elimina un registro.

Por su parte, cada respuesta incluye un código de estado numérico que informa el resultado. Los códigos de la serie 200 indican que la operación se completó con éxito. Los de la serie 400 señalan un error en la petición enviada por el cliente. Por último, los de la serie 500 indican una falla del servidor.

Para permitir el acceso, el *backend* verifica la contraseña de cada usuario. Por seguridad, las contraseñas no se almacenan en texto plano, sino que se les aplica una función de *hash*. Esta función convierte un dato de cualquier longitud en una cadena de tamaño fijo. El proceso es irreversible, por lo que a partir del resultado no es posible conocer el dato original. Además, una misma entrada produce siempre el mismo resultado, y un cambio mínimo genera uno completamente distinto. Uno de los algoritmos más utilizados es SHA-256 (*Secure Hash Algorithm*) [24], que produce resultados de 256 bits.

Antes de aplicar la función, cada contraseña se combina con un *salt*, una cadena aleatoria distinta para cada usuario. El *salt* se almacena junto con el resultado, ya que se necesita para verificar la contraseña más adelante. Cuando el usuario inicia sesión, el *backend* combina la contraseña ingresada con ese mismo *salt*, vuelve a aplicar la función y compara el resultado con el valor almacenado (Fig. 2.14).

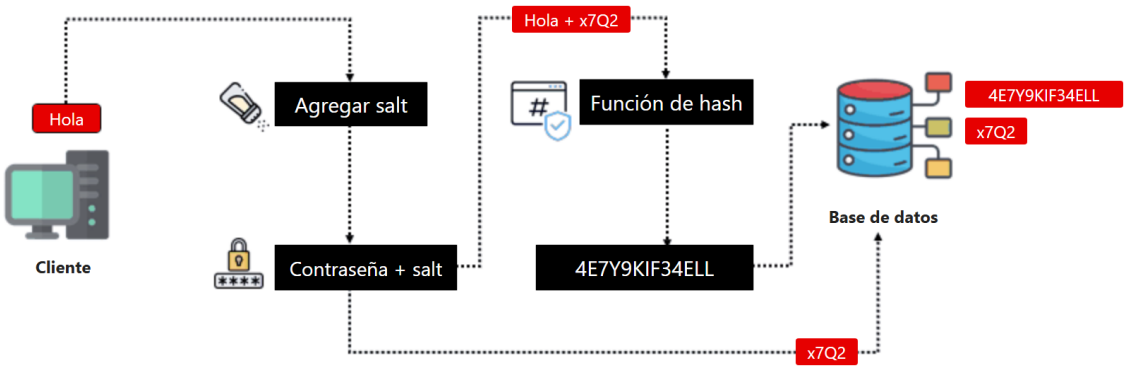


Figura 2.14: Almacenamiento de contraseñas mediante *hash* con *salt*

El protocolo HTTP no conserva información entre una petición y la siguiente, ya que trata a cada una de forma independiente. Por este motivo, el *backend* necesita un mecanismo para reconocer a un usuario que ya inició sesión. Una solución habitual consiste en entregarle al cliente, una vez verificados su usuario y contraseña, una cadena única denominada *token*. A partir de ese momento, el cliente la adjunta en cada petición, y el servidor la utiliza para identificar al usuario y comprobar sus permisos. Cuando la sesión finaliza o expira, esa cadena deja de ser válida.

2.4.5. Base de datos

Una base de datos es una colección organizada e integrada de datos almacenados de forma persistente para su posterior consulta y manipulación. En el patrón MVC, el Modelo es el único componente que accede a ella.

Dentro de los distintos paradigmas existentes, el modelo relacional organiza la información en estructuras bidimensionales denominadas tablas (o relaciones), compuestas por filas (registros o tuplas) y columnas (campos o atributos).

Los fundamentos de este modelo se basan en los siguientes conceptos (Fig. 2.15):

- Estructura y esquemas: cada tabla posee un esquema definido que determina el tipo de dato que puede almacenar cada columna (numérico, texto, fecha, etc.), lo que garantiza la consistencia del formato.
- Claves e integridad relacional: para identificar de forma unívoca cada registro se utiliza una clave primaria (*Primary Key*). La vinculación entre distintas tablas se establece mediante claves foráneas (*Foreign Keys*), lo que permite relacionar entidades sin duplicar información. El sistema puede además exigir que cada clave foránea apunte a un registro existente, condición conocida como integridad referencial.
- Lenguaje de consulta estructurado (SQL): es el lenguaje formal estándar (*Structured Query Language*) empleado para definir la estructura de los datos, realizar búsquedas mediante filtros, e insertar, modificar o eliminar información de manera declarativa.
- Índices: son estructuras auxiliares que se construyen sobre una o más columnas de una tabla y mantienen sus valores ordenados, de forma similar al índice de un libro. Permiten ubicar registros sin recorrer la tabla completa, lo que acelera las búsquedas. Como contrapartida, ocupan espacio adicional y deben actualizarse en cada inserción, modificación o eliminación. Un índice parcial abarca únicamente los registros que cumplen una condición determinada, lo que reduce su tamaño y su costo de mantenimiento.
- Transacciones y propiedades ACID: una transacción es un conjunto de operaciones que se ejecutan como una única unidad lógica. Para garantizar la confiabilidad, los sistemas relacionales implementan las propiedades ACID. Estas son:
  - Atomicidad: la operación se completa en su totalidad o no se aplica.
  - Consistencia: se respetan todas las reglas e integridad del esquema.
  - Aislamiento: las operaciones simultáneas no interfieren entre sí.
  - Durabilidad: los cambios confirmados persisten ante fallos o cortes de energía.

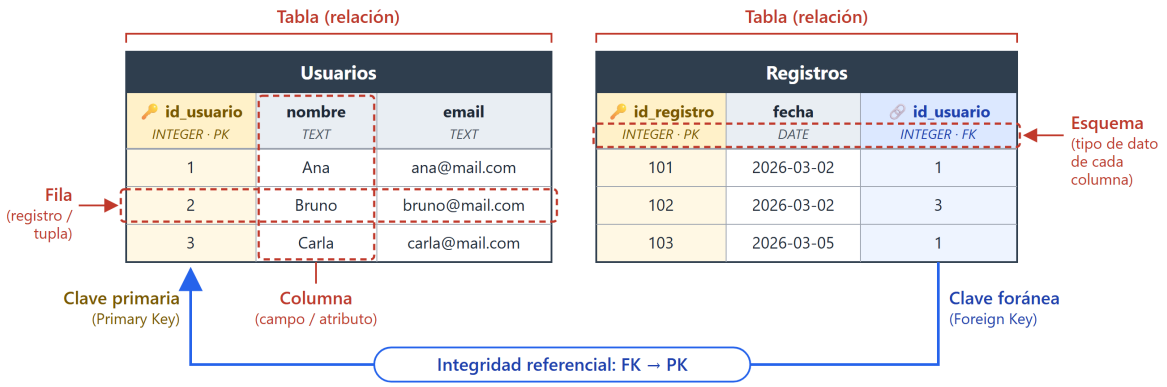


Figura 2.15: Elementos del modelo relacional

En el desarrollo de *software* existen dos formas principales de gestionar una base de datos: mediante sistemas basados en un servidor independiente (que atienden peticiones a través de una red) o mediante motores embebidos.

Un motor de base de datos embebido es una biblioteca de *software* que se integra directamente dentro del mismo código de la aplicación. En este esquema, el motor no se ejecuta como un proceso separado, sino que forma parte del propio ejecutable del sistema.

SQLite [25] es un motor de base de datos relacional embebido de código abierto ampliamente utilizado que implementa el estándar SQL. Sus características teóricas esenciales son:

- Arquitectura sin servidor (*Serverless*): los procesos leen y escriben directamente en el soporte de almacenamiento, lo que elimina la necesidad de instalar, configurar y mantener un servidor de base de datos aparte.
- Almacenamiento en archivo único: la base de datos completa (esquema, tablas, índices y datos) se guarda dentro de un único archivo en el sistema de archivos del dispositivo, lo que simplifica su copia, transporte y respaldo.
- Autonomía (*Self-contained*): no depende de bibliotecas externas complejas ni requiere un sistema operativo, lo que permite su funcionamiento en entornos con recursos de memoria y procesamiento acotados.
- Cumplimiento transaccional: a pesar de su tamaño compacto, implementa transacciones atómicas que aseguran que los datos no se corrompan ante reinicios o pérdidas de alimentación eléctrica imprevistas.

Como otros motores, *SQLite* permite ajustar su funcionamiento. Para ello emplea sentencias especiales denominadas *PRAGMA*, como por ejemplo las que definen el tamaño de la caché.

Capítulo 3

Diseño e implementación

Este capítulo describe el diseño y la implementación del dispositivo. Los requisitos del sistema guiaron la evaluación de distintos métodos de detección de la mirada, y el método adoptado determinó la arquitectura general. Sobre esa arquitectura se apoyan el *hardware* de cada módulo, la red que los comunica y el *software*, que comprende la aplicación web y la interfaz táctil del módulo central. El capítulo se completa con la alimentación, el diseño físico y el costo del prototipo.

3.1. Requisitos y especificaciones del sistema

En esta sección se detallan los requisitos planteados al inicio del proyecto. Derivan de los objetivos y del alcance presentados en el capítulo 1. Se clasifican en requisitos funcionales, que describen qué debe hacer el sistema; requisitos técnicos, que establecen las condiciones que debe cumplir el *hardware* y la comunicación; y restricciones de diseño, que delimitan cómo debe construirse.

3.1.1. Requisitos funcionales

Los usuarios del sistema (alumno, profesor, tutor y compañero) se definieron en el capítulo 1. A continuación se listan los requisitos que debe cumplir el dispositivo para atender a cada uno de ellos, agrupados según su función.

Interacción mediante la mirada:

- Acciones oculares: se detectarán seis acciones. Cuatro correspondientes a la mirada hacia cada una de las esquinas de la pantalla (arriba a la izquierda, arriba a la derecha, abajo a la izquierda y abajo a la derecha). Las otras dos serán la mirada hacia el centro y el parpadeo.
- Selección de respuestas: cada pregunta tendrá entre dos y cuatro opciones, asociadas a las esquinas de la pantalla. El parpadeo confirmará la opción elegida.
- Posición neutra: la mirada hacia el centro no ejecutará ninguna acción. Así, el alumno podrá corroborar su respuesta o descansar la vista.

Evaluaciones:

- Gestión de cuestionarios: creación, edición y administración de cuestionarios de opción múltiple personalizados.
- Resolución: el alumno debe ser capaz de resolver los cuestionarios únicamente con la mirada. El sistema registrará sus respuestas y los resultados obtenidos.

- Seguimiento académico: consulta de todos los cuestionarios, junto con las respuestas y los resultados del alumno, para realizar un seguimiento integral.

Comunicación con el alumno:

- Preguntas informales: envío de preguntas esporádicas de opción múltiple por parte de personas de su entorno, sin necesidad de registrarse.
- Sin persistencia: estas preguntas no se almacenarán en el sistema.

Usuarios y datos:

- Autenticación: registro e inicio de sesión de profesores y tutores.
- Permisos por rol: las funciones disponibles dependerán del rol de cada usuario.
- Base de datos local: almacenamiento de usuarios, cuestionarios y respuestas.

Acceso a la interfaz:

- Facilidad de uso: interfaz intuitiva y sencilla.
- Acceso universal: se debe poder utilizar desde cualquier dispositivo con navegador web, sin instalar *software* adicional.
- Comunicación local: los distintos bloques del sistema se comunicarán de forma inalámbrica, sin acceso a internet.

3.1.2. Requisitos técnicos

Los requisitos técnicos traducen las funciones anteriores en condiciones concretas para el *hardware* y la comunicación del sistema.

- Tiempo real: la detección debe responder con la rapidez suficiente para una interacción fluida. Por este motivo, la cámara tiene que ofrecer una tasa de fotogramas por segundo (FPS) adecuada.
- Robustez ante la iluminación: la detección no debe depender de las condiciones de luz del ambiente. Esta variable es incontrolable y cambia de manera constante.
- Captura de imagen: no se exige una cámara de alta resolución, ya que un exceso de datos puede saturar la memoria del microcontrolador. El campo de visión tampoco debe ser muy amplio, para evitar que estímulos ajenos a la pupila entorpezcan el análisis. Mediante el estudio empírico de imágenes con distintos ángulos, se determinó que la tarea requiere un ángulo de visión de entre 60 y 120 grados.
- Recursos limitados: el sistema debe funcionar con microcontroladores de memoria y capacidad de procesamiento acotadas. Esta condición es coherente con la restricción de costo.
- Conectividad inalámbrica: el microcontrolador que aloja la aplicación debe contar con conexión Wi-Fi. De este modo, se comunica con los dispositivos de los usuarios sin necesidad de acceso a internet.
- Independencia del sistema operativo: la interfaz debe ser una página web y no una aplicación instalable. Este tipo de programa depende del sistema operativo del dispositivo en el que se ejecuta. Una página web, en cambio, funciona en cualquier equipo con navegador, sea una *laptop*, un celular, una *tablet* o una computadora de escritorio.

3.1.3. Restricciones de diseño

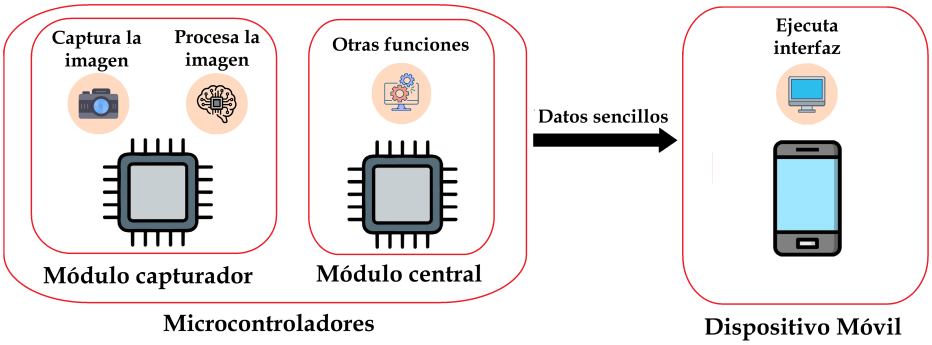
Las restricciones de diseño limitan la forma en que se construye el dispositivo, más allá de las funciones que debe cumplir.

- Costo: el dispositivo debe ser económico. Por este motivo, se emplean microcontroladores de bajo precio y fáciles de adquirir.
- Portabilidad: el sistema debe poder trasladarse sin depender de una instalación fija.
- Ergonomía: el módulo de captura debe montarse en la visera de una gorra común. Esta ubicación ofrece a la cámara un buen ángulo del ojo y un uso cómodo y seguro. Además, mantiene el sensor en una posición fija respecto al ojo.
- Autonomía energética: el dispositivo debe ser inalámbrico y autosuficiente al momento de utilizarlo. Para ello, cada módulo cuenta con su propia alimentación, independiente de cualquier fuente externa durante su funcionamiento.

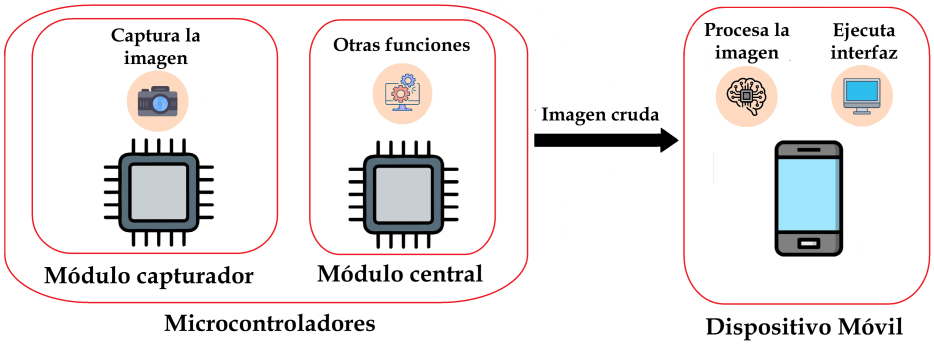
3.2. Detector ocular

El detector ocular es el componente central del sistema. Su función es determinar la dirección de la mirada del alumno a partir de la imagen de su ojo. Los métodos evaluados deben cumplir con dos condiciones fundamentales, que son la rapidez de respuesta y la precisión de la detección.

En esta sección se describe la evolución del detector. Primero se presentan los métodos evaluados de procesamiento local, que se ejecutan en el propio microcontrolador (Fig. 3.1a). Luego se exponen los de procesamiento remoto, que corren en el dispositivo cliente (Fig. 3.1b). Por último, se detalla la solución adoptada.



(a) Procesamiento local



(b) Procesamiento remoto

Figura 3.1: Diferencias entre procesamiento local y remoto

3.2.1. Procesamiento local como planteamiento inicial

En un principio se estableció que el procesamiento de las imágenes se realizaría en un microcontrolador, mediante una red neuronal artificial de tipo convolucional. Cuando la placa gestiona todas las tareas de detección, se habla de procesamiento local.

Antes de implementar una red neuronal se optó por intentar un método más sencillo. Se consideró que este tipo de modelo podía ser una complicación innecesaria para este problema. Este primer método se desarrolló en el *ESP32-CAM* [26], una variante de las placas de desarrollo *ESP32* que incluye un módulo para cámaras. En principio, este microcontrolador capturaría y procesaría las imágenes. Sin embargo, al no alcanzar la precisión requerida, se recurrió a las redes neuronales. Para poder ejecutarlas, se lo sustituyó por el *ESP32-S3* [27].

Esta última placa se eligió porque su procesador incorpora instrucciones vectoriales diseñadas para acelerar el cómputo de redes neuronales. Dichas instrucciones permiten procesar varios datos en una sola operación. *Espressif* las pone a disposición mediante la biblioteca *ESP-NN*, cuyas funciones optimizadas están orientadas a modelos cuantizados a números enteros de 8 bits (*int8*). La cuantización reduce el tamaño del modelo y el costo de cada operación. Por estos motivos, el *ESP32-S3* resulta una opción adecuada para ejecutar redes neuronales con recursos limitados.

Aun con esta aceleración, lograr que un microcontrolador ejecute una red neuronal de este calibre no es sencillo. La mayoría de estos modelos exige una capacidad de memoria y una velocidad que suelen exceder lo que ofrece el *ESP32-S3*. Por este motivo, se exploraron formas de diseñar y entrenar redes lo más livianas y sencillas posible. Este paradigma busca implementar inteligencia artificial con recursos de *software* y *hardware* limitados. Se lo conoce como *Tiny*

Machine Learning (o *TinyML*) y es la filosofía que se aplicó en las etapas iniciales del desarrollo de este dispositivo.

A continuación se desarrollan los tres métodos evaluados.

Umbral de oscuridad y brillantez

Este método de visión por computadora clásica consiste en capturar una imagen en escala de grises de forma continua para analizar sus patrones de oscuridad (intensidad lumínica) en tiempo real.

Al tomar una imagen del ojo, el punto más oscuro suele ser la pupila (Figura 3.2). El resto del cuadro presenta porciones irregulares con tonos grises más claros. El algoritmo aprovecha este contraste extremo para localizar las coordenadas pupilares y, a partir de ellas, inferir la dirección de la mirada.

Para entender este método, conviene aclarar antes cómo el microcontrolador procesa una imagen. Al capturar un fotograma, el *ESP32* lo almacena en un *framebuffer*, es decir, un bloque de memoria reservado y estructurado como una matriz bidimensional. En formato de escala de grises, cada elemento de esta matriz contiene un número entero entre 0 y 255. Este valor representa la intensidad lumínica del píxel en esa posición espacial (un 0 equivale a oscuridad absoluta y 255 a saturación blanca).

Con el fin de intensificar el contraste y eliminar posibles sombras indeseadas, el sistema incorpora un diodo emisor de luz infrarroja (LED IR) como fuente de iluminación complementaria. Debido a que su longitud de onda supera a la del espectro visible, esta radiación resulta imperceptible para el ojo humano, aunque los sensores fotográficos digitales sí logran captarla. Esta característica proporciona una iluminación constante que no encandila ni genera fatiga visual al usuario durante su funcionamiento.

Cabe destacar que el sensor del *ESP32-CAM* incluye de fábrica un filtro físico bloqueador de infrarrojos, por lo que fue necesario retirarlo manualmente. Tras este procedimiento, el LED IR se posicionó de forma adyacente a la lente de la cámara para asegurar una incidencia directa sobre la región ocular.



Figura 3.2: Distintas partes del ojo y su nivel de oscuridad

El algoritmo realiza un barrido inicial donde revisa cada píxel para identificar el valor de intensidad mínimo (el punto más oscuro del fotograma). Como depender de un único píxel comprometería la estabilidad del sistema, se establece un umbral de oscuridad dinámico. De esta manera, el programa no aísla un solo punto, sino toda una región cuyos valores caen por debajo de dicho límite.

Una vez delimitada la región de interés (*ROI*) que en teoría abarca la pupila, el programa ejecuta un segundo barrido. Este calcula las coordenadas centrales de la región mediante un centroide ponderado. En un promedio simple, se sumarían todas las coordenadas para luego dividirlas por la cantidad de píxeles detectados. En cambio, el centroide ponderado asigna a cada coordenada un peso en función de su nivel de oscuridad. Mientras más oscuro es un píxel, mayor resulta su influencia en la ecuación final. Esta ponderación es fundamental para la estabilidad del sistema: otorga un peso dominante al núcleo de la pupila (altamente oscuro) y reduce las desviaciones causadas por otros elementos captados dentro del umbral de detección (grises más claros). En la figura 3.3 se ilustra un ejemplo de este método en funcionamiento.

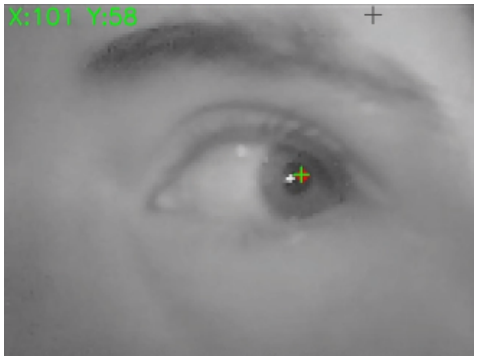


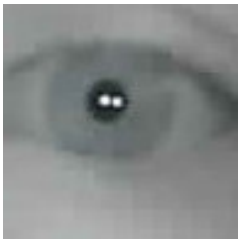
Figura 3.3: Detección por umbral de oscuridad

Este algoritmo es liviano y se ejecuta con rapidez en el *ESP32*. Sin embargo, al evaluar este método en la práctica, se descubrió que la precisión era insuficiente. La presencia de cejas, pestañas o armazones de anteojos confundía fácilmente al detector. Si bien la pupila se mantenía como uno de los puntos de menor intensidad lumínica, la gran similitud de tonos con estos elementos externos volvía inestable el seguimiento.

Ante este obstáculo, se decidió cambiar de enfoque. En lugar de aislar objetos por su nivel de oscuridad, se optó por aprovechar un fenómeno físico conocido como *efecto de brillo corneal*. Al apuntar una fuente de luz hacia el globo ocular (en este caso el LED IR), la superficie húmeda y curva de la córnea actúa como un espejo esférico. Esto hace que la luz rebote y forme un punto blanco concentrado e intenso en la imagen, denominado *glint* (Figura 3.4a).

Como la ubicación de este destello suele coincidir con la de la pupila, parecía una alternativa viable para reemplazar el algoritmo anterior por un detector de brillantez.

Esta solución tampoco resultó satisfactoria. Durante las pruebas de seguimiento, se comprobó que el *glint* no se desplazaba de la misma forma que la pupila. Su posición relativa dependía estrictamente de la geometría del ojo frente a la fuente de luz (Fig. 3.4b).



(a) En el centro de la pupila



(b) Desplazado respecto de la pupila

Figura 3.4: Posición del *glint* respecto de la pupila

Tras estos resultados, se descartó la visión por computadora clásica y se dio paso a las redes neuronales.

Edge Impulse

Un primer acercamiento al *TinyML* es la plataforma *Edge Impulse* [28]. Se trata de un entorno de desarrollo basado en la nube, orientado específicamente al aprendizaje automático con bajos recursos computacionales (Figura 3.5). Fue fundada en 2019 con el objetivo de facilitar esta tarea. Su función principal es trasladar el procesamiento desde servidores en la nube directamente al *hardware* local. Además, es una herramienta versátil ya que ofrece compatibilidad con el *framework* de *Arduino* [29].

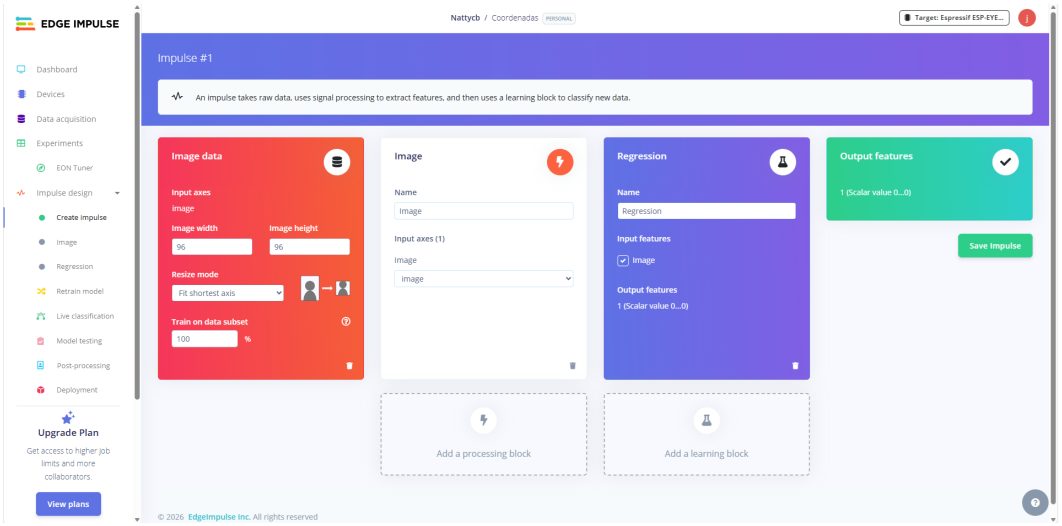


Figura 3.5: Pantalla de creación de modelos de *Edge Impulse*

Su funcionamiento se basa en una estructura de trabajo lineal (conocida como *pipeline* o *Impulse*). Este flujo unifica todas las etapas del desarrollo de un modelo predictivo. Comienza con la adquisición y el etiquetado de datos estructurados. Continúa con bloques de procesamiento digital de señales, que extraen las características más relevantes de las imágenes. Sigue con el entrenamiento de la red neuronal propiamente dicha y finaliza con el empaquetado del modelo. Todo esto se realiza mediante una interfaz de usuario intuitiva.

Gracias a sus técnicas de cuantización y a sus métodos de aceleración de *hardware*, resulta una opción prometedora que optimiza todo el proceso de aprendizaje. Funciona muy bien en tareas sencillas, como el reconocimiento de objetos o rostros (Figura 3.6).

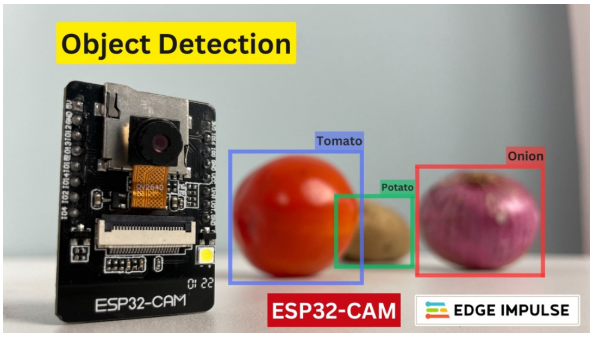


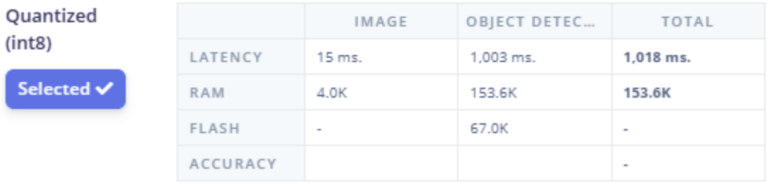
Figura 3.6: Reconocimiento de vegetales con *Edge Impulse*

La plataforma permite elegir entre distintas arquitecturas de red preentrenadas para la detección de objetos. En este proyecto se seleccionó *FOMO* (*Faster Objects, More Objects*), un algoritmo diseñado específicamente para dispositivos con recursos extremadamente limitados.

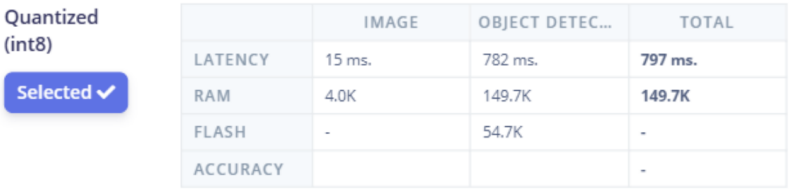
A diferencia de las redes tradicionales que calculan coordenadas geométricas complejas, esta arquitectura utiliza *MobileNetV2* como motor de extracción de características y luego aplica un enfoque más simple y eficiente: divide la imagen en una cuadrícula y evalúa cada celda de forma individual. En lugar de predecir el ancho y alto del objeto, el sistema actúa como un clasificador espacial que determina estadísticamente si cada celda corresponde al fondo de la imagen o si contiene el centro (centroide) de un objeto de interés. Esta simplificación matemática es lo que le permite ocupar menos de 100 KB de memoria y admitir entradas en escala de grises o RGB a cualquier resolución.

Bajo este esquema, se eligieron las variantes *MobileNetV2 0.1* y *MobileNetV2 0.35* como redes base para comparar su rendimiento. Si bien la confección del conjunto de datos y el etiquetado inicial se trabajaron mediante cuadros delimitadores (*bounding boxes*), la red extrae de ellos la posición central para su entrenamiento. Finalmente, los modelos resultantes fueron cuantizados al formato *int8*. Esta conversión fue fundamental para reducir el consumo de memoria y aprovechar al máximo la aceleración de *hardware* del *ESP32-S3*.

Antes de cargar cada modelo en la placa, *Edge Impulse* calcula un valor aproximado de su tiempo de inferencia en el *ESP32* clásico. Con *MobileNetV2 0.35*, ese valor rondaba 1 segundo (Fig. 3.7a), mientras que con *MobileNetV2 0.1* era cercano a 800 milisegundos (Fig. 3.7b).



(a) *MobileNetV2 0.35*



(b) *MobileNetV2 0.1*

Figura 3.7: Tiempo de inferencia estimado por *Edge Impulse* para el *ESP32* clásico

Solo se cargó en la placa el modelo basado en *MobileNetV2 0.35*. En el *ESP32-CAM*, que utiliza el *ESP32* clásico, cada inferencia demoró aproximadamente 1 segundo, en línea con la estimación. Luego, el mismo modelo se ejecutó en el *ESP32-S3*, donde el monitor serial registró alrededor de 200 milisegundos, es decir, un tiempo cinco veces menor. Esta diferencia es coherente con las instrucciones de cálculo vectorial que incorpora este microcontrolador.

El modelo basado en *MobileNetV2 0.1* no se llegó a probar en la placa. Si se mantuviera la misma proporción, su tiempo en el *ESP32-S3* rondaría los 160 milisegundos, un valor cercano al anterior. En ambos casos, la frecuencia resultante, de entre cinco y seis imágenes por segundo, resulta insuficiente para seguir la mirada en tiempo real. Por este motivo, se descartó esta alternativa.

TensorFlow Lite

Una herramienta muy utilizada en *Machine Learning* es el *framework* de código abierto *TensorFlow* [30] (*TF*). Está diseñado para entrenar y ejecutar modelos de redes neuronales propios o ya existentes. Sin embargo, no está optimizado para correr redes en sistemas con *hardware* limitado como los *ESP32*. Por este motivo, *Google* desarrolló una versión más liviana, llamada *TensorFlow Lite* (*TFLite*). Su variante específica para arquitecturas sin sistema operativo es *TensorFlow Lite for Microcontrollers* (*TFLM*).

Como siguiente paso, se optó por entrenar un modelo propio con imágenes capturadas directamente por el *ESP32-CAM*. A diferencia de *Edge Impulse*, que automatiza el proceso pero restringe las decisiones de diseño, este nuevo enfoque ofrece mayor libertad de personalización. Permite tener control sobre cada etapa del flujo de trabajo, como elegir individualmente las capas matemáticas, ajustar los parámetros de entrenamiento o definir la estrategia de compresión y exportación. Esta independencia permite adaptar el modelo a las limitaciones del microcontrolador y eliminar la sobrecarga de código genérico que suele imponer una plataforma cerrada.

Para facilitar la creación del modelo, *TensorFlow* incluye *Keras* [31], una API de alto nivel que simplifica la programación de la arquitectura neuronal. El entorno de desarrollo seleccionado fue *Google Colab* [32] (Figura 3.8). Se trata de una plataforma basada en la nube que permite ejecutar código *Python* en *notebooks* interactivos.

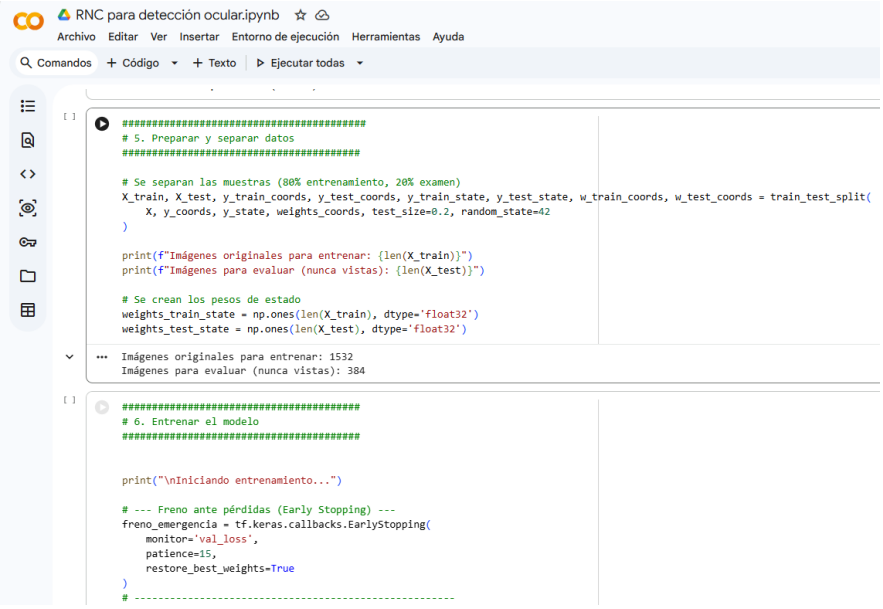


Figura 3.8: Entorno de *Google Colab*

Antes de empezar a codificar es necesario un *dataset* (conjunto de datos de entrada). Para una red de este tipo, normalmente se utilizan cientos de imágenes asociadas a lo que se denomina *etiquetas*. Una etiqueta representa el valor objetivo que el modelo debe predecir y aprender. En este proyecto, las etiquetas son números enteros fijos que representan las coordenadas espaciales (X e Y) del centro de la pupila.

Las imágenes de entrenamiento se capturaron con el módulo *ESP32-CAM*, en escala de grises (*Grayscale*) y con una resolución de 96x96 píxeles. Esta configuración se seleccionó para minimizar el consumo de recursos computacionales sin comprometer los detalles necesarios de la imagen. Para garantizar cantidad y variabilidad, se tomó un conjunto total de 1315 capturas de cinco personas distintas (Figura 3.9). Se contemplaron distintas condiciones de iluminación y variaciones físicas, como la presencia y la ausencia de anteojos.

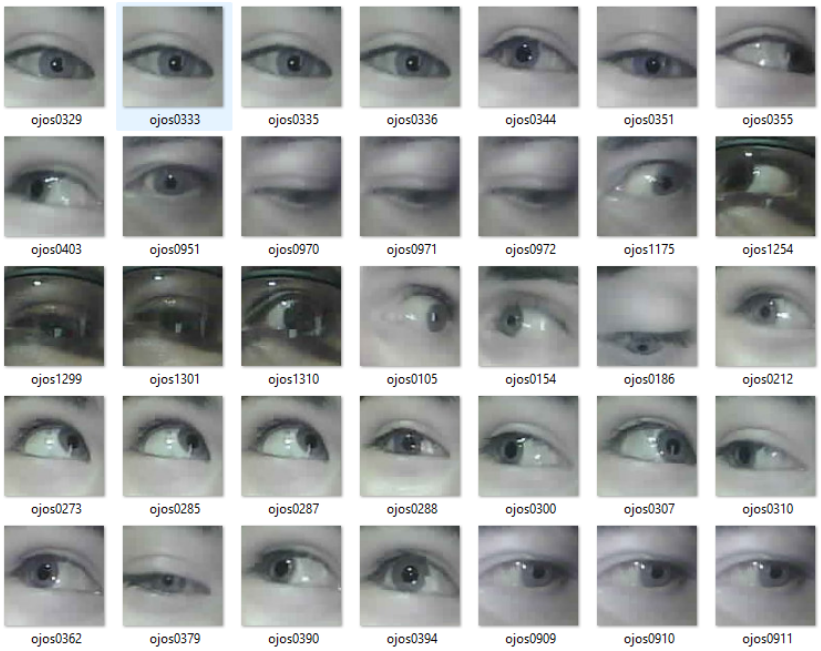


Figura 3.9: Una muestra de las imágenes utilizadas para el *dataset*

En paralelo, las coordenadas de cada imagen (expresadas en píxeles dentro de una imagen de 96x96) se anotaron en una hoja de cálculo. De este modo, los datos quedaron estructurados para su posterior utilización en el entrenamiento (Figura 3.10).

labels

Archivo Editar Ver Insertar Formato Datos Herramientas G

Menús 100% € % .0 .00 123

K7

	A	B	C	D	E
1	filename	x	y		
2	ojos0001	41	47		
3	ojos0002	41	47		
4	ojos0003	41	47		
5	ojos0004	41	47		
6	ojos0005	41	47		
7	ojos0006	41	47		
8	ojos0007	41	47		
9	ojos0008	43	46		
10	ojos0009	24	48		
11	ojos0010	18	48		
12	ojos0011	18	48		
13	ojos0012	18	48		
14	ojos0013	18	48		
15	ojos0014	18	48		
16	ojos0015	50	28		
17	ojos0016	50	28		
18	ojos0017	50	28		
19	ojos0018	50	28		
20	ojos0019	50	28		
21	ojos0020	50	28		

Figura 3.10: Planilla con etiquetado de imágenes

A continuación se listan y explican brevemente las etapas de la creación de un modelo de Red Neuronal Convolucional (CNN) en *Google Colab* con *Python/Keras*:

1. Preparación: se importan en el código las herramientas necesarias para construir la CNN, entre ellas *TensorFlow*, *Pandas*, *NumPy* y *OpenCV*. Se monta *Google Drive* para que el entorno acceda a las imágenes y etiquetas almacenadas allí.
2. Formateo del *dataset*: se extraen de *Google Drive* las 1315 imágenes y se leen y almacenan las etiquetas en arreglos.
3. Creación de la red: se define el formato de las entradas, las capas internas y la función de activación. Al final se compila el modelo y se establece cómo trabajará con los errores.
4. Separación de datos: aquí se aplica una técnica para entrenar y evaluar la red al mismo tiempo. Se dividen las imágenes del *dataset* en dos grupos: el de entrenamiento (80 % de las imágenes) y el de evaluación (20 % restante). Esta separación es aleatoria y sirve para comprobar el resultado de la construcción del modelo.
5. Entrenamiento: se ejecuta el aprendizaje de la red. Se elige el número de *épocas* (veces que el modelo analiza el grupo de imágenes) y se aplican *callbacks* (funciones que detienen el entrenamiento cuando se alcanza la precisión deseada). Con esto último se evita que la red memorice fotos en lugar de aprender de ellas.
6. Evaluación: se pone a prueba la red con la porción de datos que nunca vio durante el entrenamiento. Se muestran los resultados y el análisis de distintos parámetros estadísticos, como el porcentaje de aciertos, los errores y la muestra con peor desviación.
7. Exportación: se guarda el modelo en un archivo con formato `.tflite`. Este se utiliza luego en *Visual Studio Code* (con la extensión *PlatformIO* [33]), el entorno en el que se programa el microcontrolador.

Se iteró sobre la arquitectura de la red y se ajustaron sus hiperparámetros (la cantidad de capas y los filtros de procesamiento, entre otros) para maximizar la precisión (Figura 3.11). Luego, el modelo se exportó y se compiló en el entorno de C++ para evaluar su rendimiento en vivo.

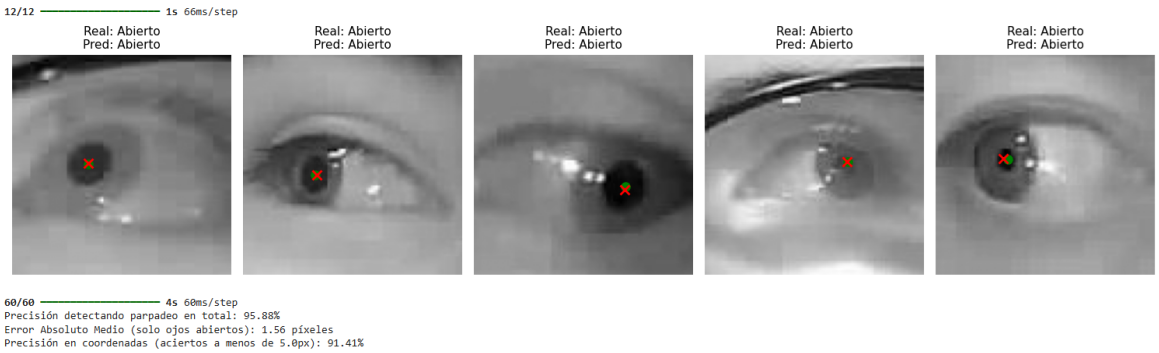


Figura 3.11: Resultados del modelo desarrollado en *Google Colab*. Con un punto verde se indica el valor etiquetado, mientras que la cruz roja muestra la posición que la red predice

Este modelo resultó significativamente más rápido, eficiente y controlable que el generado con *Edge Impulse*. Sin embargo, las pruebas empíricas en tiempo real mostraron que no cumplía los requisitos de precisión para el dispositivo final. Durante la operación, las coordenadas predichas eran muy inestables. Oscilaban de forma pequeña y constante (*jittering*) y no lograban mantenerse en un valor firme. En la figura 3.12 se muestra un ejemplo de estos errores de precisión. Aunque se trata de pocos píxeles, el error resulta significativo cuando se busca un objeto tan pequeño como la pupila.

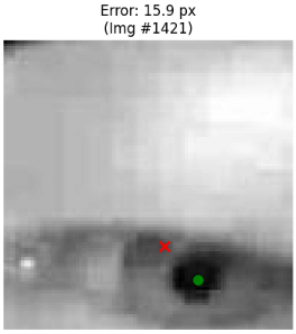


Figura 3.12: Error de precisión con *TFLite* (15,9 píxeles)

Para reducir esa inestabilidad, se intentó aumentar la complejidad y la profundidad de la red neuronal. Sin embargo, esto elevó la latencia de procesamiento a niveles inaceptables. Por el contrario, simplificar la arquitectura para recuperar velocidad degradaba la exactitud de las coordenadas. Se encontró así un límite de *hardware* insalvable.

Conclusión del procesamiento local

Ninguno de los tres métodos cumplió las dos condiciones establecidas inicialmente. El umbral de oscuridad y brillantez no alcanzó la precisión requerida. *Edge Impulse* resultó demasiado lento para el tiempo real. El modelo propio con *TensorFlow Lite* no logró estabilidad en sus predicciones y, al aumentar su complejidad, perdió velocidad. Por lo tanto, el procesamiento de imágenes de forma puramente local, dentro de las limitaciones de memoria del microcontrolador, no resulta viable para las exigencias de este proyecto. Esta restricción motivó el descarte de las redes neuronales en *hardware* de bajos recursos.

3.2.2. Procesamiento remoto por coordenadas

Tras descartar el procesamiento local, se decidió trasladar la detección a un dispositivo externo. Este equipo es el mismo desde el cual el usuario utiliza el sistema, como una PC, un celular o una *tablet*. Cuando la detección se ejecuta en este dispositivo cliente, y no en el microcontrolador, se habla de procesamiento remoto.

Este enfoque cambia la distribución de tareas. El microcontrolador ya no procesa las imágenes. Solo las captura y las transmite al dispositivo cliente por Wi-Fi. Este último recibe los fotogramas y ejecuta sobre ellos el método de detección. La transmisión de imágenes se detalla en la sección 3.5.3.

La principal ventaja de este esquema es la capacidad de procesamiento disponible. Un dispositivo cliente cuenta con mucha más memoria y velocidad que un microcontrolador. Por lo tanto, ya no es necesario reducir los métodos de detección a modelos livianos como los de *TinyML*. Esto permite evaluar herramientas más complejas, que resultaban inviables en el *ESP32-S3*. Como contrapartida, el sistema pasa a depender de la transmisión continua de imágenes entre ambos dispositivos.

Al mover la detección al dispositivo cliente, la aceleración de redes neuronales que ofrece el *ESP32-S3* dejó de ser necesaria. Por lo tanto, se volvió a utilizar el *ESP32-CAM* para la captura de imágenes.

A continuación se desarrollan los dos métodos evaluados, que son las redes neuronales pre-entrenadas *MediaPipe* [34] y *WebGazer* [35], y *OpenCV* con la Transformada de Hough [36, 37].

MediaPipe y WebGazer

La mayor capacidad de procesamiento del dispositivo cliente permite utilizar una red neuronal preentrenada para la detección de la mirada. Dos alternativas consideradas fueron *MediaPipe* (un conjunto de bibliotecas provisto por *Google*) y *WebGazer* (un modelo desarrollado en el ámbito académico). Ambas son herramientas basadas en tecnologías web que evitaban las limitaciones de procesamiento del microcontrolador. Sin embargo, fueron descartadas debido a dos inconvenientes.

Por un lado, las dos requieren que la imagen abarque el rostro completo. *MediaPipe* identifica puntos de referencia en toda la cara (Fig. 3.13). *WebGazer*, si bien está diseñado específicamente para estimar la mirada, ubica los ojos mediante el detector facial de *MediaPipe* (Fig. 3.14). En consecuencia, en ambos casos los ojos ocupan una cantidad reducida de píxeles. Para medir con precisión sus variaciones, se requiere una mayor resolución de imagen.

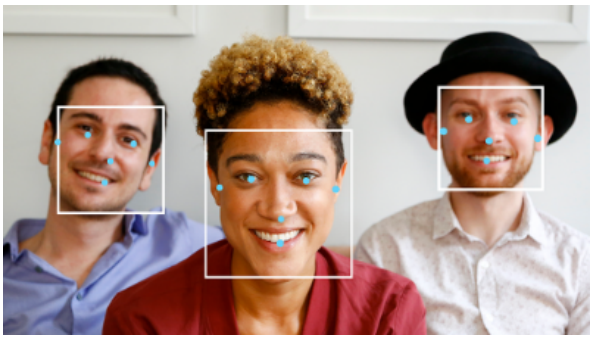


Figura 3.13: Detección de rostro con *MediaPipe*

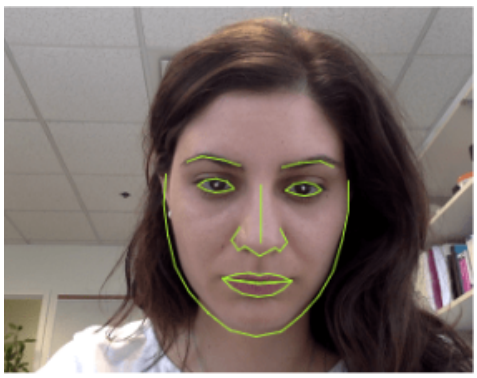


Figura 3.14: Facciones del rostro en *WebGazer*

Esta mayor resolución puede obtenerse de dos formas. La primera es aumentar la resolución de captura del *ESP32-CAM*, lo que implica un mayor tráfico de imágenes entre el microcontrolador y el dispositivo cliente. La segunda es derivar la captura a la cámara integrada del dispositivo cliente. Sin embargo, esto haría que la detección dependiera del equipo utilizado. Cada equipo cliente posee una cámara con propiedades distintas, cuya resolución, ubicación, distancia focal e inclinación varían de un modelo a otro. Todas estas son variables incontrolables que afectarían la precisión del seguimiento espacial. Además, las cámaras integradas incluyen un filtro que atenúa considerablemente la luz infrarroja, y retirarlo no es viable en un equipo de uso cotidiano. Por lo tanto, registrarían la iluminación del LED IR con poca intensidad. Así, la imagen quedaría más expuesta a las sombras y a la luz ambiente. Un dispositivo propio, que mantiene siempre su cámara en una posición fija respecto al ojo y permite retirar ese filtro, evita estos problemas.

Por otro lado, durante las pruebas, el cursor que indicaba en pantalla la posición estimada de la mirada vibraba de forma constante. Estas vibraciones se debían a los micromovimientos naturales del ojo. Como resultado, el seguimiento no resultaba estable.

Por estos motivos empíricos y de diseño, se decidió prescindir definitivamente de las redes neuronales como método de detección espacial.

OpenCV y Transformada de Hough

Tras descartar las redes neuronales, se evaluó *OpenCV*, la biblioteca de visión por computadora descrita en el capítulo 2. A diferencia de *MediaPipe* y *WebGazer*, sus funciones pueden aplicarse directamente sobre una imagen del ojo sin necesidad de detectar el rostro completo. Por este motivo, se perfiló como una alternativa viable para este trabajo.

Las pruebas de este método se realizaron con un *script* de *Python* ejecutado en una computadora, que procesaba las imágenes capturadas por el *ESP32-CAM*. Este *script* funcionaba como prototipo. En caso de obtener buenos resultados, el detector se iba a trasladar a *OpenCV.js*, la versión de la biblioteca para navegadores web, que se ejecutaría en el dispositivo del usuario.

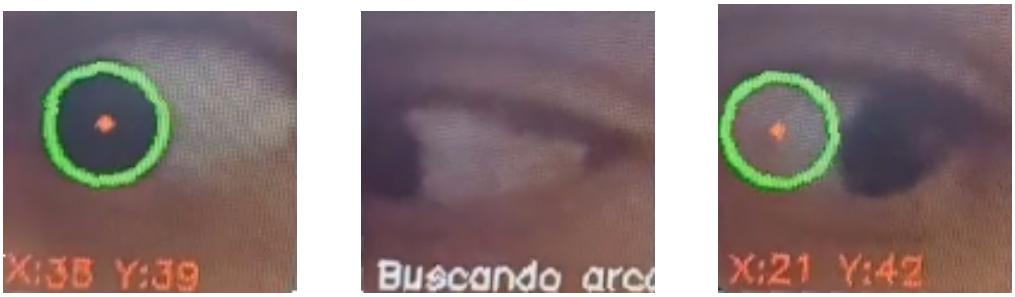
Para aprovechar al máximo las capacidades de *OpenCV*, se decidió complementarlo con la Transformada de Hough, un método clásico de extracción de características en el análisis de imágenes digitales. Su variante para círculos, conocida como *HoughCircles* en *OpenCV*, analiza los bordes detectados en un fotograma mediante un sistema matemático de votación. El algoritmo busca conjuntos de píxeles que compartan una curvatura y se encuentren a una distancia constante (el radio) de un posible punto central. Si suficientes píxeles votan por el mismo centro, el algoritmo confirma la existencia de un círculo en esa región. En este contexto, el círculo corresponde al iris del ojo, y las coordenadas de la pupila se obtienen al calcular la ubicación del centro de dicho círculo.

Una vez recibido el fotograma, *OpenCV* realiza un filtrado espacial (preprocesamiento) para limpiar la imagen de ruido digital y prepararla para el análisis. A continuación se aplica la función de círculos de Hough. Cuando se detecta el contorno del iris, los resultados se grafican sobre la imagen a modo de depuración visual. Un trazo verde delimita la circunferencia detectada y un punto rojo en su centro representa las coordenadas de la pupila (Fig. 3.15a).

Aquí surge un problema importante. La función de círculos de Hough está diseñada para detectar circunferencias perfectas, y el iris humano no siempre se presenta como tal frente a la cámara. Con frecuencia, los párpados ocluyen una parte superior o inferior y alteran su geometría circular (Fig. 3.15b). Esto implicaría que el usuario debería forzar permanentemente sus músculos oculares para mantener los ojos abiertos de forma antinatural y así garantizar el correcto funcionamiento del sistema. Esta situación resulta poco práctica para el alumno.

Una posible solución consiste en disminuir la exigencia de *OpenCV* para que tolere ciertas deformaciones, como arcos incompletos o formas elípticas, y no detecte únicamente círculos perfectos. Sin embargo, esto genera un problema secundario grave. El algoritmo se vuelve demasiado permisivo y registra falsos positivos al detectar curvas en párpados, cejas, arrugas y otras facciones del rostro.

Ambos problemas presentan un compromiso técnico recíproco, ya que si se intenta reducir uno, el otro empeora. Se realizaron sucesivas pruebas con herramientas complementarias, como umbrales de radio y detectores de brillantez y oscuridad apoyados por el LED infrarrojo. Aun así, la precisión alcanzada no resultó adecuada para un seguidor ocular en tiempo real (Fig. 3.15c), por lo que esta solución se descartó.



(a) Detección correcta (b) Fallo por oclusión (c) Falso positivo

Figura 3.15: Detección del iris con el método de círculos de Hough

Conclusión del procesamiento remoto por coordenadas

Ninguno de los dos métodos alcanzó un resultado satisfactorio. *MediaPipe* y *WebGazer* requerían una mayor resolución de imagen, que no podía obtenerse sin comprometer el diseño del dispositivo, y su estimación de la mirada resultaba inestable. Con *OpenCV* y la Transformada de Hough no se logró detectar el iris con precisión, ya que la oclusión de los párpados y los falsos positivos presentaban un compromiso que no pudo resolverse.

Las dos alternativas compartían un mismo enfoque, que consistía en detectar coordenadas espaciales específicas del ojo. Por este motivo, se cambió el enfoque del detector. Se dejó atrás la detección de coordenadas y se pasó a identificar patrones de orientación de la mirada. Este cambio busca simplificar el detector y eliminar las causas de error encontradas en los métodos anteriores. El objetivo es lograr un algoritmo independiente de la luz ambiental, de la resolución de la cámara, del nivel de oscuridad de los elementos del rostro y de las variaciones en la forma del ojo. Esta estrategia se describe en la siguiente subsección.

3.2.3. Template Matching como solución adoptada

Para identificar la orientación de la mirada se utilizó *Template Matching*, técnica descrita en el capítulo 2. En este caso, las plantillas son imágenes del ojo del alumno, una por cada una de las seis acciones posibles. Como cada dispositivo está destinado a un único estudiante, las plantillas corresponden siempre a su propio ojo.

Con este método, las sombras que se forman sobre el ojo tienden a ocultar la pupila dentro de una región oscura uniforme. Esto le dificulta al sistema distinguir hacia dónde se mueve. Por este motivo, se vuelve a incorporar el LED IR. Su luz infrarroja borra las sombras indeseadas, garantiza una iluminación uniforme y mejora el contraste de la imagen (Fig. 3.17a y 3.17b).

El algoritmo se ejecuta en el navegador del dispositivo cliente mediante *OpenCV.js*. De esta forma, el detector no requiere instalar ninguna aplicación adicional en el equipo del usuario.

A continuación se describen las tres etapas del método, que son la calibración, el preprocesamiento de fotogramas y la comparación de plantillas.

Calibración

El método requiere, como paso inicial, un proceso de calibración que proporcione las plantillas. Durante esta etapa se captura una imagen del ojo del alumno para cada una de las acciones posibles dentro de la interfaz. Para garantizar la estabilidad del algoritmo y reducir las confusiones entre plantillas, estas acciones se limitaron a seis (Fig. 3.16). A continuación se detalla la plantilla que se obtiene para cada una.

- Centro: imagen del ojo con la mirada hacia el centro de la pantalla. Corresponde a la zona neutra, en la que el alumno puede descansar la vista sin realizar ninguna acción. Si antes resaltó una opción, esta se conserva hasta que la confirma o mira hacia otra.
- Esquina superior izquierda: imagen del ojo con la mirada hacia arriba a la izquierda, asociada a la opción 1.
- Esquina superior derecha: imagen del ojo con la mirada hacia arriba a la derecha, asociada a la opción 2.
- Esquina inferior izquierda: imagen del ojo con la mirada hacia abajo a la izquierda, asociada a la opción 3.
- Esquina inferior derecha: imagen del ojo con la mirada hacia abajo a la derecha, asociada a la opción 4.
- Parpadeo: imagen del ojo cerrado. Permite confirmar la opción elegida después de mirar hacia una de las esquinas.



Figura 3.16: Plantillas para comparación

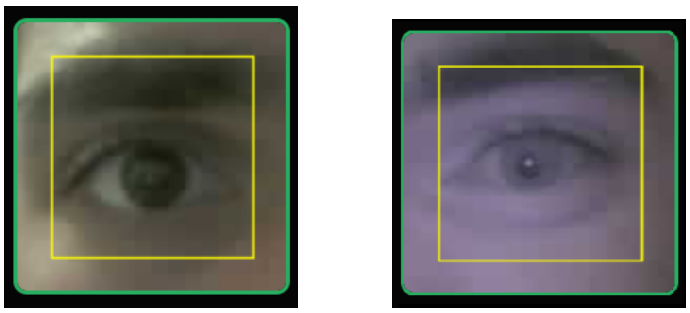
Cada plantilla se guarda con una resolución de 96x96 píxeles en escala de grises. Luego se almacenan en el navegador, para ser utilizadas como referencia al comparar cada fotograma durante la detección. Allí se conservan hasta que se borren los datos de navegación, por lo que permanecen disponibles aunque se cierre la pestaña o se reinicie el dispositivo.

Preprocesamiento de fotogramas

Con el sistema en funcionamiento, el *ESP32-CAM* captura el ojo en tiempo real con una resolución de 320x240 píxeles (*QVGA*), en color, y comprime cada fotograma en formato JPEG. Los motivos de esta configuración se detallan en la sección 3.5.3.

Luego de tomar la imagen, se envía al dispositivo cliente a través de la red local. Allí se realiza todo el procesamiento de las imágenes.

Antes de la comparación, cada fotograma se acondiciona. Primero se recorta internamente a un tamaño de 128x128 píxeles (región de interés), y luego se cambia su formato a escala de grises. Ambos pasos reducen la carga del procesamiento posterior. El recorte es más grande que las plantillas (Fig. 3.17) por un motivo que se explica en la siguiente etapa. Con el mismo fin, la tasa de análisis se limita a 15 fotogramas por segundo (FPS). Experimentalmente, esta frecuencia resultó suficiente para lograr una interacción fluida en tiempo real sin sobrecargar el dispositivo cliente.



(a) Sin LED infrarrojo (b) Con LED infrarrojo

Figura 3.17: Imagen de 128x128 píxeles con un recuadro de 96x96

Comparación de plantillas

Cada fotograma acondicionado se compara con las seis plantillas mediante el *coeficiente de correlación normalizado con media cero*, descrito en el capítulo 2.

Como se observa en el numerador de la fórmula, a cada píxel se le resta el valor promedio de su matriz. Por este motivo, el coeficiente recibe el nombre de media cero. Así, la fórmula no evalúa la intensidad lumínica absoluta de cada píxel, sino cuánto se diferencia del promedio de la imagen. Por ejemplo, si el rostro del alumno se ilumina de forma repentina por una fuente externa, todos los píxeles aumentan su valor, pero la diferencia relativa entre ellos se mantiene. De esta forma, el método resulta inmune a las variaciones uniformes de la iluminación ambiental.

La plantilla mide 96x96 píxeles y el fotograma 128x128, aunque en ambos casos el ojo tiene el mismo tamaño. Esta diferencia permite que la plantilla recorra el fotograma mediante el barrido explicado en el capítulo 2, en lugar de compararse en una única posición.

El barrido compensa los errores de posicionamiento entre el ojo y la cámara. Si la comparación se realizara en un solo punto de 96x96, un desplazamiento mínimo del dispositivo, incluso de un milímetro, desalinearía las matrices y haría fallar el cálculo. Al recorrer todo el fotograma, el método obtiene un margen para compensar estas variaciones involuntarias. Así, el sistema tolera los pequeños movimientos de la cabeza del alumno.

De la matriz de resultados de cada plantilla se toma el coeficiente más alto, que corresponde a la posición de mayor coincidencia. Este proceso se repite con las seis plantillas. La plantilla con el mayor de los seis valores indica la acción que realiza el alumno.

En las pruebas también se detectaron falsos positivos. En ocasiones, el algoritmo resultaba demasiado permisivo y confundía las orientaciones del ojo. Para reducir este efecto se incorporó un umbral de similitud, por debajo del cual toda coincidencia se descarta. A partir de los ensayos, el umbral se fijó por defecto en un 80 %, y el usuario puede ajustarlo desde la interfaz web (Fig. 3.18). Con esta medida, el problema quedó resuelto.

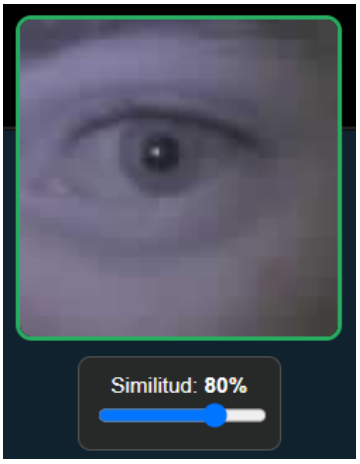


Figura 3.18: Video en vivo con el umbral de similitud

Tras numerosas evaluaciones, este método basado en la similitud de patrones resultó el más efectivo, estable y eficiente de los evaluados. Por lo tanto, es la solución implementada en el dispositivo final.

3.3. Arquitectura del sistema

El sistema definitivo se compone de tres tipos de elementos que se comunican a través de una red Wi-Fi local. El módulo de captura, montado sobre una gorra, obtiene las imágenes del ojo del alumno y las envía a su dispositivo. El módulo central genera la red, aloja la aplicación web y almacena la información del sistema. Además, cuenta con una pantalla táctil que permite configurar el dispositivo y consultar su estado sin depender de otro equipo. Por último se encuentran los dispositivos cliente, a través de los cuales los distintos usuarios acceden a la aplicación web desde su navegador. Solo uno de ellos corresponde al alumno, y en él se realiza la detección de la mirada (Fig. 3.19).

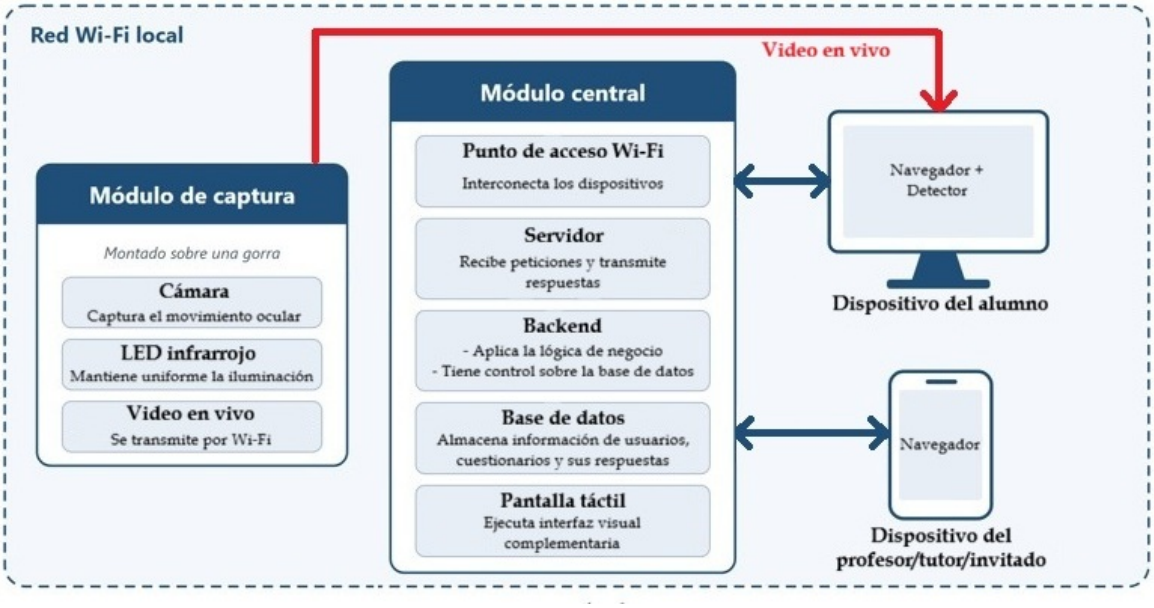


Figura 3.19: Diagrama de bloques del sistema definitivo

Esta distribución permite que cada elemento cumpla una función específica. A continuación se describen las funciones de cada uno y la forma en que interactúan entre sí. Por último, se presenta el mapa de memoria del módulo central, que muestra dónde se almacena cada tipo de información.

3.3.1. Funciones de cada módulo

Módulo de captura

La función del módulo de captura es obtener imágenes del ojo del alumno en tiempo real y enviarlas de forma continua a su dispositivo. No realiza ningún procesamiento sobre ellas, ya que la detección de la mirada ocurre en el navegador. Además, ilumina el ojo con luz infrarroja, lo que evita las sombras que generan las fuentes de luz externas y mantiene constantes las condiciones de captura.

Módulo central

El módulo central coordina el funcionamiento de todo el sistema. En primer lugar, genera la red Wi-Fi local a la que se conectan los demás elementos. Así, el conjunto funciona sin acceso a internet. En segundo lugar, actúa como servidor. Entrega las páginas de la aplicación web y la biblioteca *OpenCV.js* a cada dispositivo que las solicita y responde a sus pedidos de información.

Además, funciona como el núcleo de datos del sistema. Se encarga de autenticar a profesores y tutores, controlar las funciones disponibles para cada rol y administrar los cuestionarios. Durante las evaluaciones, recibe y registra las respuestas del alumno en una base de datos local. De esta forma, toda la información queda unificada en un único lugar, al cual los distintos usuarios acceden desde sus dispositivos.

Por último, el módulo central cuenta con una interfaz propia en su pantalla táctil. Desde ella se realiza la configuración inicial del dispositivo, se muestra un código QR para acceder a la aplicación web y se consulta el estado del sistema y el progreso de los exámenes.

Dispositivos cliente

Los dispositivos cliente son la vía de acceso de los usuarios al sistema. Cualquier equipo con navegador web puede cumplir esta función, sin instalar *software* adicional. Todos utilizan la misma aplicación web, pero la función de cada dispositivo depende de quien lo opere.

Desde estos dispositivos, los profesores y tutores gestionan los cuestionarios y realizan el seguimiento de los resultados del alumno, según los permisos de cada rol. Además, cada uno puede editar sus datos personales. Los profesores, a su vez, acceden a los datos de contacto de los tutores, mientras que los tutores consultan los de los profesores.

El dispositivo del alumno cumple un papel particular, ya que también realiza la detección de la mirada. Recibe el video del módulo de captura, lo compara con las plantillas de la calibración y traduce cada acción del ojo en una selección dentro de la interfaz. Así, el alumno puede resolver cuestionarios y responder preguntas solo con la mirada.

Por su parte, los compañeros acceden como invitados, sin registrarse, y pueden enviar preguntas informales al alumno. Estas preguntas no se guardan en la base de datos.

3.3.2. Interacción entre módulos

Toda la comunicación ocurre dentro de la red Wi-Fi que genera el módulo central. A través de ella, los módulos intercambian dos tipos de información. Por un lado, el video del ojo del alumno, que se transmite de forma continua. Por otro lado, los datos del sistema, como los cuestionarios, las respuestas y el estado de cada elemento.

El video viaja desde el módulo de captura hasta el dispositivo del alumno, que lo solicita solo durante la calibración y el examen. Como punto de acceso, el módulo central hace circular esta transmisión por su red, pero no la procesa ni la almacena.

Los datos del sistema, en cambio, son procesados por el módulo central. Al conectarse, cada dispositivo cliente descarga desde él las páginas de la aplicación web. El dispositivo del alumno descarga además la biblioteca *OpenCV.js*, que conserva en el navegador para los accesos siguientes. Luego, cada vez que un usuario realiza una acción, su dispositivo envía la información al módulo central, que la gestiona y la pone a disposición de los demás.

Durante un examen, este intercambio ocurre de forma continua. El alumno confirma cada respuesta con un parpadeo y su dispositivo la envía en ese momento al módulo central. Este la registra en la base de datos y actualiza el progreso del examen.

Las preguntas de los compañeros también pasan por el módulo central, pero no se guardan en la base de datos. En lugar de eso, se conservan temporalmente en la memoria de trabajo y se entregan al dispositivo del alumno.

3.3.3. Mapa de memoria

El módulo central, basado en un microcontrolador de la familia *ESP32*, dispone de cuatro memorias con características distintas (Fig. 3.20). La ubicación de cada tipo de información responde a tres criterios. El primero es si debe conservarse al apagar el equipo. El segundo es con qué frecuencia cambia. El tercero es cuánto espacio ocupa.

La memoria flash retiene su contenido sin alimentación, pero su acceso es lento y su capacidad es reducida. Por eso guarda información persistente, que rara vez cambia y que ocupa poco espacio. Se divide en tres partes, cada una destinada a un tipo de dato distinto.

- NVS (*Non-Volatile Storage*): almacena las claves maestras de profesores y tutores y el nombre del alumno. Estos datos se definen una única vez, durante la configuración inicial, y se consultan al encender el equipo. La NVS está diseñada para guardar pocos valores de pequeño tamaño, por lo que resulta adecuada para esta configuración.
- *Firmware*: contiene el código ejecutable del sistema, que solo cambia al actualizar el programa.
- *Filesystem*: aloja las páginas de la aplicación web. Son archivos que no cambian durante el uso y que el módulo central entrega tal como están. Como son livianos, pueden ubicarse en la flash.

La SRAM interna es rápida, aunque tiene un tamaño limitado y se borra al apagar el equipo. Funciona como memoria de trabajo, ya que guarda información que cambia de forma constante, que debe leerse con rapidez y que solo tiene sentido mientras el sistema está encendido.

- IRAM: contiene el código de las interrupciones y de las funciones críticas, que deben ejecutarse con rapidez.

- DRAM: almacena los datos que el sistema solo necesita mientras está encendido. Entre ellos se encuentran las sesiones activas con sus *tokens*, que cambian cada vez que un usuario ingresa o sale del sistema y que el módulo central consulta en cada pedido. Al reiniciarse el equipo, los usuarios vuelven a ingresar. También guarda las preguntas de los invitados, que son informales y no se registran en la base de datos. Además, contiene las variables de control y los bloques pequeños de memoria dinámica que el sistema solicita durante su funcionamiento.

La PSRAM es una memoria externa al chip, más lenta que la SRAM interna pero de mayor capacidad, y también se borra al apagar el equipo. Se utiliza para ampliar la memoria de trabajo. Con la configuración por defecto del entorno de desarrollo, el sistema ubica en ella de forma automática los bloques de memoria dinámica de más de 4 KB, lo que libera la SRAM interna para los datos que requieren mayor velocidad.

- Caché de la base de datos: guarda las partes de la base de datos que se consultan con mayor frecuencia, para no leerlas de la microSD en cada pedido. Ocupa 512 KB, un tamaño que no podría reservarse en la SRAM interna.

La tarjeta microSD no pierde los datos ante la ausencia de alimentación y ofrece un espacio mucho mayor que la flash. Por eso guarda la información que debe mantenerse a largo plazo y que no cabe en esta.

- Base de datos: almacena los usuarios, los cuestionarios y las respuestas del alumno. Esta información debe mantenerse tras el apagado y crece con el uso, a medida que se crean cuestionarios y se registran respuestas. Por eso requiere una memoria permanente y de gran capacidad.
- *OpenCV.js*: contiene la biblioteca de procesamiento de imágenes que se entrega al dispositivo del alumno. Su tamaño excede la capacidad del *Filesystem* de la flash, a diferencia de las páginas de la aplicación web, que son más livianas.

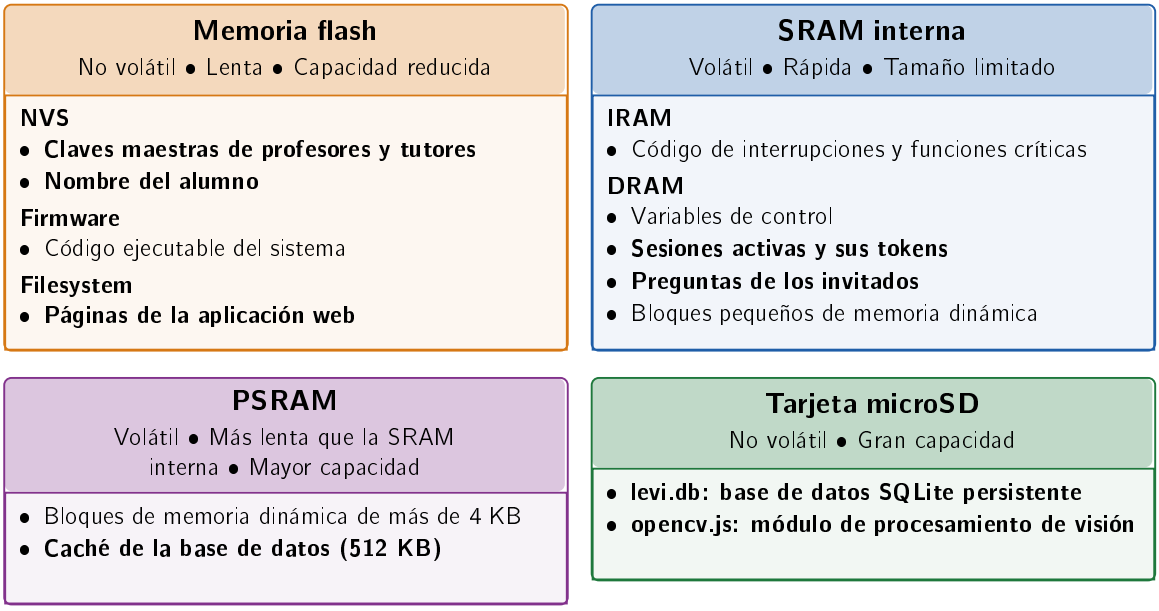


Figura 3.20: Mapa de memoria del módulo central

3.4. Microcontroladores y periféricos

Tanto el módulo de captura como el módulo central se implementan con una placa de desarrollo basada en un microcontrolador. La elección de estas placas responde a las restricciones del proyecto, que exigían que fueran económicas y fáciles de conseguir. Además, ambas necesitaban conectividad Wi-Fi, ya que toda la comunicación del sistema ocurre de forma inalámbrica. El módulo de captura requería, a su vez, una interfaz física para conectar y manipular una cámara.

En esta sección se justifica la elección de la familia *ESP32* frente a otras alternativas del mercado. Luego se describen las placas utilizadas en cada módulo. Para el módulo de captura se detalla también la elección de la cámara. Para el módulo central se requería, además, una ranura para tarjeta microSD, y se optó por una placa con pantalla táctil integrada.

3.4.1. Elección de la familia ESP32

Para implementar los dos módulos se compararon distintas placas de desarrollo. Los criterios de evaluación fueron el costo, la conectividad Wi-Fi integrada y la facilidad de adquisición. También se consideró que cada módulo necesita periféricos específicos. El módulo de captura requiere una cámara, y el módulo central, un medio para almacenar la base de datos.

Se optó por la familia *ESP32*, de la empresa *Espressif Systems*, cuyas características se describieron en el capítulo 2. Sus placas integran la conectividad Wi-Fi en el mismo chip y tienen un costo de entre 6 y 25 USD a octubre de 2026. Además, se consiguen con facilidad, tanto en el mercado local como a través de tiendas internacionales. La familia también ofrece variantes que reúnen en un solo dispositivo los periféricos que necesita cada módulo, como una cámara, una pantalla o una ranura para tarjeta microSD. Esto permitió implementar ambos con microcontroladores de la misma familia y, por lo tanto, con las mismas herramientas de desarrollo. Los dos se programaron sobre el *framework* de *Arduino*, en *Visual Studio Code* con la extensión *PlatformIO*. Se eligió esta extensión porque facilita la estructuración del código y, al compilar, procesa únicamente los archivos modificados, lo que reduce el tiempo de cada prueba.

A continuación se detallan las alternativas consideradas y los motivos de su descarte. Los precios corresponden a las tiendas oficiales de cada fabricante a octubre de 2026.

- *Arduino Uno*: es una de las placas más difundidas, pero no cuenta con conectividad Wi-Fi. Además, sus 2 KB de memoria RAM resultan insuficientes para alojar una aplicación web.
- *Arduino UNO R4 WiFi*, *Arduino Nano 33 IoT* y *Arduino Nano ESP32*: incorporan Wi-Fi y cuentan con el respaldo de la marca *Arduino*. Sin embargo, su costo, de entre 28 y 50 USD, es mayor que el de las placas de *Espressif*. Además, ninguna integra una interfaz para cámara ni una ranura para tarjeta microSD.
- *Raspberry Pi Pico 2 W*: incorpora Wi-Fi y tiene un costo de 7 USD. Sin embargo, tampoco dispone de interfaz para cámara ni de lector de tarjetas microSD. Por lo tanto, ambos módulos requerirían componentes externos adicionales.
- *Raspberry Pi Zero 2 W*: incluye Wi-Fi y un conector para cámara, con un costo de 30 USD. Sin embargo, no es un microcontrolador, sino una computadora que requiere un sistema operativo, y el proyecto se plantea sobre microcontroladores de bajos recursos. Además, la cámara se adquiere por separado a un precio de entre 25 y 35 USD.

3.4.2. Módulo de captura

En la presente subsección, la placa y la cámara del módulo de captura se analizan por separado, cada una frente a las alternativas consideradas. Luego se describe la conexión entre ambas.

Placa

El módulo de captura se implementa con la placa *ESP32-CAM* (Fig. 3.21). Esta combina un microcontrolador *ESP32* con una interfaz para cámara y se vende junto con el módulo *OV2640* [38]. También incorpora una memoria PSRAM de 4 MB, que permite almacenar los fotogramas antes de transmitirlos. Sin embargo, no dispone de un puerto USB, por lo que la carga del *firmware* requiere una placa programadora externa.



Figura 3.21: Placa *ESP32-CAM* con su cámara.

Dentro de la familia *ESP32* existen otras placas que también incorporan un conector para cámara, como las basadas en el *ESP32-S3*. Estos modelos ofrecen mayor capacidad de procesamiento y memoria, pero tienen un costo mayor. Una placa genérica con el módulo *ESP32-S3-WROOM N16R8* y cámara cuesta aproximadamente 20 USD a octubre de 2026. El *ESP32-CAM*, en cambio, se basa en el *ESP32* clásico y es la más básica y económica de las opciones con cámara de la familia. Como el módulo de captura solo obtiene y transmite imágenes, no requiere la capacidad adicional del *ESP32-S3*, por lo que esta alternativa resulta suficiente.

Frente a placas de otras marcas, la principal ventaja del *ESP32-CAM* es el bajo costo de inclusión de una cámara. En aquellos modelos, incorporarla requiere módulos adicionales o adquirirla por separado, lo que eleva el precio final. La siguiente tabla compara el costo del *ESP32-CAM* con el de estas alternativas, a octubre de 2026.

Tabla 3.1: Costo de placas compatibles con cámaras, a octubre de 2026

Opción	**Placa**	**Cámara**	**Costo total**
ESP32-CAM	5 a 18 USD	Incluida (*OV2640*)	5 a 18 USD
Raspberry Pi Pico 2 W	7 USD	*Arducam Mini 2MP*, 26 USD	33 USD
Arduino UNO R4 WiFi	30 USD	*Arducam Mini 2MP*, 26 USD	56 USD
Raspberry Pi Zero 2 W	30 USD	*Camera Module 3*, 25 a 35 USD	55 a 65 USD

Cámara

Para seleccionar la cámara se tuvieron en cuenta dos criterios principales. Por un lado, se requería una resolución que permitiera trabajar con imágenes de 320x240 píxeles. Por otro lado, debía tener un campo de visión dentro del rango de entre 60 y 120 grados establecido con anterioridad. Para ello se contaba con múltiples posibilidades, entre ellas, la cámara que se adquiría junto a

la compra del *ESP32-CAM*, que contaba con una resolución de 2 megapíxeles y un ángulo de visión de 68°.

Se consideraron como alternativas una cámara *OV3660* de 3 megapíxeles, que ofrecía resultados similares, y una variante de la *OV2640* con un conector *flex* de mayor longitud, para permitir mayor flexibilidad a la hora del montaje. Ambos modelos se comercializan con distintos ángulos de visión y longitudes de conector. En la Fig. 3.22 se presentan algunas versiones de la *OV2640*. Sin embargo, adquirir cualquiera de estos modelos implicaba un costo adicional que las pequeñas ventajas expuestas no lograban compensar. Por lo tanto, se concluyó que el uso del modelo incluido por defecto resultaba suficiente. Solo fue necesario retirarle el filtro infrarrojo de fábrica para que lograra registrar la luz del LED IR.



Figura 3.22: Cámaras *OV2640* con distintos ángulos de visión y longitud de conector

Conexión de la cámara

La cámara *OV2640* se conecta a la placa *ESP32-CAM* mediante un conector *flex* de 24 pines (Fig. 3.23). Por este conector, el microcontrolador y la cámara se comunican por dos vías, una interfaz paralela de 8 bits para los datos de cada imagen y el bus SCCB para la configuración. Como la cámara incorpora un codificador JPEG por *hardware*, puede entregar las imágenes ya comprimidas.

El *ESP32* no cuenta con una interfaz de cámara dedicada. Por este motivo, la lectura de los datos la realiza su periférico I2S, que admite un modo de entrada paralela. Este periférico guarda cada fotograma en la PSRAM mediante acceso directo a memoria (DMA), es decir, sin intervención del procesador. La siguiente tabla describe las señales que intervienen.

Tabla 3.2: Señales de conexión entre el *ESP32* y la cámara *OV2640*

Señal	**Conexión**	**Función**
D0 a D7	Periférico I2S	Transportan los datos de cada imagen.
PCLK	Periférico I2S	Reloj de píxel. Indica en qué momento leer cada dato.
VSYNC	Periférico I2S	Sincronismo vertical. Marca el comienzo de cada fotograma.
HREF	Periférico I2S	Sincronismo horizontal. Marca el comienzo de cada línea.
SIOD y SIOC	Controlador I2C	Forman el bus SCCB, de dos hilos y similar al I2C. Permiten configurar la cámara, por ejemplo su resolución y su formato de salida.
XCLK	Generador PWM	Lleva la señal de reloj que el *ESP32* genera para que la cámara funcione.
PWDN	Pin directo	Permite apagar la cámara.

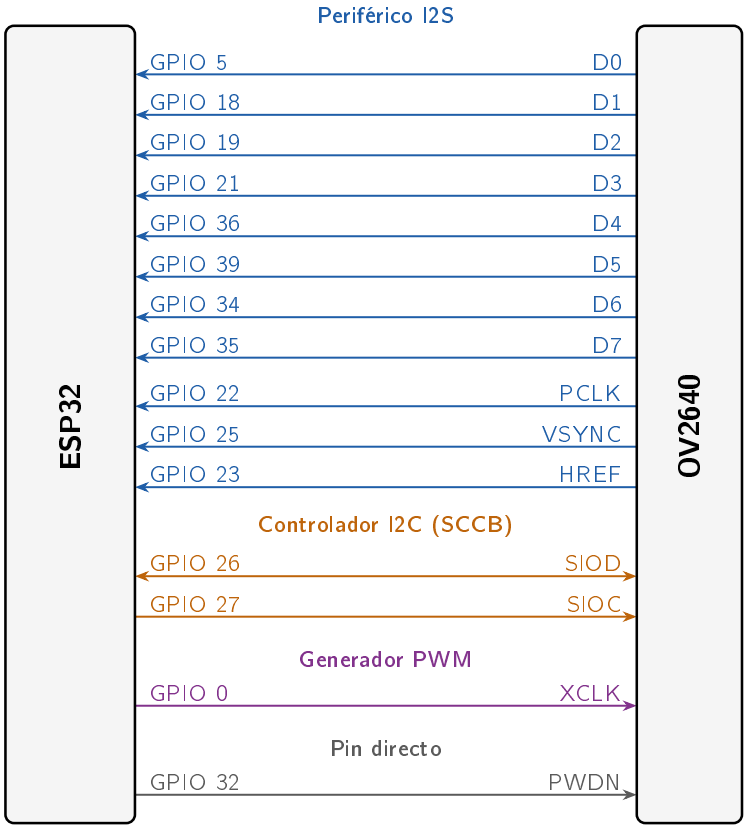


Figura 3.23: Conexión entre el *ESP32* y la cámara *OV2640* en la placa *ESP32-CAM*

3.4.3. Módulo central

El módulo central se implementó con dos placas distintas a lo largo del proyecto. Ambas se presentan a continuación junto con una comparación entre ellas. Las conexiones con la tarjeta microSD y con la pantalla táctil se describen sobre la placa definitiva.

Placa inicial

El módulo central se implementó inicialmente con la placa *ESP32-2432S028R*, conocida como *ESP32-CYD* (*Cheap Yellow Display*). Esta placa se basa en el módulo *ESP32-WROOM-32*, con el *ESP32* clásico, e integra una pantalla táctil de 2,8 pulgadas y una ranura para tarjeta microSD. Su costo es de entre 13 y 26 USD a octubre de 2026. En un principio, se consideró que los recursos de este chip serían suficientes para las tareas del módulo central, ya que el procesamiento de las imágenes ocurre en el dispositivo del alumno.

El principal criterio de elección fue la ranura para tarjeta microSD. El módulo central almacena una base de datos que crece con el uso y aloja la biblioteca *OpenCV.js*, cuyo tamaño excede la capacidad de la memoria flash. La microSD ofrece el espacio necesario para ambos archivos sin agregar componentes externos. Otro aspecto favorable es la versatilidad de almacenamiento de la placa, descrita en el mapa de memoria.

La pantalla táctil, en cambio, no era imprescindible. Algunas de sus funciones, como la consulta del estado del sistema, podrían haberse implementado en la aplicación web. Sin embargo, la placa la incluye y la diferencia de precio con otros modelos *ESP32* no es significativa. Por este

motivo, se decidió aprovecharla como una funcionalidad adicional. Permite configurar datos de inicio y supervisar el estado del sistema directamente desde el módulo central.

La *ESP32-CYD* se utilizó durante la mayor parte del desarrollo, y con ella se logró un sistema funcional, aunque de rendimiento limitado. En las etapas finales se encontró un modelo con mejores prestaciones, que pasó a ser la placa final del módulo central. El análisis de este cambio se presenta en el capítulo 4.

Placa adoptada

La placa que se utilizó en el sistema definitivo es la *ONX3248G035* [39] (Fig. 3.24), perteneciente a la línea *Open Nextion* de la empresa *ITEAD*. Se basa en el microcontrolador *ESP32-S3*, con una memoria flash de 16 MB y una memoria PSRAM de 8 MB. Integra una pantalla táctil capacitiva de 3,5 pulgadas y una ranura para tarjeta microSD. También incorpora un circuito de carga para batería de litio, que se describe en la sección 3.8. Su costo es de 35 USD a octubre de 2026.

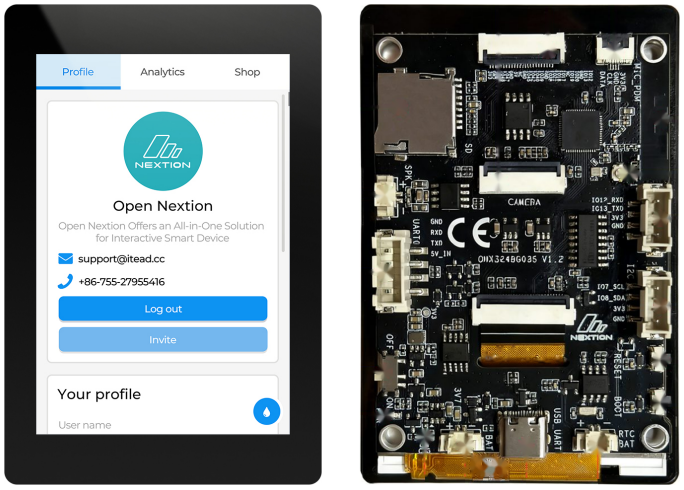


Figura 3.24: Placa *ONX3248G035* (vista frontal y trasera)

Este modelo conserva la pantalla táctil integrada, que fue el motivo para elegir una placa con pantalla en lugar de una *ESP32-S3* sin ella. Además, a diferencia de la *ESP32-CYD*, no es un modelo genérico, por lo que el fabricante ofrece documentación completa y de libre acceso, con esquemáticos, hojas de datos, modelo 3D y códigos de ejemplo.

Una diferencia clave con la *ESP32-CYD* es la memoria PSRAM, que resultó fundamental para mejorar el rendimiento del módulo central. La siguiente tabla compara las características principales de ambas placas.

Tabla 3.3: Comparación de placas para el módulo central

Característica	**ESP32-CYD**	**ONX3248G035**
Microcontrolador	*ESP32* clásico	*ESP32-S3*
Memoria flash	4 MB	16 MB
Memoria PSRAM	No posee	8 MB
Pantalla	2,8", 240x320 (*ILI9341*)	3,5", 320x480 (*ST7796*)
Panel táctil	Resistivo (*XPT2046*)	Capacitivo (*CST826*)
Circuito de carga	No posee	Integrado
Costo (oct. 2026)	13 a 26 USD	35 USD

El costo de la *ONX3248G035* es mayor que el de la *ESP32-CYD*. Sin embargo, la diferencia se justifica por sus mejores prestaciones.

Almacenamiento en microSD

Las tarjetas microSD admiten dos modos de comunicación, el modo SPI y el modo SD nativo. El *ESP32-S3* cuenta con un controlador dedicado al modo nativo, llamado SD/MMC, que es el que se utiliza en este proyecto, al igual que en los ejemplos del fabricante. En la *ONX3248G035*, la ranura tiene conectada al microcontrolador una sola línea de datos, por lo que el controlador trabaja en el modo de un bit. La siguiente tabla describe las líneas de la tarjeta que intervienen.

Tabla 3.4: Conexiones de la tarjeta microSD

Línea	**Conexión**	**Función**
CLK	Controlador SD/MMC	Lleva la señal de reloj.
CMD	Controlador SD/MMC	Transmite los comandos y sus respuestas.
D0	Controlador SD/MMC	Transfiere los datos.
D3	Expansor *PCF8574*	Se mantiene en nivel alto, condición necesaria para que la tarjeta opere en modo nativo.

El *PCF8574* es un expansor de pines que agrega salidas al microcontrolador y se controla por el bus I2C.

Al iniciar, el microcontrolador monta la tarjeta con el sistema de archivos FAT32, que es el formato de fábrica de las tarjetas de hasta 32 GB.

Pantalla y panel táctil

La pantalla de la *ONX3248G035* es un panel TFT de 3,5 pulgadas, con una resolución de 320x480 píxeles y el controlador *ST7796*. Recibe los comandos y la imagen por un bus SPI de solo escritura, ya que el microcontrolador no necesita leer datos de ella. Además, cuenta con dos líneas de control. La retroiluminación se enciende y se apaga desde un pin del microcontrolador. El reinicio, en cambio, lo genera el expansor *PCF8574*, que el microcontrolador maneja por el bus I2C.

El panel táctil es capacitivo y utiliza el controlador *CST826*, que se comunica por el bus I2C. Por este bus, el microcontrolador consulta periódicamente si hubo un toque y, en ese caso, obtiene sus coordenadas. La siguiente tabla resume las conexiones de ambos.

Tabla 3.5: Conexiones de la pantalla y del panel táctil

Línea	**Conexión**	**Función**
SCLK	Bus SPI	Lleva la señal de reloj.
MOSI	Bus SPI	Transfiere los comandos y la imagen hacia la pantalla.
CS	Bus SPI	Habilita la comunicación con la pantalla.
DC	Bus SPI	Indica si lo que se envía es un comando o un dato de imagen.
BL	Pin del microcontrolador	Enciende y apaga la retroiluminación.
RST	Expansor *PCF8574*	Reinicia el controlador de la pantalla al encender el módulo.
SDA	Bus I2C	Transfiere los datos del panel táctil.
SCL	Bus I2C	Lleva la señal de reloj.

La Fig. 3.25 reúne las conexiones de la pantalla, del panel táctil y de la tarjeta microSD con el microcontrolador.

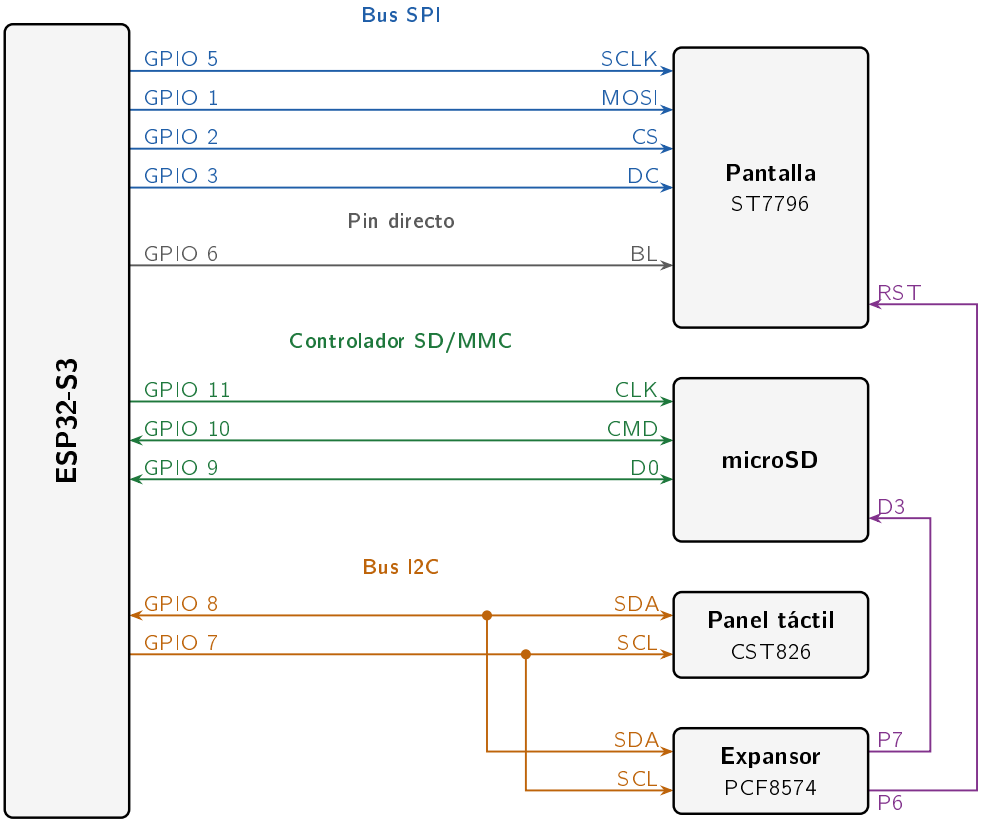


Figura 3.25: Conexiones de la placa *ONX3248G035*

3.5. Red y comunicación

En esta sección se describe la infraestructura de comunicación del sistema. Primero se presenta la red Wi-Fi local que genera el módulo central y la forma en que los usuarios acceden a la aplicación. Luego se detalla la transmisión del video desde el módulo de captura. Por último, se explica cómo se comunican los dispositivos cliente con el módulo central.

3.5.1. Red Wi-Fi local

El módulo central genera una red Wi-Fi propia mediante el modo punto de acceso (*SoftAP*) del *ESP32*. La red se identifica con el nombre LEVI y está protegida con contraseña. Opera en la banda de 2,4 GHz, la única que admite el *ESP32*, sobre el canal 1. Además, acepta hasta cuatro clientes conectados en simultáneo. Si bien el *ESP32* permite más conexiones, cada dispositivo conectado ocupa parte de su memoria, que es acotada. Por este motivo, se fijó este límite. Durante el uso del sistema intervienen hasta tres de ellos, que son el módulo de captura, el dispositivo del alumno y el de un usuario externo. El cuarto lugar funciona como margen, en caso de que se conecte un equipo adicional a la red.

Dentro de esta red, el módulo central ocupa la dirección 192.168.4.1, que funciona además como puerta de enlace (Fig. 3.26). El módulo de captura, en cambio, utiliza una dirección IP estática, la 192.168.4.50. Así, el dispositivo del alumno siempre encuentra el video en la misma ubicación. Los dispositivos cliente, por su parte, reciben una dirección de forma dinámica al conectarse. El módulo central asigna estas direcciones en orden creciente, a partir de la 192.168.4.2. En consecuencia, para la cámara se eligió un valor alejado del inicio. Como la red tolera como máximo cuatro conexiones, resulta prácticamente imposible que la asignación dinámica alcance esa dirección y que dos equipos compartan la misma.



Figura 3.26: Red Wi-Fi local del sistema

Ambas placas utilizan un archivo de configuración con los mismos parámetros de red. Este archivo define el nombre y la contraseña de la red, junto con las direcciones IP de cada módulo. De esta forma, el módulo de captura dispone de las credenciales necesarias para conectarse, y cada placa conoce de antemano la dirección de la otra. Al encenderse, el módulo de captura se une a la red en modo estación. Si la conexión falla, vuelve a intentarlo hasta lograrla.

Una vez establecida la red, el módulo central supervisa su estado. Por un lado, registra la cantidad de dispositivos conectados a partir de la tabla de clientes que mantiene el punto de acceso. Cuando un celular o una computadora abandona la red, envía un aviso de desconexión y el punto de acceso lo quita de la tabla de inmediato. En cambio, si un equipo deja de responder sin

avisar, permanece en la tabla hasta que transcurren cinco minutos sin actividad. Esta situación se da cuando un dispositivo sale del área de cobertura o se apaga de forma repentina.

Por otro lado, el módulo de captura se encuentra en este último caso cada vez que se apaga, ya que no se desconecta de la red antes de hacerlo. Si solo se consultara la tabla, la cámara figuraría como conectada durante varios minutos. Por este motivo, su estado se verifica de forma independiente. Mientras la pantalla está encendida, el módulo central envía un mensaje de eco (*ping*) a la dirección de la cámara cada dos segundos. Si no obtiene respuesta en 500 milisegundos, la considera desconectada. Así, el sistema conoce su estado prácticamente en tiempo real. Para los demás dispositivos esta verificación no resulta necesaria, ya que normalmente se desconectan de forma explícita. La pantalla táctil utiliza ambos datos para informar el estado del sistema.

3.5.2. Acceso a la aplicación

Para utilizar el sistema, el usuario conecta su dispositivo a la red LEVI y luego ingresa a la aplicación web desde su navegador.

El módulo central publica el nombre `levi.local` mediante mDNS, descrito en el capítulo 2. Así, se accede a la aplicación al escribir ese nombre en la barra de direcciones o al escanear un código QR que lo contiene (Fig. 3.26). Si un usuario ingresa con la dirección IP, el servidor lo redirige automáticamente a `levi.local`. De esta forma se mantiene una única dirección de acceso.

En el primer acceso, el navegador obtiene los archivos de la aplicación desde el *Filesystem* de la memoria flash del módulo central. La aplicación funciona como una página única (*Single Page Application*). Por lo tanto, se transfiere una sola vez y, a partir de ese momento, cambia de pantalla sin pedir nuevas páginas al servidor. Si se solicita una ruta que no corresponde a ningún archivo, el servidor responde con el archivo `index.html`.

El dispositivo del alumno descarga además la biblioteca *OpenCV.js*, que el módulo central entrega desde la tarjeta microSD. Debido a su tamaño, es el archivo más pesado que transmite el sistema. Por este motivo, el navegador lo guarda en su almacenamiento, de modo que en los accesos siguientes no vuelve a solicitarlo.

3.5.3. Transmisión de video

El módulo de captura transmite el video mediante la técnica MJPEG, detallada en el capítulo 2. Se optó por esta técnica porque el navegador la reproduce de forma nativa, sin bibliotecas ni código adicional. Para implementarla, el módulo ejecuta un servidor HTTP propio, independiente del que funciona en el módulo central. Este servidor atiende las solicitudes en el puerto 80 de la dirección fija del módulo de captura, en la ruta /stream, y entrega un único flujo de video a la vez, que corresponde al dispositivo del alumno (Fig. 3.27).

La cámara opera con una señal de reloj de 16 MHz y toma cada fotograma con una resolución de 320x240 píxeles. Luego lo comprime en formato JPEG antes de enviarlo. Se eligió este formato porque es el único que coincide entre los que entrega la cámara y los que puede mostrar el navegador. Para esto se utiliza su codificador por *hardware*, por lo que la compresión no consume recursos del microcontrolador. Además, se configuró la calidad en un nivel intermedio, lo que reduce el tamaño de cada imagen sin perder los detalles del ojo necesarios para la detección. La captura ocurre de forma continua y los fotogramas se guardan en la PSRAM, donde siempre se conserva el más reciente. Junto con cada fotograma, el módulo indica su tamaño, lo que permite al navegador reconocer dónde termina uno y comienza el siguiente.

La resolución de 320x240 píxeles (QVGA) es uno de los tamaños predefinidos de la cámara. Una resolución mayor generaría fotogramas más pesados, que ocuparían más la red y reducirían la cantidad de imágenes por segundo. Una menor, en cambio, perdería la nitidez necesaria para reconocer la pupila. Por otro lado, las imágenes se transmiten en color porque el codificador JPEG de la cámara siempre genera este formato. Además, en una imagen JPEG la información de color ocupa poco espacio en comparación con la de brillo, por lo que la escala de grises reduciría apenas el uso de la red. Utilizar un tamaño de imagen no predefinido o un JPEG en escala de grises obligaría a procesar y volver a comprimir cada fotograma por *software*, lo que consumiría los recursos del microcontrolador y haría más lenta la transmisión.

La aplicación web solicita el video únicamente durante la calibración y el examen. Al recibir la solicitud, el servidor enciende el LED infrarrojo y descarta los tres primeros fotogramas, que pueden estar desactualizados o tener la exposición sin ajustar. Mientras el dispositivo del alumno mantiene la conexión, el servidor envía el fotograma más reciente de manera continua, con un máximo de 20 por segundo. Como la aplicación procesa 15 por segundo, siempre dispone de uno nuevo. Fuera de esas etapas, la aplicación cierra la conexión, la cámara deja de transmitir y el LED se apaga. Así, no se ocupa la red de forma innecesaria. Si la conexión se interrumpe por otro motivo o fallan diez capturas seguidas, la transmisión también se detiene hasta recibir una nueva solicitud.

Por otra parte, si al encenderse el módulo de captura no logra inicializar la cámara, se reinicia de forma automática a los tres segundos y vuelve a intentarlo. De esta forma, no queda en funcionamiento un servidor que no puede entregar video.

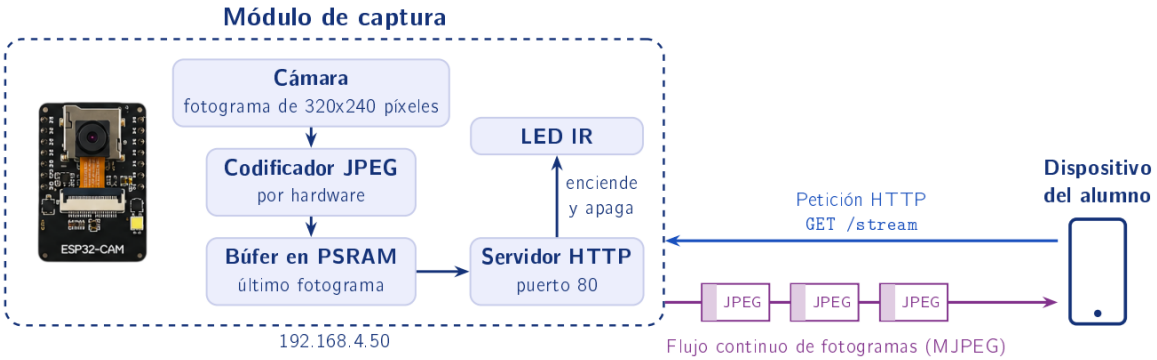


Figura 3.27: Transmisión de video desde el módulo de captura

3.5.4. Comunicación con el módulo central

Los dispositivos cliente se comunican con el módulo central mediante peticiones HTTP, a través de la red local. El servidor web del módulo central las atiende en el puerto 80, el estándar de este protocolo. Las peticiones de datos se dirigen a rutas que comienzan con /api/, mientras que el resto corresponde a los archivos de la aplicación (Fig. 3.28). La información viaja en formato JSON, descrito en el capítulo 2. El servidor procesa las peticiones de forma asíncrona, por lo que puede atender a varios dispositivos sin bloquearse.

En HTTP, cada intercambio comienza con una petición del cliente, y el servidor solo envía información como respuesta. Por este motivo, el módulo central no puede avisar por iniciativa propia cuando algo cambia, y los dispositivos lo consultan de manera periódica para mantenerse actualizados.

El dispositivo del alumno realiza una consulta cada dos segundos mientras se encuentra en la pantalla de examen y esta permanece visible. En ella obtiene la situación actual, como la pregunta del examen en curso o la pregunta enviada por un invitado. Además, todos los dispositivos con una sesión abierta envían un aviso periódico, denominado *heartbeat*, que indica que la sesión continúa activa. El del alumno se envía cada cinco segundos, y el de profesores, tutores e invitados, cada diez. Este último informa también si el usuario utilizó la pantalla desde el aviso anterior. El uso de estos avisos en la gestión de las sesiones se describe en la sección 3.6.2.

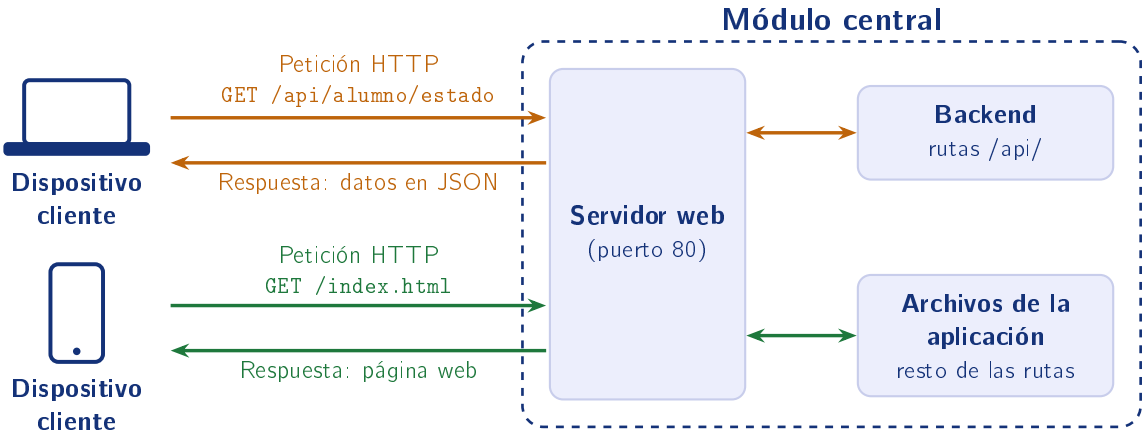


Figura 3.28: Comunicación entre los dispositivos cliente y el módulo central

3.6. Aplicación web

La aplicación web reúne todo el *software* que permite a los usuarios interactuar con el sistema. Se organiza en tres capas, que son la base de datos, el *backend* y el *frontend*. Las dos primeras se ejecutan en el módulo central. El *backend* forma parte del *firmware*, escrito en C++, y la base de datos es un archivo *SQLite* ubicado en la tarjeta microSD, que el propio *firmware* lee y escribe. El *frontend*, en cambio, funciona en el navegador de cada dispositivo cliente.

Tabla 3.6: Capas de una aplicación web

Capa	Ejecución	Ubicación	Formato	Lenguaje	Función
Base de datos	En Módulo Central	Tarjeta SD	Archivo .db	SQL	Almacenamiento de datos persistente
Backend	En Módulo Central	*Firmware* del Módulo Central	Programa en C++	C++	Interconecta el *frontend* con la base de datos y aplica la lógica de negocio
Frontend	En el dispositivo cliente	En el Módulo Central, pero se transmite al navegador	Programa empaqueta-do	HTML, CSS y *JavaS-cript*	Ejecuta la interfaz de usuario

A continuación se describe cada capa, en el orden en que se apoyan unas sobre otras. Primero se presenta la base de datos, luego el *backend* y, por último, el *frontend*.

3.6.1. Base de datos

La base de datos almacena de forma permanente la información del sistema, que incluye los usuarios registrados, los cuestionarios (con sus preguntas y opciones) y los resultados del alumno. Así, estos registros se conservan aunque el módulo central se apague. En esta subsección se explica primero cómo se creó y configuró la base, y luego la estructura de sus tablas.

Creación y configuración

La base de datos del sistema utiliza *SQLite*, el motor embebido presentado en el capítulo 2. Se lo eligió porque no requiere un servidor independiente y funciona con los recursos acotados del módulo central. Toda la información se guarda en un único archivo, `levi.db`, ubicado en la tarjeta microSD.

La estructura de la base se diseñó en una computadora mediante la herramienta *DB Browser for SQLite* (Fig. 3.29). Allí se definieron las tablas, sus campos, las relaciones, las restricciones y los índices. Luego, el archivo resultante se exportó y se copió a la tarjeta microSD. De esta forma, el *firmware* no necesita crear esta estructura al iniciar, sino que abre directamente el archivo existente.

Los índices se definieron sobre las columnas que el sistema consulta con mayor frecuencia, lo que además acelera los borrados en cascada:

- Opciones por pregunta (`idx_opciones_idPregunta`): permite obtener las opciones de una pregunta, como por ejemplo al mostrarla al alumno.
- Preguntas por cuestionario (`idx_preguntas_idCuestionario`): permite obtener las preguntas de un cuestionario, como por ejemplo al buscar la siguiente sin responder durante el examen.
- Opción correcta (`idx_preguntas_idOpcionCorrecta`): permite encontrar las preguntas cuya respuesta correcta es una opción determinada. Se utiliza cuando esa opción se elimina, para dejar vacía la referencia en la pregunta.
- Opción elegida (`idx_preguntas_idOpcionElegida`): permite encontrar las preguntas en las que el alumno eligió una opción determinada. Al igual que el anterior, se utiliza cuando esa opción se elimina.
- Cuestionario en progreso (`idx_cuestionarios_en_progreso`): es un índice parcial que contiene solo el cuestionario en curso, que el sistema busca en cada consulta periódica del dispositivo del alumno.

Al abrir la base, el *firmware* ajusta algunas opciones de funcionamiento de *SQLite* (*PRAGMAs*). Estas opciones responden a las características de la tarjeta microSD y a que el módulo central puede desconectarse en cualquier momento.

- Claves foráneas (`foreign_keys`): se activa su verificación, que *SQLite* deshabilita por defecto. Así se garantiza que las relaciones entre tablas se respeten.
- Registro de transacciones (`journal_mode`): *SQLite* conserva el archivo de registro de cambios entre una transacción y otra, y al finalizar cada una solo borra su encabezado, en lugar de eliminar el archivo y crearlo de nuevo. En el sistema de archivos de la microSD, crear y borrar archivos es lento, por lo que esta opción reduce el tiempo de cada escritura.

- Sincronización (`synchronous`): se emplea el nivel máximo, en el que *SQLite* espera a que la tarjeta confirme cada escritura. Como el dispositivo puede desconectarse en cualquier momento, esto reduce el riesgo de que la base se dañe ante un corte de energía. Dado que cada operación se guarda en una única transacción, el costo en velocidad es bajo.
- Tamaño de la caché (`cache_size`): se fija en 512 KB, ubicados en la PSRAM. Así queda margen para el crecimiento de la base sin ocupar la RAM interna.
- Almacenamiento temporal (`temp_store`): los datos intermedios que *SQLite* genera durante las consultas, por ejemplo al ordenar un listado, se guardan en memoria y no en la tarjeta, lo que acelera esas operaciones.

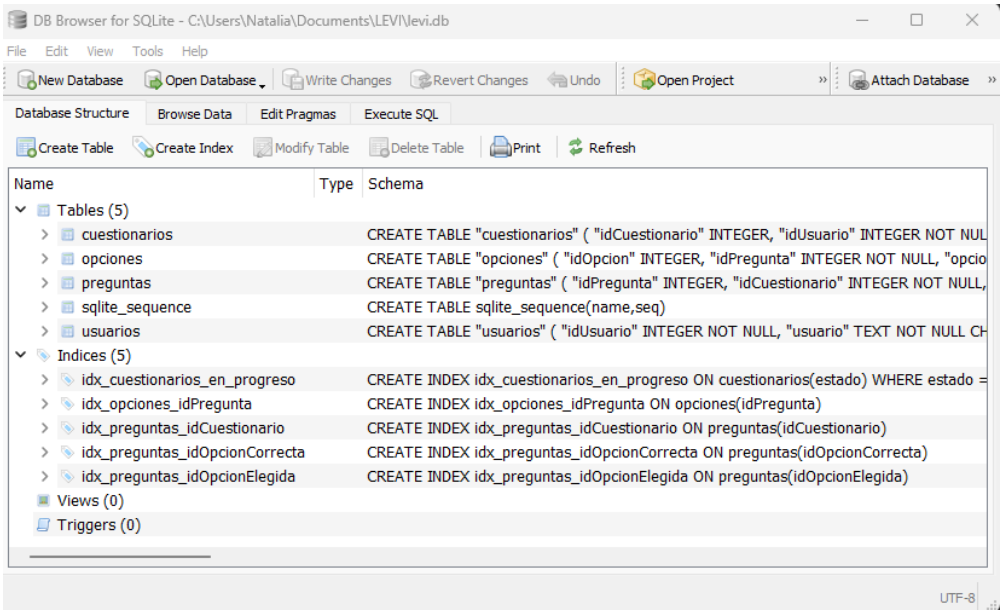


Figura 3.29: Estructura de la base de datos en *DB Browser for SQLite*

Estructura de las tablas

La base de datos se compone de cuatro tablas, que son: usuarios, cuestionarios, preguntas y opciones. En todas ellas, cada registro se identifica con un número entero (su clave primaria), que *SQLite* asigna de forma automática al insertarlo.

Como cada dispositivo pertenece a un único alumno, este no se registra en la tabla de usuarios. Del mismo modo, sus resultados no se guardan en una tabla aparte, sino en los propios cuestionarios, y cada respuesta, en la pregunta a la que corresponde. A continuación se describe el contenido de cada tabla y, por último, las relaciones entre ellas.

Usuarios

La tabla de usuarios guarda los datos de los profesores y los tutores. Los invitados no se almacenan, porque acceden sin registrarse. Cada usuario cuenta con los siguientes campos.

- Usuario: nombre que se utiliza para iniciar sesión. Debe ser único en todo el sistema y tener entre 4 y 30 caracteres, sin espacios.
- Nombre y apellido: datos personales del usuario.
- Rol: indica si se trata de un profesor o de un tutor. No admite otros valores.

- Referencia: su significado depende del rol. Para un profesor indica la materia que dicta, mientras que para un tutor indica su relación con el alumno, como madre o padre.
- Contacto: medio para comunicarse con el usuario, como un teléfono o un correo electrónico.
- *Hash* de la contraseña: resultado de aplicar la función de *hash* a la contraseña combinada con el *salt*.
- *Salt*: cadena aleatoria propia de cada usuario, que se combina con la contraseña antes de calcular el *hash*.

Ninguno de los campos de texto puede quedar vacío.

Cuestionarios

La tabla de cuestionarios guarda tanto la configuración de cada evaluación como su resultado. Cada uno cuenta con los siguientes campos.

- Profesor: referencia al profesor que creó el cuestionario.
- Título: nombre del cuestionario. No puede quedar vacío ni repetirse entre los cuestionarios de un mismo profesor.
- Puntaje para aprobar: puntaje mínimo que debe alcanzar el alumno. No puede ser negativo.
- Estado: etapa en la que se encuentra el cuestionario.
- Puntaje obtenido: resultado del alumno, que se completa al finalizar.
- Fecha de inicio: momento en el que el profesor inició el cuestionario.
- Tiempo: duración de la resolución, expresada en segundos. No contabiliza los períodos en que el cuestionario estuvo pausado.

El estado de un cuestionario solo puede tomar uno de los siguientes valores (Fig. 3.30).

- Pendiente: el cuestionario fue creado, pero todavía no se inició.
- En progreso: el cuestionario fue iniciado y el alumno puede responderlo. Solo uno puede encontrarse en este estado a la vez.
- Pausado: la resolución se interrumpió de forma temporal y puede retomarse.
- Finalizado: la resolución terminó y el resultado quedó registrado.

Al crearse, todo cuestionario queda en estado *pendiente*. Cuando el profesor lo inicia, pasa a estar *en progreso* y se registra la fecha de inicio. Desde ese estado puede pasar a *pausado* en tres casos. El primero es cuando el profesor lo pausa voluntariamente. El segundo, cuando el alumno pierde la conexión con el sistema o sale de su pantalla. El tercero, cuando se inicia o reanuda otro cuestionario, ya que solo uno puede estar en progreso a la vez. Un cuestionario pausado vuelve a estar en progreso cuando el profesor lo reanuda, o de forma automática si el alumno que se había desconectado vuelve a ingresar desde el mismo dispositivo.

Por último, el cuestionario pasa a *finalizado* cuando el alumno responde la última pregunta o cuando el profesor lo da por terminado, ya sea que esté en progreso o pausado. En ese momento se calculan y guardan el puntaje obtenido y el tiempo de resolución. Un cuestionario finalizado no vuelve a cambiar de estado.

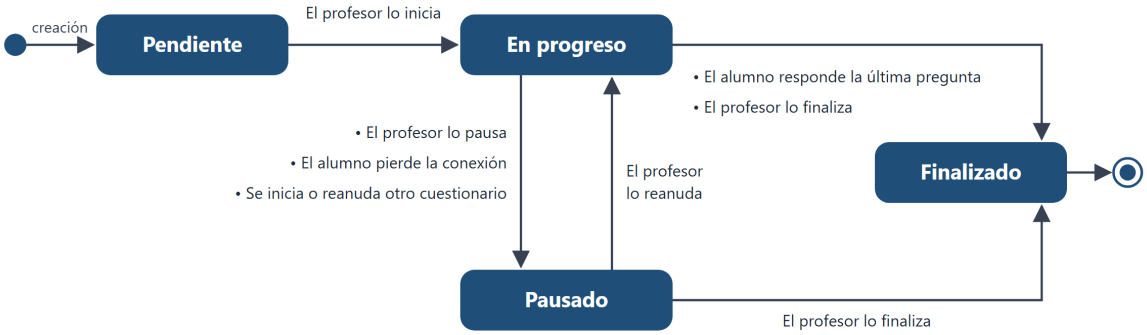


Figura 3.30: Diagrama de transición de estados de un cuestionario

Preguntas

La tabla de preguntas guarda el contenido de cada pregunta y la respuesta del alumno. Cada una cuenta con los siguientes campos.

- Cuestionario: referencia al cuestionario al que pertenece la pregunta.
- Enunciado: texto de la pregunta. No puede quedar vacío.
- Puntaje si es correcta: puntos que se suman al resultado cuando el alumno elige la opción correcta.
- Puntaje si es incorrecta: puntos que se restan al resultado cuando el alumno elige otra opción. Permite penalizar las respuestas erróneas.
- Opción correcta: referencia a la opción que responde correctamente la pregunta.
- Opción elegida: referencia a la opción que seleccionó el alumno. Permanece vacía hasta que responde.

Ambas opciones se registran en la pregunta y no en la tabla de opciones. De esta forma, cada pregunta puede tener una sola opción correcta y una sola opción elegida.

Opciones

La tabla de opciones guarda las respuestas posibles de cada pregunta. Cada pregunta tiene entre dos y cuatro opciones, cuyos campos son los siguientes.

- Pregunta: referencia a la pregunta a la que pertenece la opción.
- Texto: contenido de la opción. No puede quedar vacío.

Relaciones

Las tablas se vinculan mediante claves foráneas (Fig. 3.31). Un profesor puede crear varios cuestionarios, un cuestionario contiene varias preguntas y cada pregunta reúne varias opciones. Además, cada pregunta se relaciona con dos de sus opciones, la correcta y la elegida por el alumno.

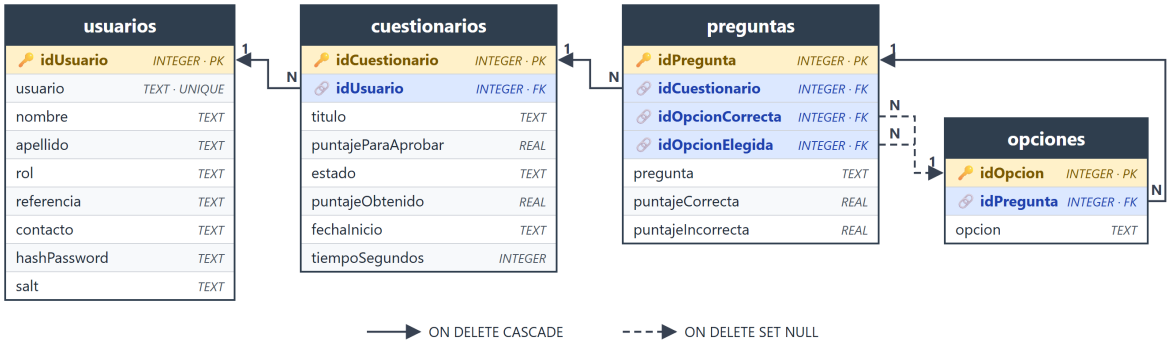


Figura 3.31: Esquema relacional de la base de datos

Estas relaciones definen el orden en que se crean los registros. Un cuestionario requiere que su profesor ya exista, y una pregunta requiere que exista su cuestionario. En el caso de las preguntas, primero se crea la pregunta, luego sus opciones y, por último, se indica cuál de ellas es la correcta. La opción elegida, en cambio, se registra recién cuando el alumno responde, por lo que para entonces la pregunta y sus opciones ya existen.

Las relaciones también determinan qué ocurre al eliminar un registro. Si se elimina un profesor, se borran en cascada sus cuestionarios, junto con sus preguntas y opciones. De esta forma no quedan registros huérfanos en la base. En cambio, si se elimina una opción, la pregunta no se borra, sino que la referencia a esa opción queda vacía.

3.6.2. Backend

Las herramientas habituales para desarrollar un *backend*, como *Node.js* o *Django*, requieren un sistema operativo y una cantidad de memoria muy superior a la del módulo central. Por este motivo, el *backend* se escribió en C++, como parte del mismo *firmware* que controla la red, la pantalla y la tarjeta microSD.

Su código se organiza según el patrón Controlador-Servicio-Repositorio, presentado en el capítulo 2. Cada petición que llega al módulo central atraviesa estas capas en orden. Primero la recibe un controlador, luego un servicio aplica las reglas del sistema y, por último, un repositorio accede a la base de datos. Las capas intercambian información mediante modelos, que son estructuras de datos comunes a todas ellas. La gestión de las sesiones, en cambio, funciona como un componente aparte que utilizan tanto los controladores como los servicios.

Modelos

Los modelos son las estructuras con las que el *backend* agrupa la información para transmitirla de una capa a otra. No realizan operaciones, sino que solo contienen datos. Según su función se distinguen tres tipos.

- Entidades: reflejan los registros de la base de datos según se almacenan. Existe una por cada tabla, con su misma estructura.
- Objetos de transferencia de datos (*Data Transfer Object*, DTO): transportan la información que se intercambia con el *frontend*, organizada según lo que requiere cada caso y no según cómo está guardada. De esta forma, lo que se expone hacia afuera queda separado del modelo interno. Así, un DTO puede agregar valores calculados, como el resultado de

un cuestionario, que reúne el puntaje obtenido, el máximo posible y el necesario para aprobar. También puede omitir campos reservados, como la opción correcta en la pregunta que recibe el alumno.

- Resultados de operación: indican si una acción se completó correctamente o, en caso contrario, cuál fue el error. Los repositorios y los servicios los devuelven a la capa que les hizo el pedido.

Los modelos que agrupan varios elementos tienen una capacidad máxima fija. Este límite responde a un criterio de diseño, ya que en un microcontrolador conviene que la memoria que ocupa cada estructura sea conocida y acotada. De esta forma, un cuestionario admite hasta 20 preguntas, y cada pregunta, hasta cuatro opciones. Los listados, por su parte, se entregan por páginas de diez elementos, por lo que la cantidad total de cuestionarios o usuarios no está limitada.

Controladores

Los controladores constituyen el punto de entrada del *backend*. Cada uno agrupa las rutas de un recurso del sistema. Existen cuatro, que corresponden a la autenticación, los usuarios, los cuestionarios y el alumno. Todas las rutas comienzan con el prefijo /api/, lo que permite al servidor distinguirlas de los archivos de la aplicación web.

Ante cada petición, el controlador verifica primero la sesión y el rol del usuario. Luego interpreta los datos recibidos en formato JSON, invoca al servicio correspondiente y, por último, construye la respuesta. En resumen, los controladores no aplican reglas del sistema, sino que solo traducen las peticiones HTTP en llamadas a los servicios.

Los métodos HTTP, presentados en el capítulo 2, se asignan según la operación:

- GET: consultas, como los listados, el estado del examen o la revisión de un cuestionario.
- POST: creación de usuarios y cuestionarios. También se emplea para acciones que no generan un registro, como iniciar sesión, enviar una respuesta o los avisos periódicos.
- PUT: modificaciones completas, como la edición de un cuestionario o de un perfil.
- PATCH: cambios de estado de un cuestionario, es decir, iniciarlo, pausarlo, reanudarlo o finalizarlo.
- DELETE: eliminación de usuarios y cuestionarios.

El contenido de todas las respuestas tiene el mismo formato. Incluye un campo que indica si la operación tuvo éxito y, ante un error, un mensaje que lo describe. Además, cada respuesta lleva un código de estado HTTP, que es 200 cuando la operación se completa, o 201 cuando además crea un registro. Los códigos que el sistema utiliza para informar fallas son los siguientes:

- 400: los datos enviados son inválidos o están incompletos.
- 401: la sesión no existe o expiró, o se rechazó el inicio de sesión.
- 403: el rol del usuario no tiene permiso para la acción.
- 404: el recurso solicitado no existe.
- 409: la acción entra en conflicto con la situación actual del sistema, como cuando un invitado intenta ingresar durante un examen o cuando la petición proviene de una pestaña que no es la activa.

Un mismo código puede corresponder a situaciones distintas. Por ejemplo, el 409 se emplea tanto cuando un invitado intenta ingresar con el panel ocupado como cuando la sesión está abierta en otra pestaña. Del mismo modo, el 401 se emplea tanto cuando la sesión expiró como cuando falla el inicio de sesión de un profesor o un tutor, ya sea porque las credenciales son incorrectas o porque el panel está ocupado. Para que la aplicación pueda distinguirlas, las respuestas de estos casos incluyen además un identificador fijo de la situación.

Gracias a este formato común, el *frontend* interpreta todas las respuestas de la misma manera.

Cuando el cuerpo de una petición supera el tamaño que el servidor entrega de una sola vez, como puede ocurrir al crear o editar un cuestionario, llega dividido en fragmentos. En ese caso, el controlador los reúne antes de interpretar el JSON.

Servicios

Los servicios concentran las reglas del sistema. Existen cuatro: servicios de autenticación, de usuarios, de cuestionarios y de respuestas del alumno. Cada uno valida los datos que recibe, decide si la acción solicitada está permitida y coordina a los repositorios necesarios para llevarla a cabo. Toda escritura en la base de datos se realiza dentro de una transacción, cuyo funcionamiento se explicó en el capítulo 2. Cuando la acción consiste en una única sentencia, como eliminar un usuario junto con sus cuestionarios en cascada, *SQLite* ya la ejecuta como una transacción. Cuando requiere varias escrituras, como crear o editar un cuestionario, cambiar su estado o registrar la última respuesta del alumno junto con el resultado del examen, el servicio las agrupa explícitamente en una sola. Si alguna escritura falla, se deshacen todas, por lo que la base nunca queda con una operación a medias. Los repositorios, en cambio, son los únicos que leen y escriben los datos.

Autenticación

El servicio de autenticación verifica las credenciales de profesores y tutores mediante el procedimiento de *hash* con *salt* descrito en el capítulo 2, con el algoritmo SHA-256. De esta forma, aun si alguien accediera al archivo de la base de datos, no podría conocer las contraseñas, que los usuarios suelen repetir en otros servicios. Para el cálculo se utilizó la biblioteca *mbedTLS*, que ya forma parte del entorno de desarrollo de la familia *ESP32*, junto con el acelerador por *hardware* para SHA del *ESP32-S3*. Así, no se requieren bibliotecas adicionales y el cálculo se realiza con rapidez.

Si el usuario no existe o la contraseña no coincide, responde con el mismo mensaje en ambos casos. De esta forma no revela qué nombres de usuario están registrados. Cuando la verificación es correcta, solicita al gestor de sesiones que abra una nueva sesión.

Usuarios

Para registrar un usuario o restablecer una contraseña olvidada, el servicio exige la clave maestra correspondiente al rol, definida en la configuración inicial del sistema. Como el sistema funciona sin conexión a internet, no puede verificar las cuentas por correo electrónico. La clave maestra reemplaza esa verificación y evita que cualquier persona conectada a la red cree una cuenta de profesor o de tutor. Además, el servicio genera el *salt* de cada usuario con el generador de números aleatorios por *hardware* del microcontrolador, y lo renueva cada vez que se cambia la contraseña.

Cuestionarios

El servicio de cuestionarios aplica las transiciones de estado descritas en la sección 3.6.1. Antes de cada cambio, comprueba que el cuestionario pertenezca al profesor que lo solicita y que su

estado actual permita la acción. Además, como solo un cuestionario puede estar en progreso, al iniciar o reanudar uno pausa automáticamente al que estuviera en curso. La edición se admite solo en cuestionarios pendientes y la eliminación en cualquier estado, salvo en progreso o pausado.

Al crear o editar un cuestionario, el servicio verifica que tenga título y al menos una pregunta, y que cada pregunta tenga entre dos y cuatro opciones, con una sola correcta.

Este servicio también lleva el cronómetro del examen. El tiempo se mide con el reloj interno del microcontrolador, a partir de las consultas periódicas que realiza el dispositivo del alumno. Entre una consulta y la siguiente se suma el tiempo real transcurrido. Si entre dos consultas pasan más de 6 segundos, ese intervalo no se suma. Así, una demora breve de la red se mide con precisión, pero ni un corte de conexión ni el tiempo que el alumno pasa fuera de la pantalla de examen, durante el cual el dispositivo no consulta, se cuentan como tiempo de examen. Al pausar, el tiempo acumulado se guarda en la base de datos, y al reanudar se recupera. Mientras el examen está en progreso, además, el tiempo se guarda una vez por minuto. Así, si el módulo central se reinicia en medio de un examen, se pierde como mucho el último minuto: en la siguiente consulta del alumno, el cronómetro se retoma desde el valor guardado.

Por último, al finalizar un cuestionario el servicio calcula el puntaje: suma los puntos de cada respuesta correcta, resta los de cada incorrecta y no considera las preguntas sin responder. El resultado se guarda junto con el tiempo de resolución.

Respuestas

El servicio de respuestas determina qué se muestra en el dispositivo del alumno. En cada consulta, si hay un examen en progreso, entrega la primera pregunta sin responder. Si no lo hay y un invitado cargó una pregunta, entrega esa pregunta. En cualquier otro caso indica que no hay nada pendiente.

Al recibir una respuesta, comprueba que la pregunta pertenezca al examen en curso y que la opción corresponda a esa pregunta. Luego la guarda y, si era la última, finaliza el cuestionario y devuelve el resultado. En cambio, la respuesta a una pregunta de invitado no se guarda y la pregunta se descarta.

Repositorios

La capa de repositorios es la única que accede a la base de datos. Existe uno por cada tabla, encargado de leer, insertar, modificar y eliminar sus registros. De esta forma, las instrucciones SQL quedan reunidas en un solo lugar y el resto del *backend* trabaja con modelos sin conocer cómo se almacenan los datos.

Todos los repositorios comparten una única conexión con la base de datos. El módulo central la abre al encender, configura las opciones de *SQLite* presentadas en la sección 3.6.1 y la mantiene abierta mientras funciona. Así se evita abrir y cerrar el archivo en cada petición, lo que demandaría tiempo y memoria adicionales.

Las consultas se ejecutan como sentencias preparadas. Primero se define la instrucción con marcadores en lugar de los valores y luego se asocian a ellos los datos recibidos. Como estos nunca forman parte del texto de la instrucción, se impide la inyección de SQL, es decir, que un dato ingresado por el usuario se interprete como una orden para la base.

Los listados, por su parte, se consultan por páginas. Cada consulta devuelve únicamente los elementos de la página solicitada, por lo que la cantidad de datos que se procesan a la vez se mantiene acotada, sin importar cuántos registros tenga la base.

Gestión de sesiones

La gestión de sesiones controla quién está conectado al sistema y con qué rol. Funciona como un componente independiente de las capas anteriores, ya que lo utilizan tanto los controladores, para verificar cada petición, como los servicios para abrir y cerrar sesiones o consultar su estado.

Tokens de sesión

Al iniciar sesión, el módulo central genera el *token*, presentado en el capítulo 2, como una cadena de 32 caracteres hexadecimales obtenida con el generador de números aleatorios por *hardware* del *ESP32-S3*. El servidor guarda ese *token* en la RAM, junto con el rol del usuario y los datos que controlan su vencimiento. Por lo tanto, las sesiones se pierden al reiniciar el módulo central y los usuarios deben volver a ingresar.

En aplicaciones web con muchos usuarios suele emplearse el *JSON Web Token* (JWT), un *token* firmado que el servidor valida sin almacenar las sesiones. En este caso no resulta conveniente, ya que el sistema necesita registrar las sesiones activas para aplicar las prioridades y la expiración que se describen a continuación.

Lugares de sesión

El sistema admite dos sesiones simultáneas. Una corresponde al dispositivo del alumno, y la otra al panel, que puede ocupar un profesor, un tutor o un invitado. El alumno no ingresa con usuario y contraseña, por lo que el sistema lo identifica por su dispositivo. Para ello, la aplicación genera un identificador y lo conserva en el navegador. Si el lugar del alumno está ocupado por otro dispositivo, se rechaza el nuevo ingreso. En cambio, cuando vuelve el mismo, recupera su sesión en lugar de quedar bloqueado por ella.

En el panel, los roles tienen distinta prioridad. El tutor tiene la mayor, seguido por el profesor y, por último, el invitado. Si alguien intenta ingresar mientras el panel está ocupado, el sistema compara ambos roles. Cuando el nuevo usuario tiene mayor prioridad, la sesión anterior se cierra y el lugar pasa a él. En cambio, si su prioridad es igual o menor, el ingreso se rechaza, salvo que se trate del mismo usuario. En ese caso, la nueva sesión reemplaza a la anterior.

Expiración

El módulo central revisa de forma continua dos tiempos en cada sesión del panel. El de vida se renueva con cualquier petición válida, incluidos los avisos periódicos descritos en la sección 3.5.4, y vence cuando el usuario cierra el navegador o pierde la conexión. El de inactividad, en cambio, se renueva solo cuando el usuario utiliza la pantalla, y vence cuando la sesión queda abierta sin uso. La sesión del alumno controla únicamente la vida, ya que durante el examen el alumno no utiliza el *mouse* ni el teclado. Cuando alguno de estos tiempos supera su límite, el sistema cierra la sesión y libera su lugar. Así, ningún lugar queda ocupado de forma indefinida.

Tabla 3.7: Tiempos de las sesiones según el rol

Rol	**Aviso periódico**	**Límite de vida**	**Límite de inactividad**
Profesor y tutor	10 s	2 min	10 min
Invitado	10 s	2 min	5 min
Alumno	5 s	15 s	–

Si la sesión que expira es la del alumno y hay un examen en progreso, el sistema lo pausa automáticamente. Lo mismo ocurre cuando el alumno sale de su pantalla o cierra la pestaña, ya que en ese caso la aplicación cierra su sesión. Si luego vuelve a ingresar el mismo dispositivo,

el examen se reanuda sin intervención del profesor, salvo que este haya cambiado su estado mientras tanto.

Cada vez que una sesión se cierra, el servidor registra el motivo, ya sea la salida voluntaria, la inactividad, la pérdida de conexión o el ingreso de otro usuario. Si después llega una petición con ese *token*, la respuesta lo indica y la aplicación muestra el aviso correspondiente.

Invitado

El invitado ingresa sin registrarse y ocupa el panel con la menor prioridad. Mientras hay un examen en progreso no puede ingresar ni enviar preguntas. Su pregunta se guarda solo en la RAM, junto con su sesión. Ambas se borran cuando la sesión se cierra, algo que también ocurre si el invitado sale de su pantalla. Así, el modo invitado no interfiere con las evaluaciones ni deja registros en la base de datos.

Pestaña activa

Cada sesión se utiliza desde una sola pestaña a la vez. Para ello, cada pestaña genera su propio identificador y lo envía junto con el *token*. El servidor solo acepta las peticiones de la última pestaña que ingresó, y las demás muestran un aviso con la opción de retomar el control. Así se evita, por ejemplo, que el alumno responda el mismo examen desde dos pestañas.

Verificación de peticiones

Antes de procesar una petición protegida, los controladores recurren a un componente intermedio (*middleware*). Este extrae el *token* de la petición y lo compara con el de la sesión correspondiente. Si no coincide o la sesión expiró, rechaza la petición. Cuando la sesión fue cerrada, la respuesta informa además el motivo del cierre. Tampoco acepta la petición si el *token* proviene de una pestaña que no es la activa o si el rol no tiene permiso para la acción. Cuando la verificación es correcta, también renueva el tiempo de vida de la sesión.

3.6.3. Frontend

El *frontend* es la parte visual del código. Su función es actuar como intermediario entre el usuario y el *backend*, que gestiona los datos y los permisos. Se ejecuta en el navegador de cada dispositivo cliente y se organiza según la arquitectura basada en componentes presentada en el capítulo 2.

Entorno y herramientas

Para su desarrollo se optó por *Preact*, una biblioteca de *JavaScript* cuyas características fueron descritas en el capítulo 2.

El código fuente se escribió en una computadora, con el *software Visual Studio Code*. Sin embargo, en ese formato no puede ser interpretado directamente por los navegadores ni almacenarse de forma eficiente en un microcontrolador. Por este motivo, antes de cargarlo en el módulo central, se utilizó la herramienta de construcción *Vite*, que compiló, minimizó y empaquetó todo el código en archivos estáticos. El resultado fue un documento HTML, una hoja de estilos CSS y un archivo *JavaScript*.

Estos archivos se cargan por USB en el *Filesystem* de la memoria flash del módulo central. Esta partición es independiente de la que contiene el *firmware*, por lo que la interfaz puede actualizarse sin volver a cargar el programa del microcontrolador y viceversa.

Una vez finalizado el desarrollo del *frontend*, se verificó la elección de la biblioteca. Para ello, la misma aplicación se compiló con *React* y se comparó el tamaño del archivo *JavaScript* resultante

con el obtenido mediante *Preact* (Tabla 3.8). Con *Preact*, el archivo ocupó un 61,7 % menos, mientras que la hoja de estilos no varió.

Tabla 3.8: Tamaño del código compilado con *React* y con *Preact*

Biblioteca	**Tamaño del archivo JavaScript**
React	321 KB
Preact	123 KB

Estructura interna

El *frontend* se estructuró en pantallas, donde cada una cumple un conjunto acotado de funciones (Fig. 3.32). Cada pantalla es un componente con su propio archivo de código y su hoja de estilos. Las pantallas destinadas a profesores y tutores, en cambio, se agruparon en un conjunto llamado *panel*. Comparten una misma hoja de estilos, de modo que mantienen una apariencia uniforme y una misma ruta base en el navegador. Además, las hojas de estilos ajustan la disposición de los elementos al tamaño de la pantalla, de modo que la interfaz puede utilizarse tanto en una computadora como en un celular.

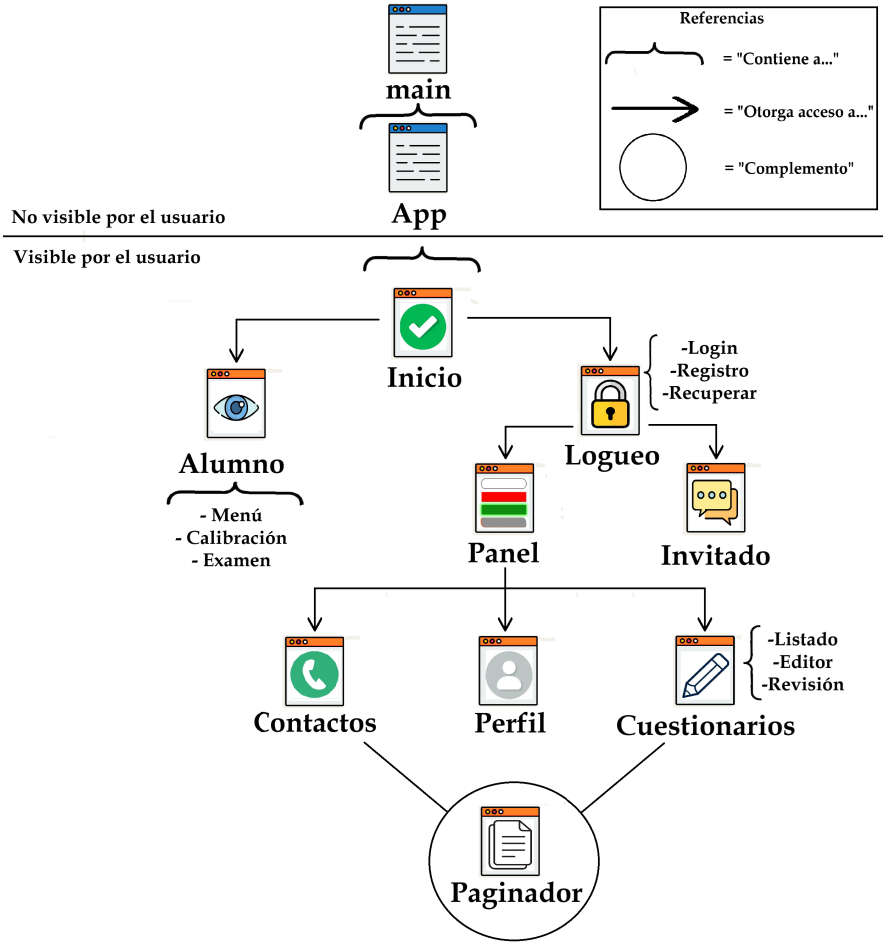


Figura 3.32: Estructura interna del *Frontend*

Los componentes se organizan de forma jerárquica: en la raíz se encuentra el componente principal que contiene a todas las pantallas. A su vez, algunas de ellas incluyen sus propios

componentes. Por ejemplo, la pantalla de cuestionarios contiene el listado, el editor y la vista de revisión. Cuando el profesor elimina un cuestionario desde el listado, este avisa a la pantalla que lo contiene y es ella la que realiza la petición al servidor y actualiza la lista.

El componente principal (`App.jsx`) actúa además como gestor de rutas. Según la dirección del navegador decide qué pantalla mostrar y, si no corresponde a ninguna, redirige al inicio. Mientras el usuario se encuentra en cualquiera de las pantallas del panel o del invitado, envía el aviso periódico, junto con la indicación de si el usuario utilizó la pantalla. Si el servidor informa que la sesión se cerró, muestra el motivo y lo devuelve a la pantalla de inicio de sesión.

El profesor y el tutor comparten las mismas pantallas del panel. Cada una consulta el rol del usuario (que el navegador guarda al iniciar sesión), y muestra u oculta sus elementos según corresponda. Por ejemplo, el campo que el profesor ve como materia, el tutor lo ve como vínculo. Así se evitó duplicar pantallas casi idénticas, lo que reduce el código y mantiene ambas versiones coherentes. Este dato solo determina qué se muestra. Los permisos los verifica el *backend* en cada petición, por lo que ocultar un botón no constituye una medida de seguridad.

Las comunicaciones con el *backend* se aislaron en un módulo independiente (`api.js`), que centraliza todas las peticiones dirigidas al módulo central. De esta forma, las pantallas solicitan datos, como el estado de un examen o la lista de profesores, mediante funciones simples. El archivo `api.js`, por su parte, arma cada petición, le adjunta el *token* de sesión y el identificador de la pestaña, e interpreta las respuestas en formato JSON. Si el servidor responde con un error, lo convierte en un mensaje que la pantalla muestra al usuario.

Por último, se cuenta con un módulo independiente de paginación, utilizado para aumentar la eficiencia del sistema al momento de transmitir y desplegar información en listas. Para ello, se limita la cantidad de elementos individuales que se pueden mostrar al mismo tiempo en pantalla. Estos límites y la segmentación de las páginas son definidos y controlados por el *backend*.

Recepción de datos y ejecución del detector

En la sección 3.5 se describió cómo el módulo central y el módulo de captura envían la información. A continuación se explica qué hace el navegador con ella. Primero se detalla cómo recibe la aplicación y la biblioteca *OpenCV.js*, y luego cómo recibe el video y ejecuta el detector a partir de esos elementos (Fig. 3.33).

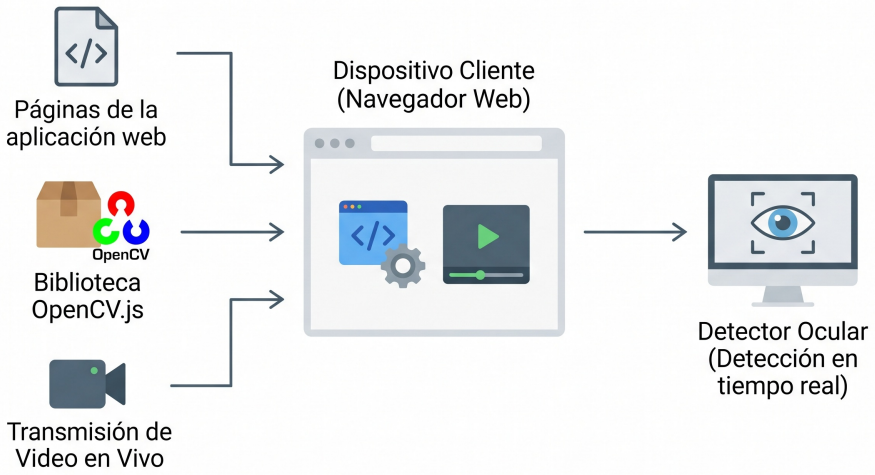


Figura 3.33: Sistema de ejecución de la aplicación web

Carga de la aplicación

Cuando un usuario ingresa a `levi.local`, el módulo central envía al navegador los tres archivos del *frontend*. El documento HTML llega prácticamente vacío y solo indica qué otros dos archivos descargar (el *CSS* y el *JavaScript*). Una vez recibidos, el navegador ejecuta el código *JavaScript*, que arma la interfaz y muestra la pantalla solicitada. A partir de ese momento, todo el funcionamiento de la interfaz ocurre en el dispositivo del usuario.

Obtención de OpenCV.js

La biblioteca *OpenCV.js* no se envía junto con la aplicación, sino solo cuando se abre la pantalla del alumno, que es la única que la necesita. Como es el archivo más pesado del sistema, el navegador guarda una copia después de la primera descarga, de modo que en los accesos siguientes no vuelve a pedirla al módulo central. Cuando debe descargarla, verifica que el archivo haya llegado completo y, si la descarga falla, la vuelve a intentar de forma automática.

La biblioteca incluye una parte de su código que el navegador debe compilar antes de ejecutarla, lo que demora algunos segundos. Durante ese tiempo, la pantalla del alumno muestra un aviso y mantiene deshabilitados los botones.

Recepción del video

Durante la calibración y el examen, el navegador del alumno se conecta directamente al módulo de captura y recibe el video. Cada fotograma JPEG que llega reemplaza al anterior, y el propio navegador se encarga de decodificarlo sin código adicional. El video completo no se muestra al alumno. En su lugar, la pantalla presenta el recorte del ojo que se analiza (128x128). Sobre esa imagen se dibuja un recuadro que marca la región que se guardará en la plantilla al momento de calibrar (96x96). Esto ayuda al alumno a mantener centrado su ojo durante el proceso.

La conexión se abre al ingresar a la calibración o al examen y se cierra al salir de ellos. Cada vez que se abre se fuerza una conexión nueva, para que el navegador no muestre una imagen antigua guardada de un acceso anterior. Si ocurre un error o el video deja de actualizarse durante más de 4 segundos, la conexión se reabre automáticamente.

Ejecución del seguimiento

Una vez compilada, la biblioteca pone sus funciones a disposición de la aplicación, cuyo código *JavaScript* las invoca de forma nativa. Las operaciones de alta carga computacional, como la conversión a escala de grises o la comparación de plantillas, ocurren dentro del entorno optimizado de la biblioteca. El código *JavaScript*, por su parte, se limita a coordinar el flujo de ejecución y a reflejar los resultados en la interfaz visual.

Para procesar las imágenes, *OpenCV.js* las representa como matrices y las aloja en una partición de memoria exclusiva. Este espacio reservado resulta demasiado reducido para retener múltiples fotogramas en simultáneo. Por eso, la aplicación genera una única vez las matrices principales, que son el fotograma completo, el recorte ocular y la versión monocromática. Luego, las reescribe en cada ciclo continuo. Sin embargo, los cálculos internos de la biblioteca exigen la creación de matrices temporales. Al finalizar cada cálculo se convierten en memoria residual, inalcanzable para la limpieza automática del navegador. Por lo tanto, el código las libera de manera explícita tras cada uso.

Respecto a las plantillas obtenidas en la calibración, el almacenamiento del navegador no admite el formato matricial de *OpenCV.js*. Para guardarlas, el sistema las convierte a un formato de imagen estándar. Luego, al cargar la pantalla de examen del alumno, el proceso se invierte. El programa recupera las imágenes guardadas y las reconstruye como matrices para que el algoritmo de detección funcione de forma correcta.

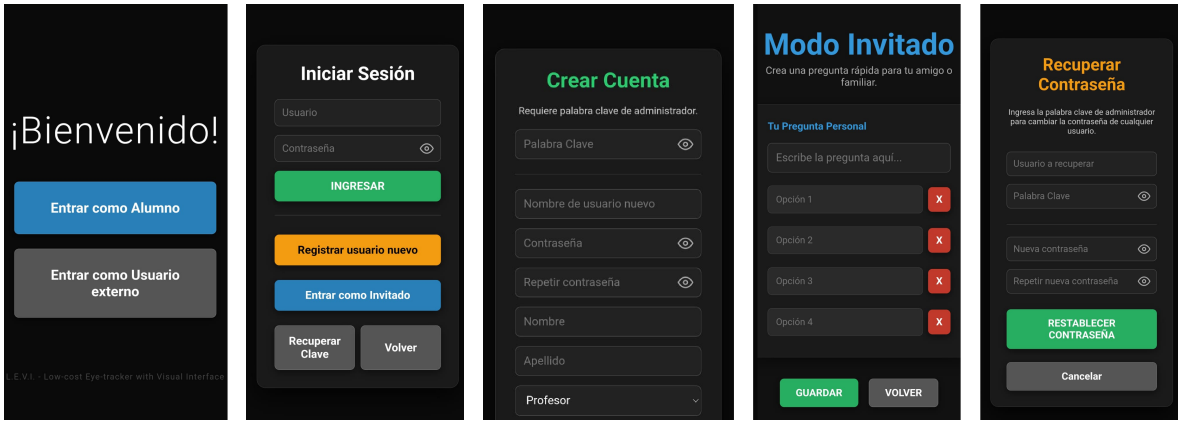
Pantallas

Las pantallas se agrupan en tres categorías, según quién las utiliza: las de acceso, la del alumno y las del panel.

Pantallas de acceso

Son las pantallas a las que cualquier usuario puede ingresar sin importar su rol.

- Inicio (o *Home*) (figura 3.34a): es la primera pantalla que se muestra al acceder a `levi.local`. Presenta un mensaje de bienvenida y dos botones: uno para ingresar como alumno y otro para hacerlo como usuario externo. El primero dirige a la pantalla del alumno y el segundo, a la de inicio de sesión.
- Inicio de sesión (o *Logueo*) (figura 3.34b): los profesores y los tutores ingresan su usuario y su contraseña. Desde allí también pueden registrarse mediante un formulario con sus datos (figura 3.34c), o recuperar una contraseña olvidada (figura 3.34e). En ambos casos se solicita la clave maestra del rol.
- Invitado (figura 3.34d): se accede desde la pantalla de inicio de sesión. Presenta un campo para la pregunta y entre dos y cuatro campos para las opciones, que pueden agregarse o quitarse. Al enviarla, la pregunta aparece en la pantalla del alumno.



(a) Inicio (b) Inicio de sesión (c) Registro (d) Invitado (e) Recuperación de contraseña

Figura 3.34: Pantallas y sub-pantallas de acceso

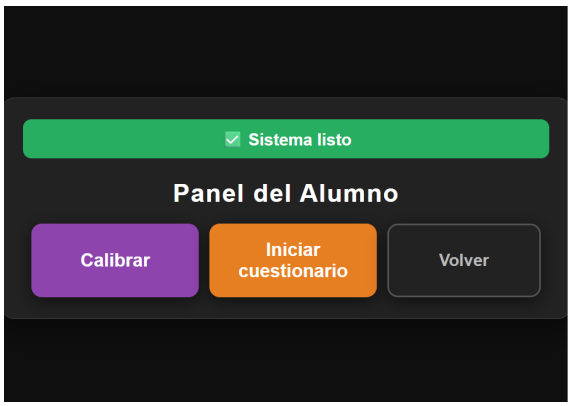
Pantalla del alumno

El alumno ingresa sin autenticarse. Su pantalla reúne cuatro vistas que se alternan según la etapa en la que se encuentra.

- Menú (figura 3.35a): presenta un indicador del estado de carga del detector y dos botones, uno para la calibración y otro para el examen.
- Calibración (figura 3.35b): muestra seis recuadros, uno por cada plantilla, y la imagen del ojo que se analiza. Presenta un botón de inicio y uno de retorno. Durante la secuencia, un mensaje indica hacia dónde debe mirar el alumno y una cuenta regresiva anuncia cada captura. El proceso puede abortarse voluntariamente con un botón. Al finalizar, las plantillas aparecen en sus recuadros y el alumno puede guardarlas o repetir el proceso. Si ya existe una calibración guardada, el botón de inicio ofrece recalibrar. Esa calibración

permanece en el navegador aunque se cierre la pestaña, como se explicó en la sección 3.2.3.

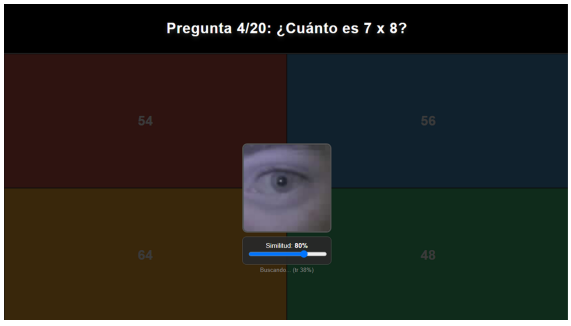
- Examen (figura 3.35c): mientras no haya un cuestionario en curso ni una pregunta de invitado, la pantalla queda en espera y muestra en una esquina la imagen del ojo, junto con un control deslizante que ajusta el umbral de similitud. Cuando llega una pregunta presenta el enunciado y cuatro cuadrantes, uno por opción, con una zona neutra en el centro. En esa zona se ubican la imagen del ojo y, debajo, el control deslizante, de modo que no se superponen con el enunciado ni con las opciones. Si se trata de una pregunta de examen, se incluye en el encabezado el número de pregunta. Cada cuadrante posee un color distintivo. La pantalla se divide siempre en cuatro, aunque la pregunta tenga menos opciones. En este último caso, los cuadrantes sobrantes quedan vacíos y se ignoran, de modo que las posiciones coinciden con las de la calibración.
- Resultado (figura 3.35d): en el caso de una pregunta de invitado, presenta la pregunta y la opción elegida. En el caso de un examen informa el puntaje obtenido, el máximo posible, el necesario para aprobar, el tiempo empleado y si el alumno aprobó. Después de unos segundos la pantalla vuelve sola a la espera, sin necesidad de pulsar ningún botón.



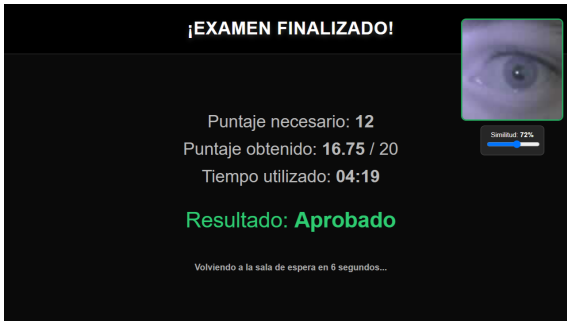
(a) Menú



(b) Calibración



(c) Examen



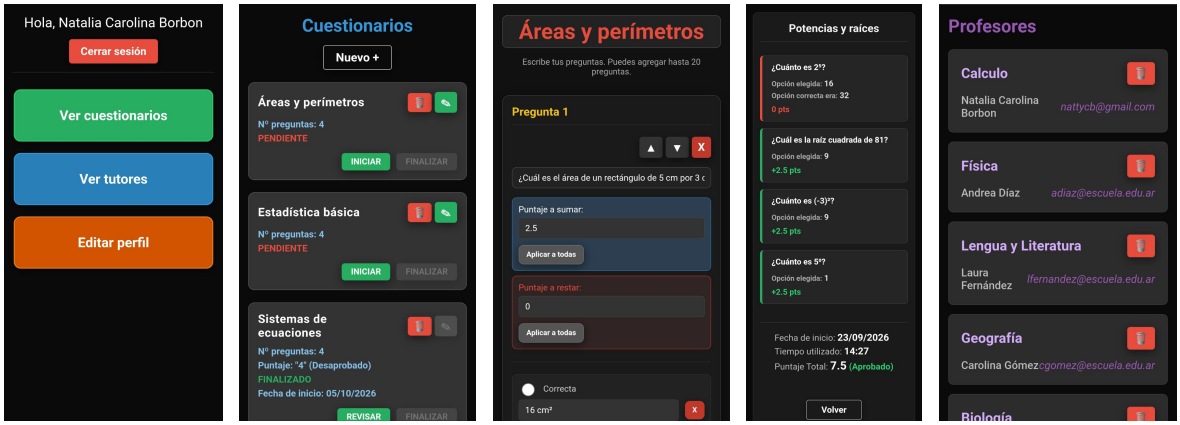
(d) Resultado

Figura 3.35: Vistas de la pantalla del alumno

Pantallas del panel

Son las pantallas que utilizan los profesores y los tutores. En todas ellas, las acciones que no pueden deshacerse, como eliminar un registro o finalizar un cuestionario, solicitan una confirmación antes de ejecutarse.

- Principal (figura 3.36a): muestra un saludo con el nombre del usuario, un botón para cerrar la sesión y tres botones de navegación. El primero dirige a los cuestionarios y el último al perfil. El botón central lleva a los datos de contacto de profesor o tutor según corresponda.
- Perfil: permite modificar los datos personales del usuario y, en una sección aparte, cambiar la contraseña, para lo cual se solicitan la actual y la nueva. No se requiere clave maestra, ya que la sesión activa confirma la identidad del usuario. También permite eliminar la propia cuenta y advierte al usuario si intenta salir con cambios sin guardar.
- Contactos: muestra los datos de contacto de los usuarios del otro rol. Además, se otorga a los tutores la capacidad para eliminar profesores (figura 3.36e). Esta es una de las dos pantallas que utilizan el componente de paginación, que limita la cantidad máxima de usuarios que se muestran por página, según lo definido por el *backend*.
- Cuestionarios: se subdivide en tres vistas.
  - *Listado*: para el profesor, presenta el listado de sus cuestionarios (figura 3.36b) con su título, su cantidad de preguntas, el puntaje obtenido, la condición de aprobación y la fecha de inicio. Cada cuestionario cuenta con botones para editarlo, eliminarlo, cambiar su estado y finalizarlo, que se habilitan o deshabilitan según su estado actual. El botón de estado solo cambia su función en cada caso, de modo que inicia, pausa o reanuda el cuestionario y, una vez finalizado, permite revisarlo. Al iniciar un cuestionario, el sistema captura la fecha y hora del dispositivo del profesor para almacenarlas en la base de datos. Esto se debe a que el sistema carece de conexión a internet, lo que le impide obtener la hora de forma autónoma. El tutor, en cambio, ve los cuestionarios de todos los profesores con la materia de cada uno, y solo puede acceder a su revisión. Ambas listas aplican el componente de paginación, que muestra una cantidad máxima de cuestionarios por página.
  - *Editor* (figura 3.36c): exclusivo para el profesor. Se accede desde el listado, tanto para crear un cuestionario como para modificar uno pendiente. Cada pregunta se carga en un bloque propio. Para agilizar la carga, un mismo esquema de puntajes puede aplicarse a todas las preguntas, y estas pueden reordenarse o eliminarse. Se incluye además un botón para cargar un cuestionario mediante un archivo JSON externo, en caso de que se quiera editar por fuera del sistema.
  - *Revisión* (figura 3.36d): muestra el resultado de un cuestionario finalizado. Se indica la fecha de inicio, el tiempo de resolución, el puntaje obtenido y el resultado final (Aprobado o Desaprobado). Detalla para cada pregunta la opción elegida, la correcta en caso de error y los puntos que sumó o restó.



(a) Principal (b) Cuestionarios (c) Editor (d) Revisión (e) Contactos

Figura 3.36: Pantallas del panel

Editor externo

Se desarrolló como una alternativa para crear y editar cuestionarios por fuera del sistema principal. Consiste en una página web independiente cuyo único propósito es generar un archivo estructurado en formato JSON. Este archivo puede ser importado posteriormente por el profesor desde su panel. Su interfaz y funcionalidad son exactamente las mismas que las del editor original, pero, en lugar de guardar el cuestionario en el sistema, lo descarga directamente en el dispositivo del usuario. También permite cargar un archivo JSON previamente generado para retomar su edición.

3.7. Interfaz de usuario táctil

Con el fin de facilitar el uso del dispositivo, se incorporó una interfaz gráfica en la pantalla táctil del módulo central. Desde ella se realiza la configuración inicial del equipo y se supervisa el estado del sistema. La interfaz se desarrolló con la biblioteca *LVGL*, presentada en el capítulo 2. A continuación se describen las bibliotecas y los controladores que utiliza, su funcionamiento dentro del módulo central y las vistas que la componen.

3.7.1. Bibliotecas y controladores

La interfaz se apoya principalmente en dos bibliotecas. *TFT_eSPI* provee los controladores de bajo nivel que comunican el microcontrolador con la pantalla mediante SPI. Por sí sola, sin embargo, solo permite dibujar elementos básicos como píxeles, líneas y texto. *LVGL*, en cambio, aporta los elementos que requiere la interfaz, como las vistas deslizables, los campos de texto, el teclado virtual y los botones, y gestiona los toques del usuario. Sin ella, todos estos elementos habrían tenido que programarse desde cero.

Además, *LVGL* puede dibujar la pantalla por partes. En lugar de reservar memoria para la imagen completa, que ocuparía unos 300 KB, utiliza un búfer de alrededor de 20 KB, equivalente a una franja de 32 líneas de la pantalla. La biblioteca dibuja cada franja en ese búfer y la envía a la pantalla antes de pasar a la siguiente. Así, la interfaz ocupa una fracción mínima de la RAM interna del módulo central.

LVGL, sin embargo, no lee el panel táctil de forma directa. Recibe las coordenadas de cada toque desde el controlador del panel y, a partir de ellas, determina qué elemento de la interfaz se presionó.

3.7.2. Funcionamiento

La interfaz forma parte del mismo *firmware* que el resto del módulo central y se actualiza en cada repetición de su ciclo principal. Los datos que muestra no viajan por la red, sino que se leen directamente de la memoria del microcontrolador. Así, la interfaz consulta el gestor de sesiones para conocer el usuario conectado, y el monitor de red para obtener el estado de la cámara y la cantidad de dispositivos.

La información del examen requiere un tratamiento particular. Consultarla en la base de datos en cada ciclo implicaría acceder a la tarjeta microSD de forma continua. Por este motivo, cada vez que el alumno consulta el estado del examen, el servicio de respuestas guarda en la RAM un resumen con los datos que muestra la pantalla, y la interfaz solo lee esa copia.

Para no ocupar al microcontrolador de forma innecesaria, el nombre del usuario conectado y el estado de la cámara se vuelven a dibujar solo cuando difieren del valor que ya se muestra. Del mismo modo, el resultado de un examen se dibuja una única vez, al finalizar.

Al encender el equipo, la pantalla se inicia antes que la tarjeta microSD, la base de datos, la red y el servidor web. De esta forma, el usuario observa la interfaz desde los primeros instantes, mientras el resto del sistema termina de arrancar.

Con el fin de aumentar la eficiencia energética del dispositivo, se implementó un sistema de suspensión. Tras 60 segundos sin toques, se apaga la retroiluminación, la interfaz deja de actualizarse y se detiene la verificación de la cámara. Durante este estado de reposo solo se sigue leyendo el panel táctil, lo que permite al usuario reactivar la pantalla en cualquier momento. El resto del sistema continúa funcionando con normalidad. Para evitar acciones involuntarias, este primer toque sirve exclusivamente para encender la pantalla, sin activar ningún elemento de la interfaz.

3.7.3. Vistas

La interfaz se organiza en cuatro vistas. Tres de ellas forman parte de un contenedor deslizante, en el que el usuario pasa de una a otra con un gesto horizontal. La cuarta, en cambio, solo aparece durante la configuración inicial del equipo.

Bienvenida

Es la primera vista que se muestra al encender el equipo ya configurado (figura 3.37). Presenta un mensaje de bienvenida y un código QR que contiene la dirección `levi.local`. Al escanearlo con un teléfono, el navegador abre directamente la aplicación, sin necesidad de escribir la dirección. Como el contenido del código no cambia, no tenía sentido generarlo en cada encendido. Por eso, se lo guardó en la memoria flash como una imagen ya generada en blanco y negro. Generarlo al iniciar el equipo, además, habría ocupado RAM de forma innecesaria.



Figura 3.37: Vista de bienvenida de la interfaz táctil

Estado del sistema

Al deslizar la pantalla se accede a un panel de supervisión (figura 3.38). Este muestra el nombre del alumno, el del usuario conectado al panel, el estado de la cámara y la cantidad de dispositivos conectados a la red. Los dos últimos datos se obtienen según lo descrito en la sección 3.5.1.

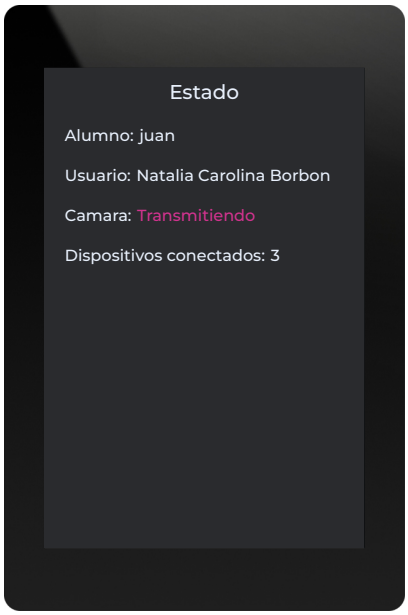


Figura 3.38: Vista de estado de la interfaz táctil

Examen

La tercera vista permite seguir el examen desde el propio módulo central (figura 3.39). Mientras no haya un cuestionario en curso muestra un mensaje de espera. Durante el examen presenta el título del cuestionario y el progreso del alumno, expresado como la pregunta actual sobre el total. Al finalizar muestra el resultado, que incluye el puntaje necesario para aprobar, el obtenido

sobre el máximo posible, el tiempo empleado y si el alumno aprobó. Después de unos segundos, la vista vuelve sola a la espera.

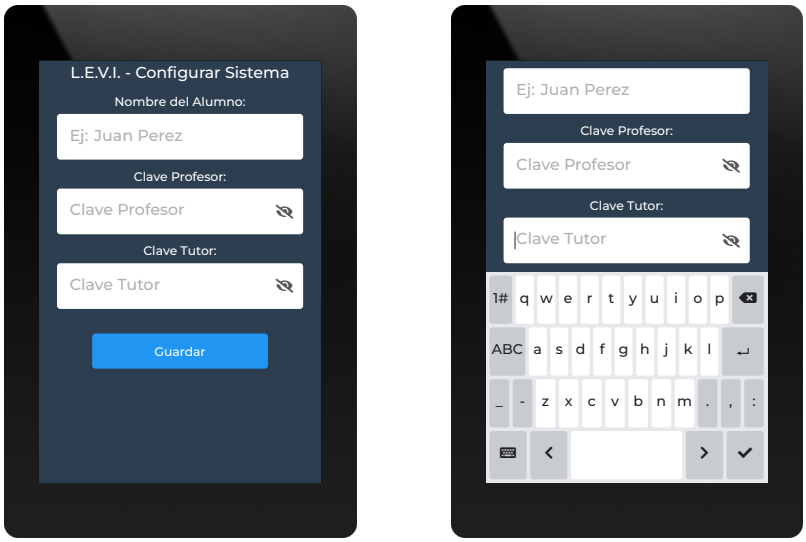


(a) En espera (b) En curso (c) Finalizado

Figura 3.39: Vista de examen de la interfaz táctil

Configuración inicial

Esta vista aparece solo cuando la NVS no contiene los datos de configuración, como ocurre en el primer encendido del equipo (figura 3.40). En ella se ingresan el nombre del alumno y las claves maestras de profesores y tutores. Para escribirlos, la interfaz despliega un teclado virtual. Las claves se muestran ocultas, aunque un botón permite verlas, y no es posible continuar con campos vacíos. Al guardar, los datos se almacenan en la NVS y el equipo se reinicia para cargar las vistas habituales.



(a) Formulario (b) Teclado virtual

Figura 3.40: Vista de configuración inicial de la interfaz táctil

3.8. Alimentación

Esta sección detalla el sistema de energía autónomo del proyecto. Primero se repasan los requisitos y el presupuesto de consumo de cada módulo. Luego se analizan y comparan las alternativas propuestas. Se justifica la elección final y se expone la configuración adoptada. Posteriormente, se describe el circuito del módulo central y, por último, se verifican los cálculos de autonomía.

3.8.1. Requisitos previos

El dispositivo final debe ser inalámbrico y energéticamente autónomo. Por este motivo, se descarta la conexión directa mediante cable USB. Otra opción habitual consiste en acoplar un *power bank* comercial. Sin embargo, esta alternativa se desestimó por criterios de diseño y ergonomía. En un principio, el proyecto apuntaba a una carcasa que integrara la placa y la fuente de energía en un mismo bloque. Dado que los bancos de carga comerciales varían en sus dimensiones físicas, resultaba imposible estandarizar un único modelo de gabinete.

En su lugar, se diseñó un sistema propio basado en celdas de litio recargables cilíndricas (formato 18650). Estas baterías conservan un tamaño universal sin importar su fabricante. Su tensión varía entre 2,7 V y 4,2 V según su nivel de carga.

Esta energía requiere acondicionamiento previo. Las placas de desarrollo del *ESP32* admiten una alimentación estable de 3,3 V o de 5 V. En el segundo caso, un regulador de la propia placa la reduce a los 3,3 V con los que trabaja el microcontrolador. Una pequeña fluctuación provocaría fallos de ejecución o reinicios del sistema.

Cada bloque del prototipo posee su propio circuito de alimentación independiente. Esta separación elimina el cruce de cables externos y otorga mayor libertad de movimiento. Originalmente, el diseño contemplaba etapas idénticas para el módulo de captura y el módulo central (equipado con la *ESP32-CYD*). No obstante, la migración hacia la placa *ONX3248G035* alteró este esquema, ya que el nuevo *hardware* incorpora su propio gestor interno.

El usuario debe poder reponer la energía sin extraer la batería del equipo. Además, la autonomía operativa total debe cubrir al menos una jornada escolar completa (aproximadamente 6 horas continuas).

3.8.2. Presupuesto de consumo por módulo

Antes de analizar posibles soluciones, se deben determinar los requisitos eléctricos totales de cada módulo. Para ello, se tomaron múltiples medidas de consumo con una fuente estable de 5 V. Estos valores se listan en la tabla 3.9.

Tabla 3.9: Presupuesto de consumo de corriente con fuente de 5 voltios

Módulo	**Consumo Promedio (mA)**
Captura (en transmisión Wi-Fi e IR encendido)	200
Central con *CYD* (Pantalla apagada)	167
Central con *CYD* (Pantalla encendida)	254
Central con *ONX* (Pantalla apagada)	130
Central con *ONX* (Pantalla encendida)	212

Apagar la pantalla inactiva ahorra entre un 34 % y un 38 % del consumo total, según el modelo de placa analizado. Para el diseño inicial, se tomó como referencia la corriente consumida por el módulo de captura y el módulo central con el *ESP32-CYD*.

El producto entre la corriente media y la tensión de entrada (5 V) define la potencia consumida. El módulo de captura exige 1 W. El módulo central equipado con la placa *CYD* requiere 1,27 W con la pantalla activa y 835 mW en estado de suspensión.

Para estimar el requerimiento sobre la batería de litio, se dividió esta potencia por la tensión nominal de la celda (3,7 V). Este cálculo establece la siguiente corriente proyectada por módulo:

Tabla 3.10: Presupuesto de consumo estimado

Módulo	**Consumo Promedio (mA)**
Captura (en transmisión Wi-Fi e IR encendido)	270
Central con *CYD* (Pantalla apagada)	226
Central con *CYD* (Pantalla encendida)	343
Central con *ONX* (Pantalla apagada)	176
Central con *ONX* (Pantalla encendida)	286

Análisis adicionales revelaron, por un lado, que la transmisión de video impacta de forma mínima en la placa central (entre un 1 % y un 3 % de variación). No obstante, este proceso introduce una mayor inestabilidad a causa de oscilaciones eléctricas frecuentes. Además, los valores máximos de corriente detectados no superaron el 5 % del promedio, por lo que no se tuvieron en cuenta para posteriores análisis.

Estos valores empíricos sirvieron como base teórica para diseñar el circuito de alimentación y dimensionar la capacidad real de la batería en las verificaciones operativas.

3.8.3. Alternativas propuestas

Para cumplir los requisitos, se evaluaron tres métodos de alimentación. Todos utilizan celdas de litio 18650 de 3000 mAh (tensión nominal de 3,7 V).

La primera alternativa consiste en utilizar un regulador lineal de tensión de baja caída (LDO), como el *AMS1117*, para ajustar el nivel de tensión a 3,3 V estables. Para recargar la batería se utiliza un cargador externo comercial. La segunda alternativa utiliza dos módulos separados, un cargador (*TP4056* [40]) y un elevador de tensión (*MT3608* [41]). El primero recarga la batería de litio y la protege durante su uso. El segundo regula el nivel de tensión para obtener 5 V estables que alimentan directamente el microcontrolador. Por último, el tercer método combina la recarga, la protección y la regulación en un solo integrado (*IP5306*).

Para seleccionar el circuito definitivo se tuvieron en cuenta aspectos como la caída de tensión entre la entrada y la salida (*dropout*), la capacidad para cargar el dispositivo a la vez que se lo utiliza, las limitaciones, la disponibilidad y el costo. La tabla 3.11 resume estos parámetros.

Tabla 3.11: Comparativa de alternativas de alimentación

Alternativa	**Dropout**	**Uso al cargar**	**Limitaciones**	**Disponibilidad**	**Costo total estimado**
LDO + Cargador externo	Alta (∼1,1 V)	No	*Dropout*: apaga el sistema cuando la celda baja a 4,1 V.	Alta	22,30 USD
TP4056 + *MT3608*	Nula (Topología *Boost*)	No	Doble conversión de energía (baja eficiencia).	Muy alta	18,80 USD
IP5306	Nula (Topología *Boost*)	Sí	Inyecta ruido de conmutación. Requiere filtrado extra.	Baja	20,00 USD

Se descartó la primera alternativa por el elevado *dropout* del regulador lineal. El *AMS1117* exige casi 1,1 V de margen. Como el *ESP32* requiere una alimentación mínima recomendada de 3,0 V, el sistema se apaga cuando la celda desciende a unos 4,1 V. Esto impide aprovechar la curva completa de descarga de la batería (de 4,2 V hasta 2,7 V). La tercera opción se desestimó porque su convertidor conmuta a una frecuencia baja, lo que produce un mayor rizado en la salida. Este ruido afecta las comunicaciones inalámbricas.

La solución adoptada fue la segunda alternativa. El *TP4056* gestiona la recarga de forma segura, mientras que el *MT3608* estabiliza la tensión a 5 V constantes. Entregar 5 V implica una pérdida de eficiencia por la doble regulación (el *ESP32* reduce internamente a 3,3 V), pero asegura la inmunidad ante caídas de tensión de la celda y garantiza un rizado despreciable a la salida. Como el módulo de carga no discrimina entre la corriente de la batería y la del sistema, el dispositivo debe permanecer apagado durante la recarga para permitir el corte automático de seguridad.

3.8.4. Detalle de la solución adoptada

Esta subsección detalla los parámetros de los módulos comerciales empleados en el módulo de captura.

El módulo *TP4056* (figura 3.41a) ejecuta un ciclo de carga de corriente constante y tensión constante (CC/CV). La tabla 3.12 resume sus especificaciones.

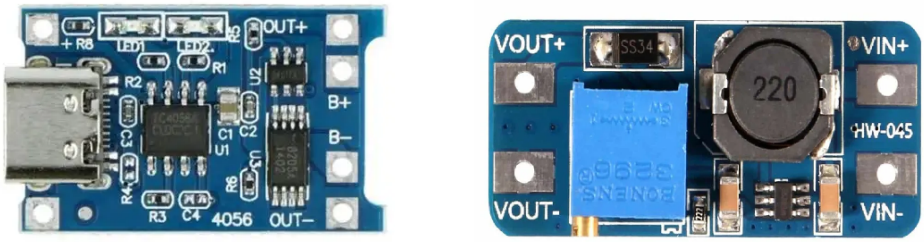
Tabla 3.12: Parámetros de operación del módulo *TP4056*

Parámetro	**Valor**
Tensión de entrada	4,5 V a 5,5 V
Tensión de corte final	4,2 V
Corriente de carga máxima	1 A (Programable por resistencia)

Por su parte, el *MT3608* (figura 3.41b) eleva la señal nominal de la celda a través de un circuito *Boost*. Mediante un potenciómetro en la placa, se fija su salida en 5 V. La tabla 3.13 expone sus límites operativos.

Tabla 3.13: Parámetros de operación del módulo *MT3608*

Parámetro	**Valor**
Tensión de entrada admitida	2 V a 24 V
Tensión de salida	Ajustable hasta 28 V
Corriente máxima	2 A
Frecuencia de conmutación	1,2 MHz



(a) Módulo de carga *TP4056* (b) Módulo elevador *MT3608*

Figura 3.41: Módulos comerciales de alimentación del módulo de captura

3.8.5. Circuito de alimentación del ONX3248G035

El microcontrolador del módulo central migró al modelo *ONX3248G035*. Esta placa incluye su propio sistema de energía, diseñado para soportar celdas de litio directas sin elevar previamente la tensión a 5 V.

El gestor de carga es el chip *IP4054V*. Opera de forma similar al *TP4056*, pero es más robusto y presenta mejores protecciones. Emplea una resistencia de programación de 3,3 kΩ. Según su hoja de datos, esto establece un flujo de carga constante de 0,3 A.

Para estabilizar la señal eléctrica a 3,3 V, la placa emplea un convertidor reductor (*Buck*) síncrono, basado en el integrado *LC2201CCB5TR*. Este componente conmuta a 2 MHz, lo que maximiza la eficiencia térmica del sistema.

Al igual que un regulador lineal, este convertidor posee un límite físico para mantener la regulación. Su transistor interno presenta una resistencia típica $R_{DS(ON)}$ de 250 mΩ. Con el consumo de la placa, esto genera una caída de tensión de aproximadamente 0,1 V, por lo que exige un mínimo de 3,4 V en la entrada para sostener los 3,3 V de salida. Sin embargo, la ventaja fundamental del *Buck* radica en su comportamiento al cruzar este umbral. Al quedarse sin margen de reducción, el chip suspende la conmutación y entra en un modo de operación al 100 % de ciclo

de trabajo (*paso directo*). En este estado, el componente actúa como un puente directo entre la batería y la carga. Como el *ESP32-S3* requiere una alimentación mínima recomendada de 3,0 V, la celda puede descender hasta unos 3,1 V sin interrumpir el funcionamiento del dispositivo. Así se aprovecha casi toda la energía remanente de la batería. Esto evita el apagado anticipado que sí provocaría el alto umbral de caída de un LDO bajo las mismas condiciones.

3.8.6. Verificaciones operativas finales

Para validar el sistema, se calcula la autonomía operativa y los tiempos de reposición de energía. Inicialmente, se comprobó el consumo de corriente real de los módulos alimentados con la batería de litio montada sobre el circuito diseñado para comparar con las estimaciones previas.

Tabla 3.14: Consumo de corriente empírico con celda de litio

Módulo	**Consumo Promedio (mA)**
Captura (en transmisión Wi-Fi e IR encendido)	295
Central con *CYD* (Pantalla apagada)	238
Central con *CYD* (Pantalla encendida)	376
Central con *ONX* (Pantalla apagada)	150
Central con *ONX* (Pantalla encendida)	252

Las discrepancias respecto a los cálculos teóricos (tabla 3.10) tienen una justificación topológica. En el módulo de captura y en la configuración *CYD*, el consumo real superó las estimaciones debido a las pérdidas térmicas del circuito intermediario entre la batería y la placa de desarrollo.

Por el contrario, los valores de la *ONX3248G035* resultaron menores a los proyectados. Esto ocurre porque su convertidor reductor síncrono (*Buck*) opera con mayor eficiencia al recibir los 3,7 V directos de la batería frente a los 5 V utilizados en las mediciones iniciales. Al existir una menor diferencia de potencial entre la entrada y la salida de 3,3 V, el componente optimiza su ciclo de trabajo y reduce la energía disipada en forma de calor.

Al dividir la capacidad de la celda (3000 mAh) por el consumo empírico, el módulo de captura (295 mA) arroja una autonomía de 10,2 horas continuas.

Para el módulo central con la placa *ONX3248G035*, se estima un tiempo de pantalla activa del 10 % del uso total. El cálculo de la corriente promedio (I_{prom}) se define con la siguiente ecuación:

$$I_{prom} = (I_{ON} \times 0,10) + (I_{OFF} \times 0,90) \tag{3.1}$$

Al sustituir las variables por los valores tabulados, el módulo central promedia un consumo de 160,2 mA. Esto implica una autonomía teórica de 18,7 horas. Se comprueba así que la capacidad energética de ambos módulos supera con creces el mínimo necesario para cubrir una jornada escolar de 6 horas.

Finalmente, el tiempo de recarga se estima a partir de la corriente inyectada por cada gestor. El módulo de captura (configurado a 1 A) demanda aproximadamente 3 horas para completar un ciclo. El módulo central (*IP4054V* a 0,3 A) requiere cerca de 10 horas.

La recarga se comprobó de manera empírica sobre el módulo de captura. Al conectar una celda completamente descargada (2,8 V), el sistema completó el ciclo y alcanzó su tensión máxima (4,1 V) exactas 3 horas y 2 minutos después.

3.9. Diseño físico

Esta sección detalla el aspecto físico final del dispositivo. Primero se presenta el circuito de alimentación del módulo de captura y la primera versión pensada para el módulo central, que utilizaba la placa *ESP32-CYD*. Luego se muestran los planos de las placas impresas y, finalmente, el ensamblaje definitivo del sistema.

3.9.1. Circuitos

La sección anterior explicó en detalle los componentes utilizados en la etapa de alimentación. A continuación, en la figura 3.42, se ilustra la totalidad de las conexiones para el módulo de captura.

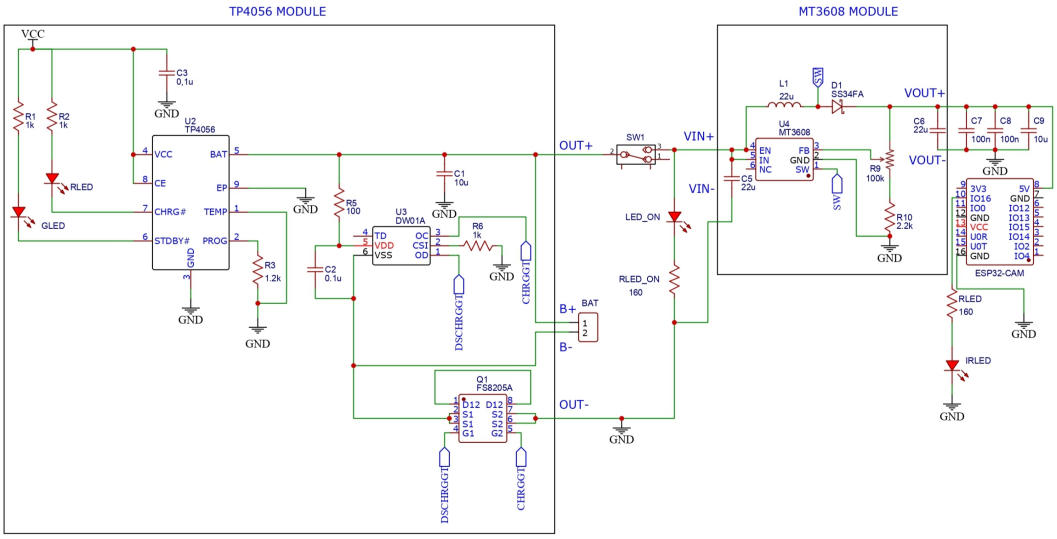


Figura 3.42: Esquema de conexiones del circuito del módulo de captura

El diagrama cuenta con los módulos *TP4056* y *MT3608*, separados por un interruptor para apagar el sistema de forma manual. Además, incorpora el LED infrarrojo y un LED de encendido, ambos con su respectiva resistencia. Se incluye el zócalo para el *ESP32-CAM* y tres capacitores. Estos últimos filtran el ruido eléctrico y suavizan los picos de corriente.

Al utilizar componentes comerciales preensamblados, el esquemático final se reduce al expuesto en la figura 3.43.

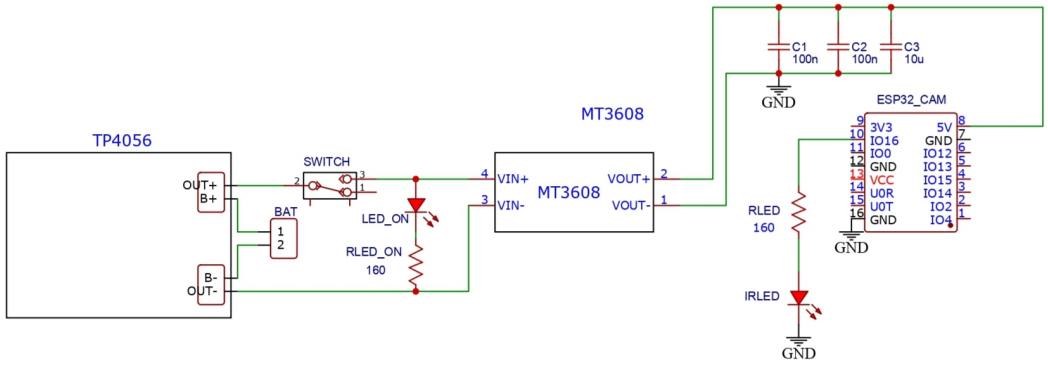


Figura 3.43: Esquema simplificado del circuito del módulo de captura

El *TP4056* recibe energía por su puerto USB-C, por lo que los pines de entrada se omiten en el diagrama.

Por su parte, el diseño del primer módulo central (Fig. 3.44) es similar al del módulo de captura, pero sin el indicador de encendido, sin la etapa infrarroja y sin el acople para la cámara.

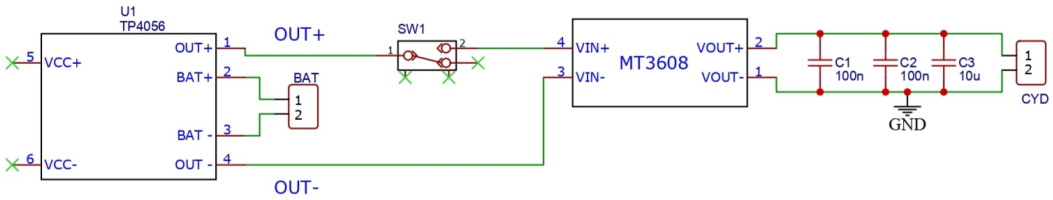


Figura 3.44: Esquema simplificado del circuito del módulo central inicial (en desuso)

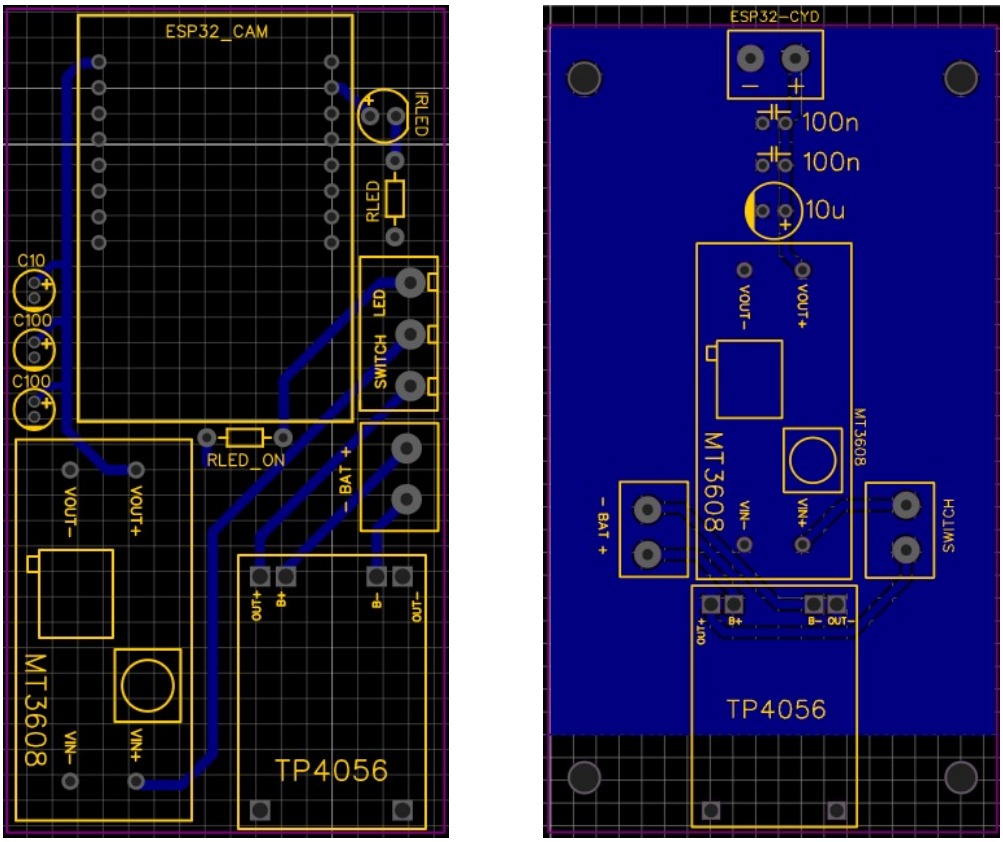
3.9.2. PCB

El diseño de la PCB del módulo de captura tuvo en cuenta su montaje dentro de la carcasa. Algunos componentes, como el LED de encendido, no se sueldan sobre la placa, sino que se fijan a la carcasa y se conectan mediante cables. Por este motivo, la PCB incorpora borneras en lugar de esos componentes, lo que permite elegir con mayor libertad su posición final en el equipo. Los materiales necesarios para armarla son los siguientes:

- 1 módulo de carga comercial *TP4056* (con protecciones *DW01A* y *FS8205A* incluidas).
- 1 módulo elevador comercial *MT3608*.
- 1 interruptor mecánico para encendido general.
- 1 diodo LED emisor infrarrojo (iluminación ocular).
- 1 diodo LED azul (indicador de estado).
- 1 resistencia de 470 Ω (1/4 W) para el LED infrarrojo.
- 1 resistencia de 220 Ω (1/4 W) para el LED azul.
- 2 capacitores cerámicos de 100 nF (filtrado de ruido de alta frecuencia).
- 1 capacitor electrolítico de 10 μF (estabilización de transitorios de corriente).

- 1 bornera de 2 terminales.
- 1 bornera de 3 terminales.
- Tiras de pines (zócalos hembra) para el acople extraíble del *ESP32-CAM*.
- 1 conector JST SM hembra de 2 pines.
- 1 cable JST SM macho de 2 pines (para el LED IR).
- 1 placa de circuito impreso (PCB) a medida.

A continuación se muestran los planos (*layouts*) de las placas impresas trazados mediante el *software* web *EasyEDA* (figura 3.45).



(a) Módulo de captura

(b) Módulo central inicial (en desuso)

Figura 3.45: *Layouts* de las PCB

Al trazar las pistas de cobre se consideró la posición de la antena Wi-Fi de cada módulo. En el módulo de captura, el *ESP32-CAM* se acopla sobre la placa mediante zócalos. Su antena está en la parte inferior del módulo, por lo que queda sobre el centro de la PCB. Un plano de masa en esa zona atenuaría la señal y reduciría el alcance de la conexión. Por este motivo, se optó por no aplicar relleno de masa en esta placa.

En el módulo central inicial, en cambio, la antena queda en un extremo de la placa. Esta disposición permitió incorporar un plano de masa, que mejora la estabilidad eléctrica del circuito. Se dejó una franja libre de cobre junto a la antena para evitar la atenuación de la señal.

Los *layouts* mostrados corresponden al diseño final de las placas. Las placas fabricadas se basaron en una versión previa, que luego recibió pequeñas modificaciones. Estas placas se imprimieron en cobre de capa simple, se perforaron y se soldaron los componentes.

3.9.3. Montaje

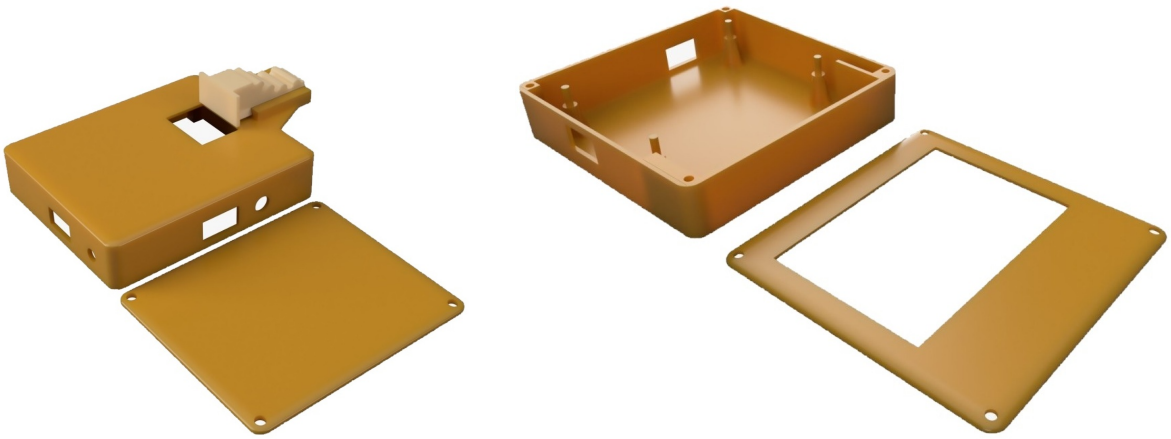
Una vez armados los circuitos, las placas de cada módulo quedan en condiciones de funcionar. Sin embargo, operarlas de forma expuesta resulta incómodo y compromete la integridad física del equipo. Por lo tanto, se diseñó una carcasa impresa en 3D para cada bloque.

Como se observa en la figura 3.46a, el estuche del módulo de captura posee una abertura rectangular por donde asoma la cámara. Este hueco cuenta con un mecanismo deslizante para ajustar el ángulo de visión e integra otra pieza móvil donde irán adheridas la cámara y el emisor infrarrojo. Además, la estructura incorpora tres ranuras: una en la que se inserta el interruptor, otra por la que asoma el LED de encendido y la última, a través de la cual se puede acceder al pin de carga del dispositivo.

En un inicio, se consideró incluir un compartimento para la batería de litio. Esta solución concentraba todo el peso en la visera de la gorra. Como consecuencia, la misma tendía a desplazarse hacia adelante, lo que volvía inviable el uso del dispositivo. Por este motivo, se optó por trasladar la batería a la parte trasera de la gorra para mejorar la distribución del peso.

Por su parte, la cubierta del módulo central inicial presentaba un espacio para insertar la batería de litio y una ventana del tamaño exacto del panel táctil. Contaba además con otros dos orificios: uno para el interruptor y otro para acceder al pin de carga.

Con el cambio de *hardware* a la placa *ONX3248G035*, se rediseñó el gabinete (figura 3.46b), con una disposición similar a la anterior y sus medidas adaptadas a esta nueva placa. Cabe mencionar que la batería se conecta al módulo a través de un conector hembra JST WT de 2 pines, por lo que fue necesario adquirir un conector macho del mismo tipo.



(a) Módulo de captura

(b) Módulo central con la *ONX3248G035*

Figura 3.46: Carcasas impresas en 3D

Tras la impresión de las piezas, los circuitos se fijaron en su interior. A continuación, se empleó adhesivo y tiras de velcro (abrojos) para adherir el módulo de captura a la visera de la gorra. El módulo central, en cambio, se mantiene como un elemento portátil e independiente, que el profesor, el tutor o el compañero pueden ubicar donde les resulte cómodo.

3.10. Costo total de desarrollo

La siguiente tabla detalla los componentes de ambos módulos junto con su costo a octubre de 2026. Los precios corresponden al mercado local, salvo los de las placas *ONX3248G035* y *ESP32-CAM*, que provienen de tiendas internacionales y se expresan en dólares. En el mercado local, el *ESP32-CAM* resulta considerablemente más caro. El total se presenta en pesos junto con su equivalente aproximado en dólares.

Tabla 3.15: Costo de los componentes del sistema (octubre 2026)

Componente	**Cantidad**	**Precio unitario**	**Subtotal**	**% del total**
Placa *ONX3248G035*	1	USD 35	USD 35	32,4 %
ESP32-CAM	1	USD 18	USD 18	16,7 %
Tarjeta microSD de 16 GB	1	$12.000	$12.000	7,2 %
Módulo de carga *TP4056*	1	$2.000	$2.000	1,2 %
Módulo elevador *MT3608*	1	$3.000	$3.000	1,8 %
Batería de litio 18650 de 3000 mAh	2	$15.000	$30.000	18,0 %
Portabaterías	2	$2.500	$5.000	3,0 %
Interruptor	2	$2.000	$4.000	2,4 %
PCB del módulo de captura	1	$8.500	$8.500	5,1 %
Componentes menores (resistencias, capacitores, borneras, LEDs y zócalos)	–	$5.000	$5.000	3,0 %
Impresión 3D	2	$7.500	$15.000	9,0 %
Total aproximado			**$166.120 (USD 108)**	**100 %**

El prototipo emplea una tarjeta microSD de 32 GB que ya estaba disponible. Sin embargo, como los archivos del sistema ocupan poco espacio, puede utilizarse una tarjeta de casi cualquier capacidad, por lo que el cálculo considera el precio de una de 16 GB, de uso habitual. Además, su valor varía mucho según la marca y otras características.

El costo total del sistema ronda los $166.120, unos USD 108. De ese monto, $92.900 corresponden al módulo central y $73.220 al módulo de captura. Estos valores surgen de compras minoristas. En una producción a mayor escala, adquirir los componentes al por mayor reduciría el costo total.

El cálculo no incluye costos de *software*, ya que las herramientas y bibliotecas empleadas en el desarrollo son gratuitas. Tampoco contempla el armado del dispositivo, como la soldadura de los componentes y el montaje de los módulos en sus carcasas. Del mismo modo, excluye la placa programadora que requiere el *ESP32-CAM* para la carga del *firmware*, ya que no forma parte del dispositivo y puede reutilizarse.

El módulo de captura resulta costoso para la función que cumple, ya que solo obtiene y transmite video. No obstante, es difícil encontrar una cámara sin filtro infrarrojo. Además, debe poder ubicarse con facilidad en una posición fija frente al ojo.

El sistema se concibió como un dispositivo completo, con carcasa propia, que una persona común pueda encargar a alguien con nociones básicas de electrónica.

En una producción a mayor escala, la fabricación y el montaje podrían encargarse a una empresa. En ese caso, los diseños actuales podrían modificarse para que la PCB del módulo de captura incluya los circuitos de carga y de elevación de tensión, en lugar de los módulos *TP4056* y *MT3608* soldados por separado.

Por otra parte, quien arme el dispositivo por su cuenta podría optar por una variante más económica, con menor integración. En el módulo de captura, esta consiste en reemplazar la etapa de alimentación por un *power bank* y conectar el LED infrarrojo con su resistencia mediante cables soldados, sin PCB. Como desventaja, la carcasa actual está diseñada para la PCB y los componentes de su lista de materiales. Algunos de ellos tienen su lugar en ella y podrían no incluirse al armarlo de esta forma.

Capítulo 4

Conclusiones

Este capítulo presenta las conclusiones finales del proyecto. Primero, se describe el resultado obtenido y se enumeran los objetivos iniciales cumplidos. Luego se plantean posibles mejoras a implementar en versiones futuras.

4.1. Resultados obtenidos

Durante la mayor parte del desarrollo, el módulo central se implementó con la placa *ESP32-CYD*, presentada en la sección 3.4.3. Con ella se logró un sistema funcional, aunque lento y muy limitado. La falta de memoria PSRAM restringía la cantidad de preguntas que podía tener un cuestionario. También obligó a optimizar el uso de RAM en todo el *firmware*, sobre todo en las consultas a la base de datos, que era donde más podía reducirse. Por el mismo motivo, la caché de *SQLite* debía ser pequeña, lo que hacía más lentas las consultas. Además, los controladores SPI del microcontrolador quedaban ocupados por la pantalla y la tarjeta microSD, por lo que el panel táctil debía leerse por *software*. Por otro lado, la placa no incluía un circuito de carga y requería una placa de alimentación aparte. Su consumo también era mayor, lo que reducía la autonomía del módulo central.

El cambio a la *ONX3248G035* resolvió estos problemas. Su PSRAM amplía la memoria de trabajo, por lo que el límite actual de preguntas se fijó por criterio de diseño y no por falta de memoria. Además, permite utilizar una caché de mayor tamaño, lo que agiliza las consultas. El panel táctil se comunica por un bus propio y el circuito de carga viene integrado. Por último, la placa consume menos y cuenta con documentación oficial del fabricante.

Como resultado de este trabajo se obtuvo un dispositivo de seguimiento ocular funcional, cómodo y de bajo costo. El desarrollo abarcó áreas muy diversas, como la programación de sistemas embebidos, el desarrollo de aplicaciones web, los protocolos de comunicación, el diseño de redes y la electrónica de alimentación. La integración de todas ellas permitió construir el sistema final.

Tal como se planteó en la sección 1.4, el objetivo general consistió en desarrollar un dispositivo de bajo costo que permita evaluar a un alumno con movilidad reducida y comunicarse con él mediante la mirada. Este objetivo se cumplió, ya que el sistema resuelve ambas funciones con dos módulos basados en microcontroladores *ESP32*. De los objetivos específicos, se alcanzaron seis de los siete propuestos.

El dispositivo resultó económico, ergonómico y portable. Su costo total, de aproximadamente 90 USD según la sección 3.10, es muy inferior al de los equipos comerciales. La cámara se monta sobre la visera de una gorra, y cada módulo funciona con su propia batería. De acuerdo con las verificaciones de la sección 3.8.6, la autonomía de ambos supera la duración de una jornada escolar. Además, el módulo central genera la red local descrita en la sección 3.5.1, por lo que el sistema no depende de internet.

La aplicación web, presentada en la sección 3.6, se aloja en el módulo central y se abre desde cualquier navegador conectado a su red. A través de ella, los profesores crean, editan y eliminan cuestionarios de hasta 20 preguntas. También los inician, los pausan y los finalizan durante la evaluación. Los tutores, por su parte, consultan todos los cuestionarios junto con las respuestas y los puntajes del alumno, lo que permite su seguimiento académico. Los compañeros, en cambio, ingresan como invitados y envían preguntas informales que no se guardan. De esta forma, se lograron los objetivos de interfaz accesible, gestión de cuestionarios, seguimiento académico y canal de comunicación.

También se cumplió el objetivo de contar con un sistema de registro e inicio de sesión y con una base de datos local. Como se detalla en la sección 3.6.2, el registro exige la clave maestra del rol, las contraseñas se guardan protegidas con *hash* y *salt*, y cada sesión se identifica con un *token*. Según el mapa de memoria de la sección 3.3.3, las claves maestras se almacenan en la NVS y las sesiones activas en la RAM, fuera de la base de datos. Esta última, explicada en la sección 3.6.1, es un archivo *SQLite* ubicado en la tarjeta microSD, que guarda los usuarios, los cuestionarios y las respuestas del alumno. Sin embargo, ni la comunicación ni la base de datos están cifradas.

En cambio, no se alcanzó el objetivo de procesar la imagen del ojo en un microcontrolador. Como se evidenció en la sección 3.2.1, ninguno de los métodos probados en el *ESP32-CAM* y en el *ESP32-S3* logró la precisión y la velocidad necesarias. Por este motivo, se adoptó la comparación de plantillas en el navegador del alumno, mientras que los microcontroladores capturan y transmiten el video y alojan la aplicación.

En conclusión, el dispositivo final adoptó una solución técnica distinta de la propuesta original. Aun así, el balance es positivo, ya que cumple con la premisa planteada desde el inicio. Se trata de una herramienta portable, intuitiva y accesible para la evaluación y la comunicación del alumno.

4.2. Trabajo futuro

A lo largo del desarrollo se identificaron distintos aspectos del sistema que podrían mejorarse en versiones futuras.

- *WebSockets*: la comunicación con el módulo central podría realizarse mediante *WebSockets*, un protocolo que mantiene abierta una conexión entre el navegador y el servidor. Por ella, ambos pueden enviar datos en cualquier momento. Así, el módulo central avisaría a los dispositivos cuando algo cambia, en lugar de que estos lo consulten de forma periódica, como se describe en la sección 3.5.4. Esto reduciría la cantidad de peticiones en la red y permitiría, por ejemplo, que el panel de profesores y tutores se actualice en tiempo real.
- Descarga de revisiones: actualmente, la revisión de un cuestionario finalizado solo puede consultarse desde la aplicación. Se podría permitir su descarga en un archivo, de modo que profesores y tutores conserven los resultados fuera del módulo central.
- Indicador de batería: actualmente, ninguno de los dos módulos muestra el nivel de carga de su batería. La *ONX3248G035* incorpora un divisor resistivo que permite medir la tensión de la batería, mientras que en el *ESP32-CAM* la implementación resulta más compleja, ya que no cuenta con este circuito. En ambos casos, la relación entre la tensión de una celda de litio y su carga no es lineal, por lo que convendría construir una tabla a partir de mediciones propias.

- Registro e inicio de sesión con huella dactilar: un lector de huellas dactilares en el módulo central permitiría que profesores y tutores se registren e inicien sesión con su huella. De esta forma, ya no serían necesarias las claves maestras, y los usuarios no tendrían que recordar un nombre de usuario y una contraseña.
- Ergonomía del módulo de captura: la cámara tiene poca movilidad, lo que dificulta ajustar su posición frente al ojo del alumno. Por este motivo, se busca mejorar la ergonomía del módulo.
- Versión con la cámara del dispositivo del alumno: el código podría ofrecer, como alternativa al módulo de captura, el uso de la cámara del propio dispositivo del alumno. Ambas versiones serían válidas. Esta opción reduciría el costo del sistema, pero tiene desventajas. La iluminación infrarroja deja de estar disponible, y la persona debería resolver cómo mantener la cámara en una posición fija. Además, el código tendría que recortar la imagen alrededor del ojo. Para esto, el usuario podría indicar de forma guiada dónde está el ojo. Otra alternativa sería sumar un segundo algoritmo que ubique y recorte el ojo de manera automática. En ambos casos, la comparación de plantillas se aplicaría sobre ese recorte.

Bibliografía

[1] Tobii. *Página oficial de Tobii*. url: https://www.tobii.com/es/company.

[2] EyeComTec. *Página oficial de EyeComTec*. url: https://eyecomtec.com.

[3] EyeGaze. *Página oficial de EyeGaze*. url: https://eyegaze.com/about-us/.

[4] EyeHarp. *Página oficial de EyeHarp*. url: https://eyeharp.org/es/#.

[5] OpenCV Team. *OpenCV Documentation*. url: https://docs.opencv.org/5.0/.

[6] Wikipedia. *Template matching*. Última edición. 30 de jul. de 2026. url: https://en.wikipedia.org/wiki/Template_matching.

[7] OpenCV Team. *Template matching con OpenCV*. 31 de dic. de 2025. url: https://docs.opencv.org/4.13.0/d4/dc6/tutorial_py_template_matching.html.

[8] O. García-Olalla Olivera. *Redes Neuronales Artificiales: Qué son y cómo se entrenan*. Xeridia. 16 de sep. de 2019. url: https://www.xeridia.com/blog/redes-neuronales-artificiales-que-son-y-como-se-entrenan-parte-i.

[9] O. García-Olalla Olivera. *¿Qué es Machine Learning?* Xeridia. 19 de mar. de 2019. url: https://www.xeridia.com/blog/que-es-machine-learning.

[10] Espressif Systems. *ESP32 Series Datasheet*. url: https://documentation.espressif.com/esp32_datasheet_en.html.

[11] LVGL Developers. *LVGL Documentation*. Última edición. 1 de sep. de 2026. url: https://lvgl.io/docs/open/integration/frameworks/platformio.

[12] IEEE. *IEEE Standard for Information Technology–Telecommunications and Information Exchange between Systems–Local and Metropolitan Area Networks–Specific Requirements–Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*. 2021. doi: 10.1109/IEEESTD.2021.9363693.

[13] Sergio Saade. *Protocolos de comunicación TCP/IP*. San Miguel de Tucumán: EDUNT, 2016.

[14] S. Cheshire y M. Krochmal. *Multicast DNS*. RFC 6762. IETF, feb. de 2013. url: https://www.rfc-editor.org/rfc/rfc6762.

[15] R. Fielding, M. Nottingham y J. Reschke. *HTTP Semantics*. RFC 9110. IETF, jun. de 2022. url: https://www.rfc-editor.org/rfc/rfc9110.

[16] T. Bray. *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259. IETF, dic. de 2017. url: https://www.rfc-editor.org/rfc/rfc8259.

[17] N. Freed y N. Borenstein. *Multipurpose Internet Mail Extensions (MIME) Part Two: Media Types*. RFC 2046. IETF, nov. de 1996. url: https://www.rfc-editor.org/rfc/rfc2046.

[18] WHATWG. *HTML Living Standard, sección 17.2: multipart/x-mixed-replace*. url: https://html.spec.whatwg.org/multipage/iana.html#multipart/x-mixed-replace.

[19] Wikipedia. *Modelo–vista–controlador*. Última edición. 16 de jul. de 2026. url: https://es.wikipedia.org/wiki/Modelo%E2%80%93vista%E2%80%93controlador.

[20] MDN Web Docs. *HTML: Lenguaje de etiquetas de hipertexto*. url: https://developer.mozilla.org/es/docs/Web/HTML.

[21] MDN Web Docs. *CSS*. url: https://developer.mozilla.org/es/docs/Web/CSS.

[22] MDN Web Docs. *JavaScript*. url: https://developer.mozilla.org/es/docs/Web/JavaScript.

[23] Preact Contributors. *Preact Documentation*. url: https://preactjs.com/guide/v10/getting-started.
[24] National Institute of Standards and Technology. *Secure Hash Standard (SHS)*. FIPS PUB 180-4. Gaithersburg, MD: NIST, ago. de 2015. doi: 10.6028/NIST.FIPS.180-4.
[25] SQLite Consortium. *SQLite Documentation*. url: https://www.sqlite.org/docs.html.
[26] AI-Thinker. *ESP32-CAM Hardware Reference*. url: https://docs.zephyrproject.org/latest/boards/aithinker/esp32_cam/doc/index.html.
[27] Espressif Systems. *Página oficial del ESP32-S3*. url: https://www.espressif.com/en/products/socs/esp32-s3.
[28] Edge Impulse. *Página oficial de Edge Impulse*. url: https://docs.edgeimpulse.com/.
[29] Espressif Systems. *Arduino-ESP32 Documentation*. url: https://docs.espressif.com/projects/arduino-esp32/en/latest/.
[30] Google. *Página oficial de TensorFlow*. url: https://www.tensorflow.org/api_docs/python/tf/all_symbols?version=stable.
[31] Keras. *Página oficial de Keras*. url: https://keras.io/getting_started/about/.
[32] Google. *Página oficial de Google Colab*. url: https://colab.research.google.com/notebooks/welcome.ipynb.
[33] PlatformIO. *PlatformIO Documentation*. url: https://docs.platformio.org/en/latest/.
[34] Google. *Página oficial de MediaPipe*. url: https://developers.google.com/edge/mediapipe/solutions/guide.
[35] WebGazer. *Página oficial de WebGazer*. url: https://www.webgazer.io/docs.
[36] OpenCV Team. *Detección de características con OpenCV*. 31 de dic. de 2025. url: https://docs.opencv.org/4.13.0/dd/d1a/group__imgproc__feature.html#ga47849c3be0d0406ad3ca45db65a25d2d.
[37] OpenCV Team. *Círculos de Hough con OpenCV*. 31 de dic. de 2025. url: https://docs.opencv.org/4.13.0/da/d53/tutorial_py_houghcircles.html.
[38] OmniVision. *OV2640 Datasheet*. url: https://www.uctronics.com/download/cam_module/OV2640DS.pdf.
[39] Nextion. *ONX3248G035*. url: https://nextion.tech/wiki/onx3248g035/.
[40] UMW. *Datasheet TP4056*. url: https://www.umw-ic.com/static/pdf/095975b8ef7d5f40719d87d331ee6bc4.pdf.
[41] Alldatasheet. *Datasheet MT3608*. url: https://www.alldatasheet.com/datasheet-pdf/view/1131968/ETC1/MT3608.html.
[42] René Iván Reyes Indacochea. «Algoritmo de visión artificial para apoyar la terapia de lenguaje en niños con parálisis cerebral». Trabajo de grado, Ingeniería en Mecatrónica. Ibarra, Ecuador: Facultad de Ingeniería en Ciencias Aplicadas, Universidad Técnica del Norte, oct. de 2022. url: https://repositorio.utn.edu.ec/bitstream/123456789/13159/2/04%20MEC%20450%20TRABAJO%20DE%20GRADO.pdf.
[43] W. Fuhl et al. «ElSe: Ellipse Selection for Robust Pupil Detection in Real-World Environments». En: *Proceedings of the Ninth Biennial ACM Symposium on Eye Tracking Research & Applications (ETRA '16)*. Charleston, SC: ACM, 2016, págs. 123-130. doi: 10.1145/2857491.2857505.
[44] Santiago Juan Queipo de Llano Mazzuchelli. «Implementación de aplicaciones de Deep Learning en microcontroladores ARM Cortex M». Trabajo Fin de Grado. Madrid: E.T.S.I. y Sistemas de Telecomunicación, Universidad Politécnica de Madrid, jul. de 2021. url: https://oa.upm.es/71117/.